A Historical Review of the Gradient Descent Method

Yashwit Polapragada (A0249548M) — Dr. Eric Han

CP3108B — 20.04.2026

Contents

1 Preface **2**

2 Why Gradients? **3**
2.1 An Algebraic Problem . . . 3
2.2 A Geometric Solution . . . 3
2.3 The Computable Algorithm . . . 7

3 Simple Constraints **8**
3.1 The Landscape Guarantee . . . 8
3.2 The Minimization Guarantee . . . 10
3.3 The Convergence Guarantee . . . 12
3.4 Adaptive Convergence? . . . 15
3.4.1 Theoretical Analysis . . . 16
3.4.2 Experimental Analysis . . . 17

4 Stochasticity **20**
4.1 The Deep Learning Demand . . . 20
4.2 Stochastic Gradient Descent . . . 21
4.2.1 Convergence . . . 22
4.3 Stochastic Variance Reduced Gradient . . . 26

5 Acceleration **27**
5.1 Looking Back in Time . . . 27
5.2 Momentum . . . 27
5.2.1 Convergence . . . 28

6 Adaptive Gradient **31**
6.1 A History of Gradients . . . 31
6.2 RMSProp . . . 31
6.2.1 Convergence . . . 32
6.3 Adam . . . 34
6.3.1 Convergence . . . 35

7 Empirical Analysis **40**
7.1 Methodology . . . 40
7.1.1 Setup . . . 41
7.1.2 Hyperparameter Tuning . . . 41
7.1.3 Full Model Optimization . . . 42
7.1.4 Final Evaluation . . . 43
7.2 Results . . . 43
7.2.1 Empirical vs Theoretical Convergence . . . 43
7.2.2 Is Convergence Everything? . . . 44
7.2.3 Durability to Hyperparameter Variance . . . 45

8 Future Work **46**

9 Appendix: A1 **47**

10 Appendix: A2 **49**

11 References **53**

1 Preface

Gradient Descent is perhaps the most prominent optimization method in the Machine Learning domain, especially with the growing dominance of Deep Neural Networks. From the simple Steepest Descent algorithm that is taught in undergraduate courses to the cutting-edge Adam optimizer used in many industry contexts, Gradient Descent has only evolved but is never absent. Still, little and less is known by most about the exact motivations behind both the initial popularity and continued relevance of Gradient Descent within increasingly sophisticated ML optimization algorithms.

The purpose of this report is therefore to survey, both analytically and empirically, the development of the Gradient Descent method from its inauguration in 1847 as a novel method of solving nonlinear systems to its deployment in now highly sophisticated and yet difficult-to-interpret Deep Learning optimizers.

As has been already summarized in the Table of Contents, this report will:

1. First motivate the most fundamental form of the Gradient Descent method before inviting certain additional constraints and simple algorithmic enhancements that make the method pliable for further theoretical analysis.
2. Next illustrate the Deep Learning context, and induct the Gradient Method there through the variation known as Stochastic Gradient Descent.
3. Then pivot to the first of many major nontrivial optimizations of the Gradient Descent method through dissecting the Nesterov Accelerated Gradient.
4. Further briefly examine the first-principles behind modern Adaptive Gradient optimizers, in-particular RMSProp and Adam.
5. Lastly ground the theoretical discussion with some empirical testing, both to complement the finer points of the theoretical analyses as well as supplement it with nuance where lacking.

In general, this report is interested in the geometric convergence of Gradient Descent - and its many forms - in terms of the proximity of its iterates to the optimal solution, the rate of convergence itself, and the zeroing of the gradient magnitude.

I would like to thank Dr. Eric Han for his continued and invaluable guidance in all areas of this report, from structuring the overall discussion to improving the rigor of both empirical and theoretical analysis. Separately, I would also like to thank the NUS School of Computing for allowing the free use of the Compute Cluster for the training and evaluation of ML models, which ultimately supported the empirical analysis presented here.

Yashwit Polapragada
20.04.2026

2 Why Gradients?

2.1 An Algebraic Problem

Consider the following simple linear system[1]:

$$2x_1 - 11 = 0 \tag{1a}$$
$$x_1 + x_2 - 7 = 0 \tag{1b}$$

Solving this is relatively easy using the method of standard elimination (for example, subtracting $\frac{1}{2}$ of Eqn. **1a** from Eqn. **1b** to eliminate x_1 and solve for x_2). Even if we increase the number of equations or the number of unknown variables, we can employ the method of Gaussian Elimination (i.e. packing our coefficients, unknowns, and constants into a matrix and vertices and solving the single equation $\boldsymbol{A}x = b$) to better organize our solution.

However, Gaussian Elimination becomes quickly computationally expensive due to the repeated scans over all equations and unknowns when performing forward-elimination. Not to mention, it is immediately outmoded when up against a non-linear system such as:

$$x_1^2 + x_1x_2 - 11 = 0 \tag{2a}$$
$$x_1 + x_2^2 - 7 = 0 \tag{2b}$$

The square-terms complicate the system, as does the shearing applied by the x_1x_2-term. Absent Gaussian Elimination as an option, we may then turn to the method of algebraic substitution. For instance, we could substitute the x_1 term in **2a** with $7 - x_2^2$ (from **2b**) and obtain $(7 - x_2^2) + (7 - x_2^2)x_2 - 11 = 0$. Notice that we already have a more complicated equation, with our polynomial graduating from a 2nd-degree to 3rd (with the new $-x_2^3$)-term). One can imagine how this may escalate rapidly in complexity, with more equations, more unknowns, or higher-degree polynomials.

What if instead we were to be able to translate a system of equations to 1-singular expression, and then use a simple method to solve this expression? Consider, as an example, the below process:

1. Condense the system above to the sum-of-squares expression $(2a)^2 + (2b)^2$ i.e. $(x_1^2 + x_1x_x - 11)^2 + (x_1 + x_2^2 - 7)^2$.[2]
2. Since the square of a term can never be negative, the minimum of the equation can only be 0.
3. Thus, identifying the x_1, x_2 that minimize the above expression is equivalent to solving the original nonlinear system. Explicitly, finding the x_1, x_2 that sets $2a = 2b = 0$ and hence sets $(2a)^2 + (2b)^2 = 0$, also happens to solve the original system.

While we've simplified the system, how to easily minimize the simplified expression $(2a)^2 + (2b)^2$?

2.2 A Geometric Solution

In 1847, Augustin Louis Cauchy proposed the **Gradient Method**: a means of solving systems of equations in multiple variables that was incredibly efficient compared to traditional methods (Lemaréchal, 2012). The method, in short, transforms the approach of solving multivariable equations from a tedious algebraic exercise, to an intuitive and highly versatile, geometric one.

The Gradient Method is motivated as such: if we knew what the objective was that we were trying to maximize or minimize, and we could represent it geometrically, then we could use information pertaining to the *landscape* of the geometric object in-order to deduce its **extrema** (i.e. minimum or maximum).

Let's view the $(2a)^2 + (2b)^2$ function from earlier as a geometric object:

As can be seen, the landscape is contoured by crests, troughs, and other geographical quirks. In essence, our **objective** is to navigate this complex 3D terrain from some arbitrary starting point (in this case, some $x_1 = \alpha, x_2 = \beta$), and eventually find the minimal (lowest) point of the landscape.

[1] In general, we shall talk about vectors when referring to points x instead of scalars, although some exceptions will be explicitly made for notational ease.

[2] This is in-fact a modified Himmelblau's Function - a multimodal function designed to be an adversary for optimization algorithms.

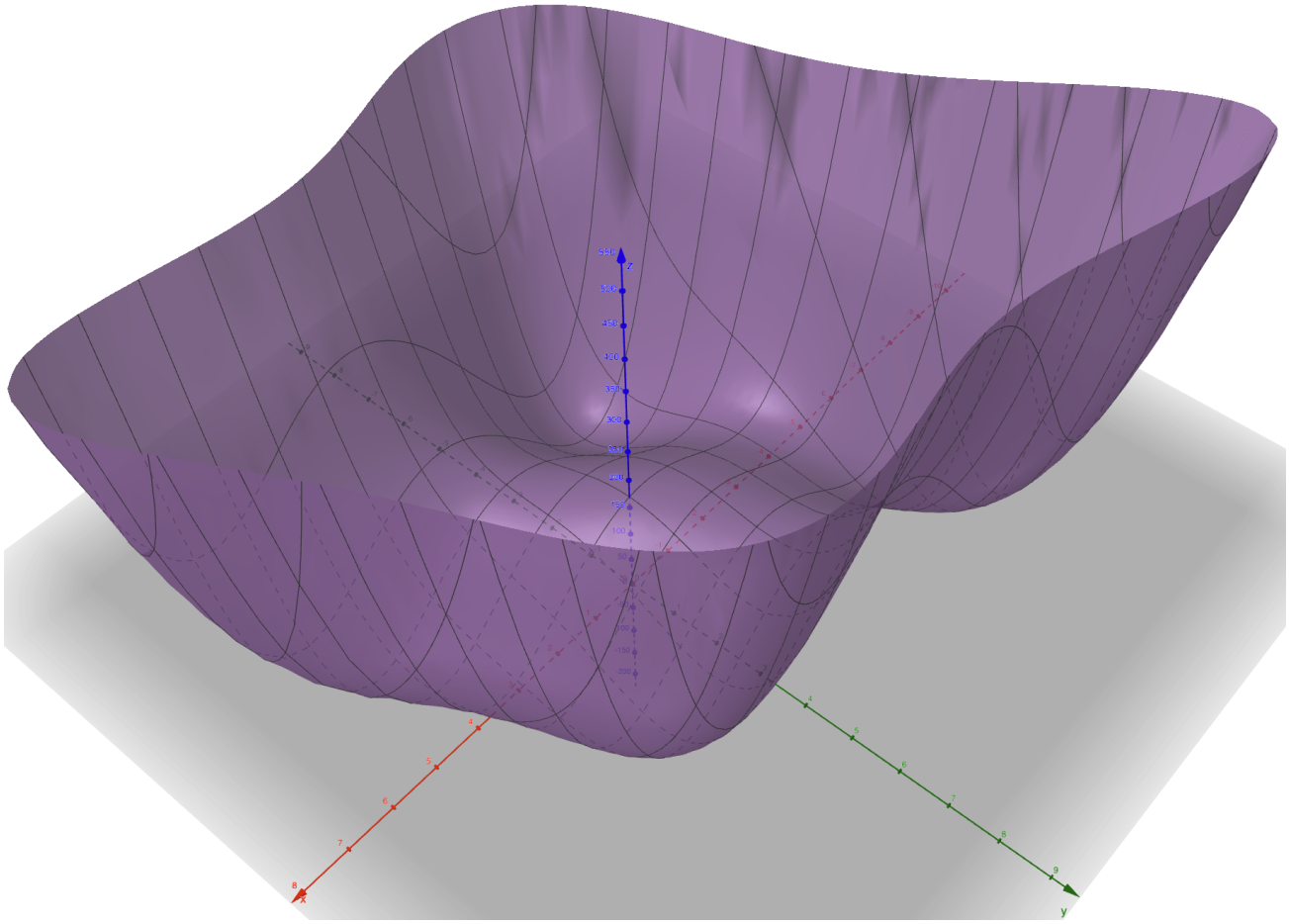


Figure 1: The Modified Himmelblau Function, $(x_1^2 + x_1 x_x - 11)^2 + (x_1 + x_2^2 - 7)^2$

More generally, for an objective function $f : \mathbb{R}^n \to \mathbb{R}$ (where n indicates the number of unknowns to maximize, or more formally the dimensionality of $\mathbf{dom}(f)$[3]), the Gradient Method aims to find a maximal or minimal point $x_* \in \mathbf{dom}(f)$[4] such that $f(x') \geq f(x_*)$ - when x_* is a maxima - or $f(x') \leq f(x_*)$ - when x_* is a minima - for all $x' \in \mathbf{dom}(f)$ in the vicinity of x_*. In the case of our example, $f(x_1, x_2) = (x_1^2 + x_1 x_x - 11)^2 + (x_1 + x_2^2 - 7)^2$ and the minimal $f(x_1, x_2) = 0$. For the rest of this paper, let us focus on the minimization objective instead of maximization. When we constrain our focus as such, the Gradient Method may be renamed to the more popularly known: **Gradient Descent**.

Importantly, since we start from some arbitrary point $x_1 \in \mathbf{dom}(f)$[5] with no information on the location of the minimal point x_*, we can only use the information available at each x_t to <u>get closer to</u> x_*(i.e. the point at which a change in **any** dimension/direction $(x_1)_*, ..., (x_n)_*$ would only result in $f(x_{t+1}) > f(x_t)$. Ideally, we would locate x_* from x_1 within some finite and reasonable number of steps $x_1, x_2, ..., x_t$. That is, if we set a limit T to the number of points we visit after x_1, we should hope that after reaching the final point x_t in step T, we have either already found x_* and minimized $f(x)$ or are at least very close to it.

So, what information exactly is available at each step x_t? Intuitively, each possible choice of $x_1, x_2, ..., x_t$ may cause the objective f to either increase or decrease (i.e. trend away or towards the ideal point x_* in the geometric landscape). We simply need to know how x_t is changing f, and update it such that its new position lies in the direction that f is decreasing. More clearly, if $x_t = [x_1; x_2]_t$, and if the **tangent** plane at $(x_1)_t$ is tilting downwards (i.e. indicating a decreasing f), then we want our $(x_1)_t$ to *increase* in that direction. Equally, if the tangent plane at $(x_2)_t$ is tilting upwards (i.e. indicating an increasing f), then we want our $(x_2)_t$ to *decrease* against that direction. Refer to Figure 2 for a fuller illustration of this idea.

In fact, there exists a very instrumental mathematical tool that formalizes this simple idea behind finding x_*: the **Taylor Series**. Devised by Brook Taylor, The Taylor Series is a way to approximate an unknown point $f(x')$ using a known point

[3]In this case, we treat the input to f, $x \in \mathbb{R}^n$ as an n-dimensional vector, representing a point in f's landscape.

[4]Similar to the above note, x_* in this case is an n-dimensional point $[x_1; x_2; ...; x_n]_*$, which happens to be an extrema in the landscape.

[5]In general, we may use x_t to represent an n-dimensional vector at step t of the process of discovering the extrema x_*. And we use $(x_i)_t$ to denote the ith-dimensional coordinate of the vector x at step t.

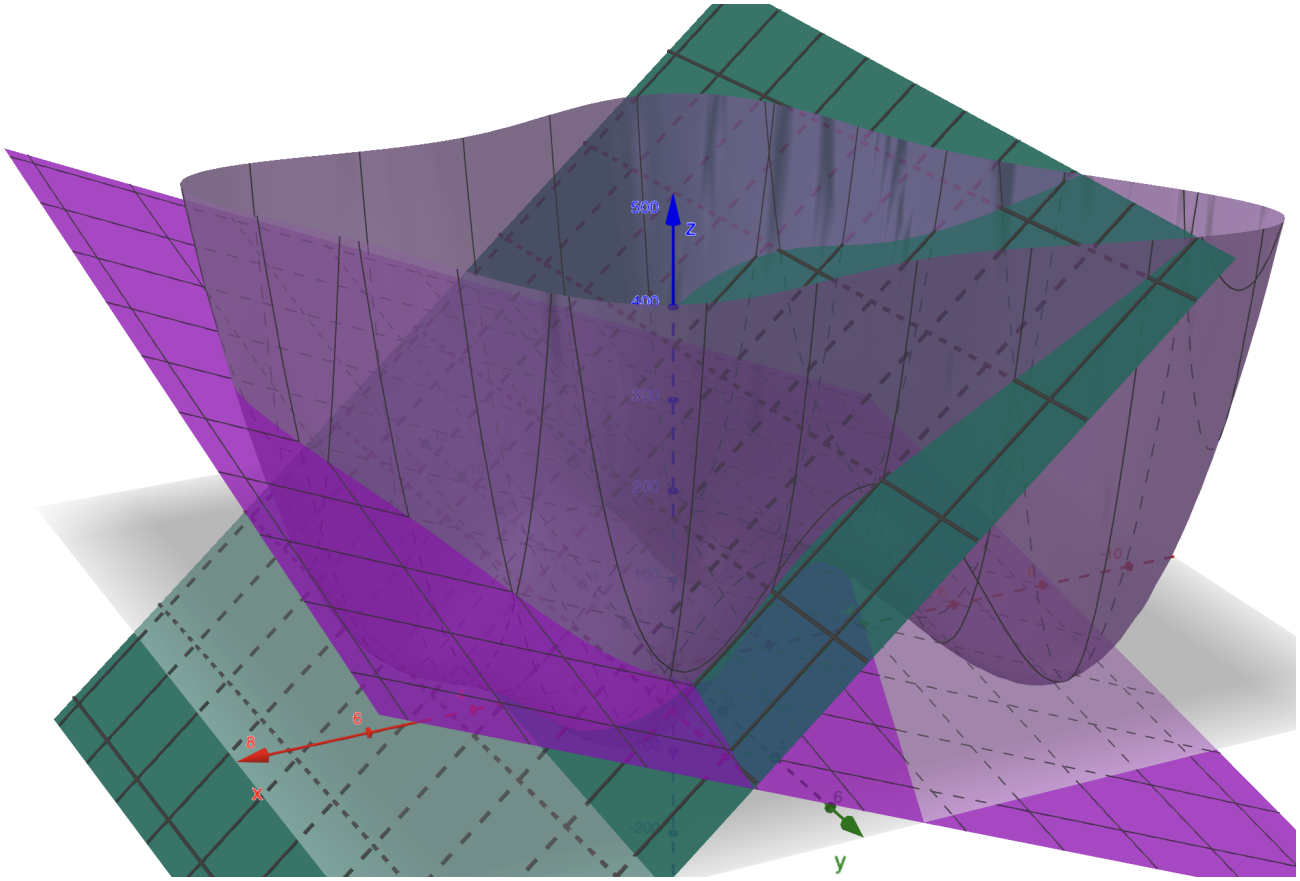


Figure 2: Example tangent planes at point x_t. The purple tangent plane represents the direction of change for $(x_1)_t$ with regards to f (downwards), while the green one is that of $(x_2)_t$ (upwards).

$f(x)$ where $x, x' \in \mathbf{dom}(f)$ by making the *assumption* that the landscape local to $f(x)$ is reliable in estimating any point $f(x')$ in the global landscape. More specifically, the Taylor Series generates an approximation of how much $f(x)$ and $f(x')$ will differ, by taking the displacement of the points $x' - x$ and augmenting this with information on the **rate of change** of f (slope, curvature, etc.). Consider the following **1st-Order** Taylor Expansion[6]:

$$f(x') \approx f(x) + f'(x)(x' - x) \tag{3}$$

where the notation $f'(x)$ denotes the 1st-Order derivative of $f(x)$, or when in our case dealing with vectors,

$$f(x') \approx f(x) + \langle \nabla f(x), x' - x \rangle \tag{4}$$

wherein the notation $\nabla f(x)$ is a vector of partial derivatives $[\frac{\partial f}{\partial x_1}; \frac{\partial f}{\partial x_2}, ..., \frac{\partial f}{\partial x_n}]$[7], and the notation $\langle x_1, x_2 \rangle$ represents the inner product of 2 vectors (or multidimensional points) which is a graduation of the simple scalar multiplication.

In the case of our objective, the known point x is the point x_t in the current step, and the unknown point x' is the point x_{t+1} in the following step. In this way, we easily induce an application of the Taylor Expansion in *Eqn. 4* to our objective.

The observant reader will notice, however, that while *Eqn. 4* approximates $f(x_{t+1})$ based on having only $x, x_t, f(x_t), f'(x_t)$, Gradient Descent demands a bit more. We need to find an x_{t+1} based on $x_t, f(x_t), f'(x_t)$ for which $f(x_{t+1}) \leq f(x_t)$ for any $x_{t+1} \in \mathbf{dom(f)}$. Therefore, it seems that the *Eqn. 4* will have to be manipulated further for us to derive from it an actual update rule for x_t that leads to an x_{t+1} for which we know that the objective is being minimized. Fortunately, the idea is the same: minimize *Eqn. 4* with respect to x_t and solve for an x_{t+1} that satisfies this minimization!

..... × .....

Derivation 1. *Start with the 2nd-Order (Quadratic) Taylor Expansion where in this case the point to be approximated is x_{t+1} and the current point is x_t.*

$$f(x_{t+1}) \approx f(x_t) + \langle \nabla f(x), x_{t+1} - x_t \rangle + \frac{1}{2} \langle x_{t+1} - x_t, \nabla^2 f(x_t)(x_{t+1} - x_t) \rangle$$

Setting the 2nd Derivative term $\nabla^2 f(x_t) = \gamma I$ where γ is an assumed positive constant curvature term and I is the identity matrix, we obtain the simplified 2nd Order approximation:

$$f(x_{t+1}) \approx f(x_t) + \langle \nabla f(x), x_{t+1} - x_t \rangle + \frac{\gamma}{2} ||x_{t+1} - x_t||_2^2$$

where $||x_{t+1} - x_t||_2^2$ is the squared euclidean norm of $x_{t+1} - x_t$, induced by the inner product of $x_{t+1} - x_t$ with itself. Then, consider $f'(x_{t+1})$ w.r.t x. That is, the minimization objective of the simplified approximation:

$$f'(x_{t+1}) \approx \nabla f(x) + \gamma(x_{t+1} - x_t)$$

Setting $f'(x_{t+1})$ to 0 (since we're solving for the minimum x_{t+1}) and rearranging,

$$x_{t+1} = x_t - \frac{1}{\gamma}(\nabla f(x))$$

Setting $\eta = \frac{1}{\gamma}$, we have the Gradient Method Update Rule:

$$x_{t+1} = x_t - \eta(\nabla f(x))$$

..... × .....

This derivation encapsulates the original simple idea to observe how f at the current point x_t is behaving, and identify an x_{t+1} that lies in the appropriate direction of how we want f to change. Intuitively, we would want to identify an x_{t+1} that is as far in the right direction from x_t as can be (to reach the optimal point x_*) faster. So, we should be minimizing the 1st-Order Expansion to solve for the smallest possible x_{t+1} (i.e. one for which the approximation $f(x_{t+1}) = 0$).

So why start from the 2nd-Order (Quadratic) Expansion? Simply, the linear approximation is unbounded in its rate of change. The "minimum" x_{t+1} could very well be at $-\infty$ along the tangent at x_t. While we could introduce an arbitrary

[6] The full Taylor Series is actually infinite, increasing in accuracy of approximating $f(x')$ as more specific information about the rate of change of f at x is used. The "1st"-Order in the term thus refers to approximating $f'(x)$ using only the 1st-derivative (i.e. slope) of f at x. In essence, this is a linear approximation, which assumes that f will only increase or decrease indefinitely.

[7] When the objective f is high-dimensional (influenced by multiple unknowns $x_1, x_2, ..., x_n$), it is generally more tractable to compute the partial derivatives of f at each point, instead of the full derivative

constraint on x_{t+1}, the quadratic (2nd-Order) term in the Taylor Series gives us a dynamic bound. Since the quadratic term factors in the curvature of f when approximating x_{t+1}, we simply need to fix some positive curvature to ensure that even as we cascade rapidly along the tangent at x_t, we will eventually stop at some point and start "curving" back around. Hence, we include a $\frac{\gamma}{2}||x_{t+1} - x_t||^2$ term in the minimization objective which acts as a proximity term (a "leash"), where the fixed positive curvature is γ. Therefore, we identify an x_{t+1} that is as "low" as it can be along the linear plane from the point x_t without breaching the domain constraints of f (i.e. that $x_{t+1} \geq 0$).

2.3 The Computable Algorithm

In Algorithm 1 is finally explicated the basic unconstrained Gradient Descent algorithm. The algorithm assumes we have an objective f to minimize, some starting point x^0 (where $\mathbf{dom}(f) \subseteq \mathbb{R}^n$), a step-size η, and some finite number of steps T.

Algorithm 1 Unconstrained Gradient Descent

Input: f, x_0, η, T
Output: x_*

1: $last_point = x_0$
2: **for** $t := 1$ to T **do**
3: $\quad last_point = x_t$
4: $\quad x_{t+1} = x_t - \eta(\nabla f(x_t))$
5: **end for**

6: $x_T = last_point$
7: return x_T

As has been explained more thoroughly in the previous section, this algorithm aims to find a sequence of $x_1, x_2, ..., x_t$ such that f is decreasing at every step and ideally eventually bottoms out at some minimal x_*. The update rule in *Step 4* is the same as the one derived from the minimization of the 1st-Order Taylor Approximation of the unknown $f(x_{t+1})$ around the known point x_t. This is also known as the **Steepest Descent** algorithm because at every step, it attempts to locate an x_{t+1} that minimizes the linear approximation of f as much as possible ("steepest" along the tangent plane at x_t) compared to x_t, while still respecting the domain constraints of f. Then, η in this case, is called a "step-size" because it is a **hyperparameter** that controls the rate of descent. As mentioned, if the descent is too steep, the algorithm may overshoot the minimum x_*.

3 Simple Constraints

While *Algorithm 1* in the previous chapter was mostly clear, there are still 3 main areas of concern.

3.1 The Landscape Guarantee

Algorithm 1 uses the critical Gradient Descent update rule $x_{t+1} = x_t - \eta \nabla f(x_t)$, which was itself derived from the 2nd-order Taylor Expansion with fixed curvature $\nabla^2 f(x_t) = \gamma$. However, the silent compromise in using this expansion was that it is merely an approximation. Indeed, a function f at point x' is only equal to its Taylor Expansion at a point x when the expansion is expressed as an infinite power series:

$$f(x') = f(x) + \langle \nabla f(x), x' - x \rangle + \frac{1}{2}\langle x' - x, \nabla^2 f(x)(x' - x)\rangle + ... = \sum_{n=0}^{\infty} \frac{f^n(x)}{n!}(x' - x)^n. \tag{5}$$

So, given that we use only the truncation

$$f(x') \approx f(x) + \langle \nabla f(x), x' - x \rangle + \frac{1}{2}\langle x' - x, \nabla^2 f(x)(x' - x)\rangle$$

how do we account for inevitable error in approximation? Fortunately, Taylor bails us out again.

Theorem 1 (Taylor's Theorem)**.** *Let $k \geq 1$ be an integer and let the function $f : \mathbb{R} \to \mathbb{R}$ be k-times differentiable at the point $x \in \mathbb{R}$. Then there exists a function $h_k : \mathbb{R} \to \mathbb{R}$ such that:*

$$f(x') = \sum_{i=0}^{k} \frac{f^i(x)}{i!}(x' - x)^i + h_k(x')(x' - x)^k$$

and $\lim_{x' \to x} h_k(x') = 0$

The key term $h_k(x')(x' - x)^k$ appended to the Taylor approximation (to turn it into an equality) is the **Peano** error term[8]. The concept behind the Peano Term (dubbed as $R_k(x')$, to mean "Remainder" term), is that it asymptotically bounds the approximation error of the truncated kth-order Taylor Expansion. For our 2nd-order Taylor Approximation, the error term collected would thus be:

$$R_2(x') = h_2(x')(x' - x)^2$$

The nice property of this error term is that $\lim_{x' \to x} h_2(x') = 0$, which means that as x' increases in proximity to x, the error term h_2 shrinks to 0 sooner than the proximity term $(x' - x)^2$. In general,

$$R_k(x) = h_k(x')(x' - x)^k = o((x' - x)^k)$$

That is, as we add more terms to the Taylor Expansion (i.e. increase k), it becomes a better local model for f with the error R_k vanishing increasingly faster. The error term scales only by the actual proximity term $(x' - x)^k$ instead of any arbitrary added multiple h_k.

Now that we know how to asymptotically bound the error term in an approximated Taylor Expansion, how do we actually **minimize** it? As mentioned, one possible way is to continuously expand the Taylor Expansion to higher-orders until we are satisfied with the degree of accuracy. However, this quickly becomes computationally infeasible for complex f. Instead, let us redirect our focus to f itself and introduce the first domain constraint we shall require for f: $L-$**smoothness**.

Definition 1 (L-Smoothness)**.** *A differentiable function $f : X \to \mathbb{R}$ is called **L-smooth** if its gradient is L-Lipschitz Continuous. That is,*

$$||\nabla f(x) - \nabla f(x')||_2 \leq L||x - x'||_2 \;\; \forall x_1, x_2 \in X$$

Plainly, L-Smoothness says that there is a way to bound the fluctuation in change in gradients between two points as a function of the distance between the underlying points themselves. In particular, bounded by some constant L times of the

[8] Named after Italian mathematician Giuseppe Peano.

distance between the points. It should be intuitive why this is beneficial to an algorithm such as Gradient Descent, that relies on a local estimator informed by the gradient (namely, the Taylor Expansion) in approximating a next point. If the gradient of f can be assured to not change by too much, then we may be assured that we can continue performing steepest descent, and perhaps locate x_*.

Now, we introduce an important bound on L-smooth functions by borrowing and expanding on some of the working from Farina, 2025's lecture:

Theorem 2 (L-Smooth Function Bound)**.** *A differentiable, L-smooth function $f : X \to \mathbb{R}$ over a convex domain X, admits the quadratic upper bound:*

$$f(x') \leq f(x) + \langle \nabla f(x), x' - x \rangle + \frac{L}{2}||x' - x||_2^2$$

$\forall x', x \in X$

Proof.

$$\begin{aligned}
f(x') - f(x) &= \int_0^1 \langle \nabla f(x + t(x' - x)), x' - x \rangle \, dt \\
&\quad \textit{(Fundamental Thm. of Calculus)} \\
&= \int_0^1 \Big\langle \nabla f(x + t(x' - x)) - \nabla f(x) + \nabla f(x), x' - x \Big\rangle \, dt \\
&= \int_0^1 \langle \nabla f(x + t(x' - x)) - \nabla f(x), x' - x \rangle \ + \langle \nabla f(x), x' - x \rangle \, dt \\
&\quad \textit{(Linearity of Inner Product)} \\
&= \int_0^1 \langle \nabla f(x + t(x' - x)) - \nabla f(x), x' - x \rangle \, dt + \int_0^1 \langle \nabla f(x), x' - x \rangle \, dt \\
&= \int_0^1 \langle \nabla f(x + t(x' - x)) - \nabla f(x), x' - x \rangle \, dt + \langle \nabla f(x), x' - x \rangle \\
&\leq \int_0^1 ||\nabla f(x + t(x' - x)) - \nabla f(x)||_2 \cdot ||x' - x||_2 \, dt + \langle \nabla f(x), x' - x \rangle \\
&\quad \textit{(Cauchy-Schwarz Inequality)} \\
&\leq \int_0^1 tL||x' - x||_2^2 \, dt + \langle \nabla f(x), x' - x \rangle \\
&\quad \textit{(f is assumed L-smooth)} \\
&= \frac{L}{2}||x' - x||_2^2 + \langle \nabla f(x), x' - x \rangle
\end{aligned}$$

So, finally we get

$$f(x') - f(x) \leq \frac{L}{2}||x' - x||_2^2 + \langle \nabla f(x), x' - x \rangle$$

which is equivalent to

$$f(x') \leq f(x) + \langle \nabla f(x), x' - x \rangle + \frac{L}{2}||x' - x||_2^2$$

which is the statement of the theorem. □

. . . . . × . . . . . .

Derivation 2. *Recalling the 2nd-Order Taylor-Expansion with error R_2 from earlier,*

$$f(x') = f(x) + \langle \nabla f(x), x' - x \rangle + \frac{1}{2}\langle x' - x, \nabla^2 f(x)(x' - x) \rangle + o((x' - x)^2)$$

Now, with an L-smooth f, we have:

$$\begin{aligned} f(x') &= f(x) + \langle \nabla f(x), x' - x \rangle + \frac{1}{2}\langle x' - x, \nabla^2 f(x)(x' - x)\rangle + o((x' - x)^2) \\ &\leq f(x) + \langle \nabla f(x), x' - x \rangle + \frac{L}{2}||x' - x||_2^2 \end{aligned}$$

That is to say,

$$\frac{1}{2}\langle x' - x, \nabla^2 f(x)(x' - x)\rangle + o((x' - x)^2) \leq \frac{L}{2}||x' - x||_2^2$$

And thus we have the bound,

$$f(x') \leq f(x) + \langle \nabla f(x), x' - x \rangle + \frac{L}{2}||x' - x||_2^2$$

..... × .....

Recall now the approximation from *Derivation 1* with fixed curvature γ,

$$f(x') \approx f(x) + \langle \nabla f(x), x' - x \rangle + \frac{\gamma}{2}||x' - x||_2^2$$

Basically, not only have we bounded the 2nd-Order term **together with the error term** by eliminating the unknown $\nabla^2 f(x)$ curvature term, the once arbitrarily chosen fixed curvature γ now becomes the Lipschitz Contintuity of the gradient of f, which is L! The implication of this will be addressed in a following section on Convergence Guarantee.

In summary, the intuition is that if we require that the objective f be L-smooth, we can:

- Allow ourselves to use the truncated 2nd-Order Taylor Expansion to derive the Gradient Descent update rule without needing to include an unbounded error term. This is done by eliminating uncertainty from the unknown curvature $\nabla^2 f(x)$, allowing us to safely use the 1st-Order term $f(x) + \langle \nabla f(x), x' - x \rangle$ to approximate $f(x')$.
- Use a meaningful fixed curvature L instead of unknown γ.

3.2 The Minimization Guarantee

Until now, we have been assuming that the last point x_t in our sequence of descent is truly the minimizer of the objective f, based on the fact that the gradient of f at x_t is 0 (which, as mentioned, signals that a change in x_t in any direction would result in an increase in f). However, gradients are blind to the concept of **local** and **global** minima. Consider Figure 3, as a simple 2D example of this phenomenon.

If we are lucky, and our arbitrary starting point $x_1 = 1.012$, then we may still identify the minimum $x_* = 2.10646, f(x_*) = 0.45112$ as we descend along f from 0.58047 to 2.10646. However, what if $x_1 = 4.157$? The algorithm would identify the steepest descent to be toward $x = 5.19614$ where $f(x_t) = 2.74533$. Technically, this is not an invalid result since at this x_t, there is no direction in which f is still decreasing, and yet we have not truly minimized f since at the point $x_* = 2.10646$, f is lower.

In this scenario, the point $x = 2.10646$ is called a **global** minima while $x = 5.19614$ is called a **local** minima, and an objective with both local and global minima, cannot be minimized deterministically by Gradient Descent (since a lot depends on how lucky we get with the choice of x_1).

Fortunately, there exists a set of function for which any local minimum is also a global minimum. Formally, these are called **convex** functions, and are defined as:

Definition 2 (Convex Function)**.** *A real-valued function $f : X \to \mathbb{R}$ is called **convex** if, for all $x_1, x_2 \in X$ and $\lambda \in [0, 1]$, the following inequality holds:*

$$f(\lambda x_1 + (1 - \lambda)x_2) \leq \lambda f(x_1) + (1 - \lambda)f(x_2)$$

In plain terms, for any two points in $\mathbf{dom}(f)$ (which denotes X in the above definition), the line connecting the 2 points $\lambda f(x_1) + (1 - \lambda f(x_2))$[9] lies entirely above f at $\lambda x_1 + (1 - \lambda x_2)$ over the interval $\lambda \in [0, 1]$. Figure 4 depicts how the negative example f depicted in Figure 3 fails in fulfilling this property.

[9]The expression of this line can also be rewritten as $f(x_1) + \lambda(f(x_2) - f(x_1))$. This formulation may be more intuitive if one defines the line connecting x_1 and x_2 as a collection of points generated by the expression $\lambda(x_2 - x_1)$, and parameterized by the interval $\lambda \in [0, 1]$.

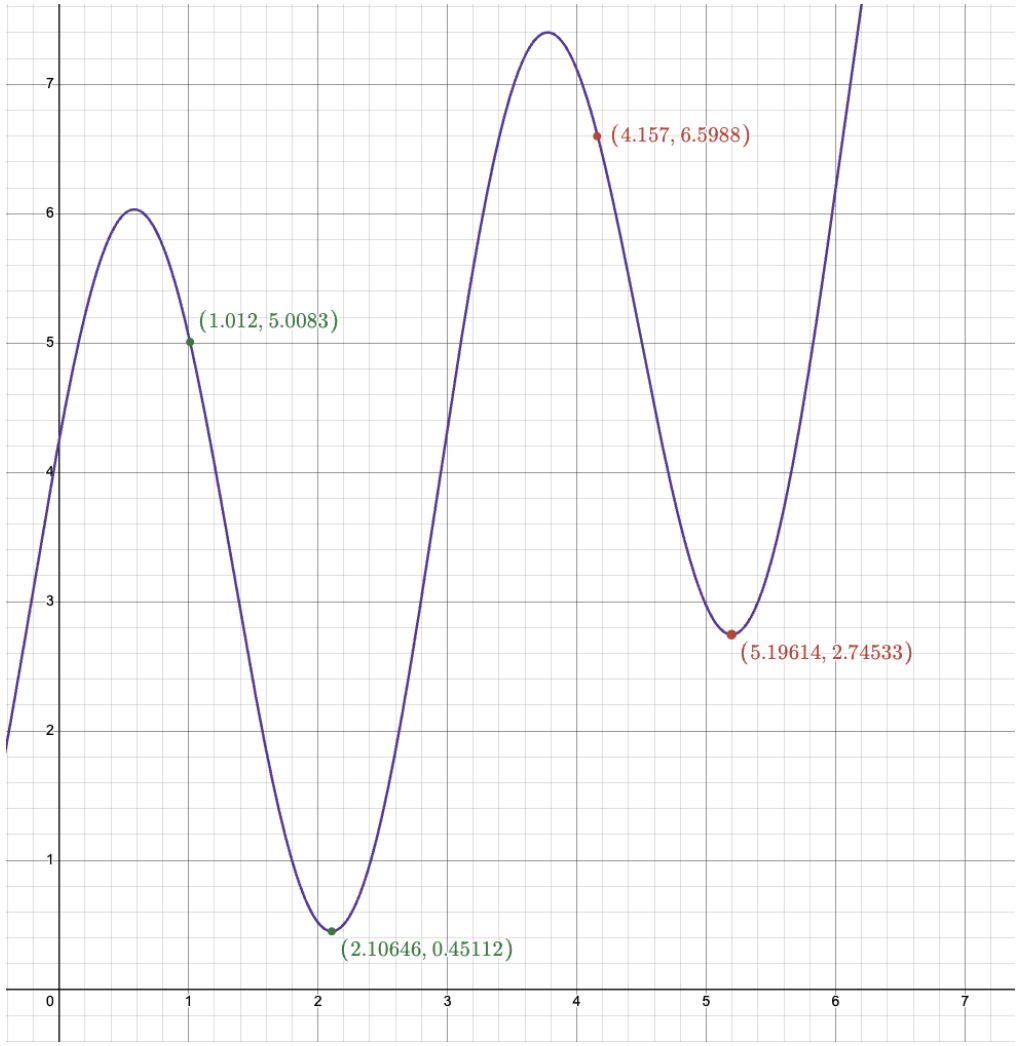


Figure 3: A snapshot of the function $0.1x^2 - 3cos(2x+2) + 3$ which experiences a 0 gradient (a "minima") at more than 1 point.

Having defined convexity, we can prove a useful theorem:

Theorem 3 (Convex Function Landscape)**.** *For a convex function f, if $x_1 \in$ **dom**(f) is a local minima, it is also a global minima for f.*

Proof. Suppose, by way of contradiction, that x_1 is a local but not global minima. By definition, this means that there exists some global minima $x_2 \in \mathbf{dom}(f), x_1 \neq x_2$ for which $f(x_2) < f(x_1)$

Then,

$$\begin{aligned} f(\lambda x_1 + (1-\lambda)x_2) &\leq \lambda f(x_1) + (1-\lambda)f(x_2) \ (\textit{since } f \textit{ is convex}) \\ &< \lambda f(x_1) + (1-\lambda)f(x_1) \ (\textit{by hypothesis}) \\ &= f(x_1) \end{aligned}$$

Thus, we have that

$$\lambda f(x_1) + (1-\lambda)f(x_2) < f(x_1) \text{ for } \lambda \in [0,1]$$

If we pick some arbitrary λ, say 0.5, we get a point $0.5f(x_1) + 0.5f(x_2) = x_3$ near x_1 for which $f(x_3) < f(x_1)$. Proving that x_1 is in-fact not a local minima - a contradiction! $\square$

Using *Theorem 3*, we extricate:

Corollary 1 (Minimization Guarantee)**.** *If f is convex, and Gradient Descent terminates with a local minima x_t for which $\nabla f(x_t) \equiv 0$, then it has found a global minima x_* to minimize f.*

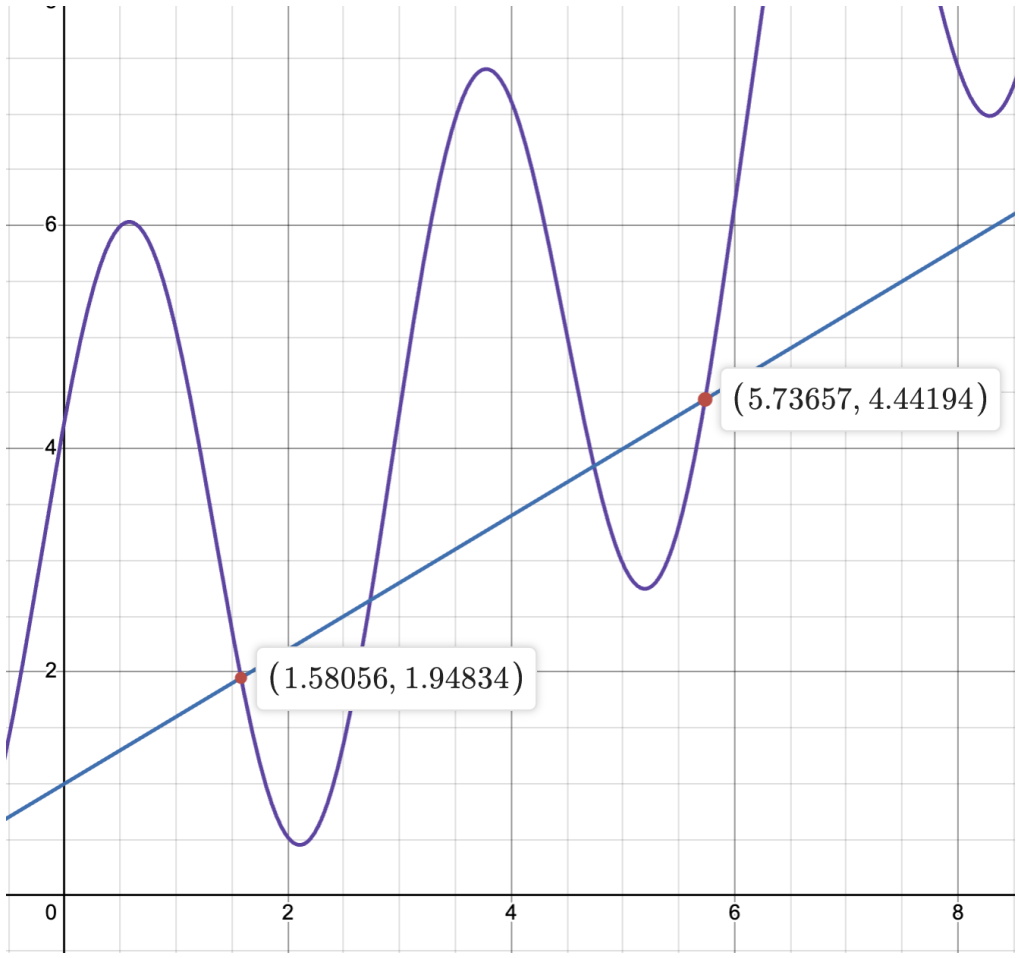


Figure 4: The line connecting $x_1 = 1.58065$ and $x_2 = 5.73657$ lies below the 2nd crest i.e. f between $x_1^{'} = 2.73805$ and $x_2^{'} = 4.74114$
.

Ultimately, we introduce our 2nd constraint to ensure the correctness of Algorithm 1 in minimizing a given objective f, which is that f has to be convex.

3.3 The Convergence Guarantee

From the previous section, *Theorem 3* guarantees that <u>if Gradient Descent converges at a local minima x_t</u>, then it has found the global minimizer to f. The noticeable assumption here is that the algorithm converges to a minima at all.

As mentioned in Chapter 1, Steepest Descent aims to minimize f as quickly as possible while being corrected against domain violations by the fixed curvature $\frac{1}{\gamma}$ which eventually becomes the step-size hyperparameter η. But, there was no mention of how to select γ or in its later form, η.

A poor choice of η can lead to poor, or no convergence on the minima. In this case, "poor convergence" is usually defined as the distance between the algorithm's final candidate x_t and the true minima x_* being out of the bounds of some reasonable expectation. Consider Figure 5, for a demonstration of the consequence of poor vs good choices of step-size on the convergence ability of Gradient Descent[10].

As one would expect however, we must do better than hoping by luck to choose a η that produces a good convergence. Moreover, there still requires a bound to indicate what exactly a "good" convergence is. For this, we take inspiration from and expand on working from Tibshirani, 2015's lecture.

[10]The code that produced this figure can be found here.

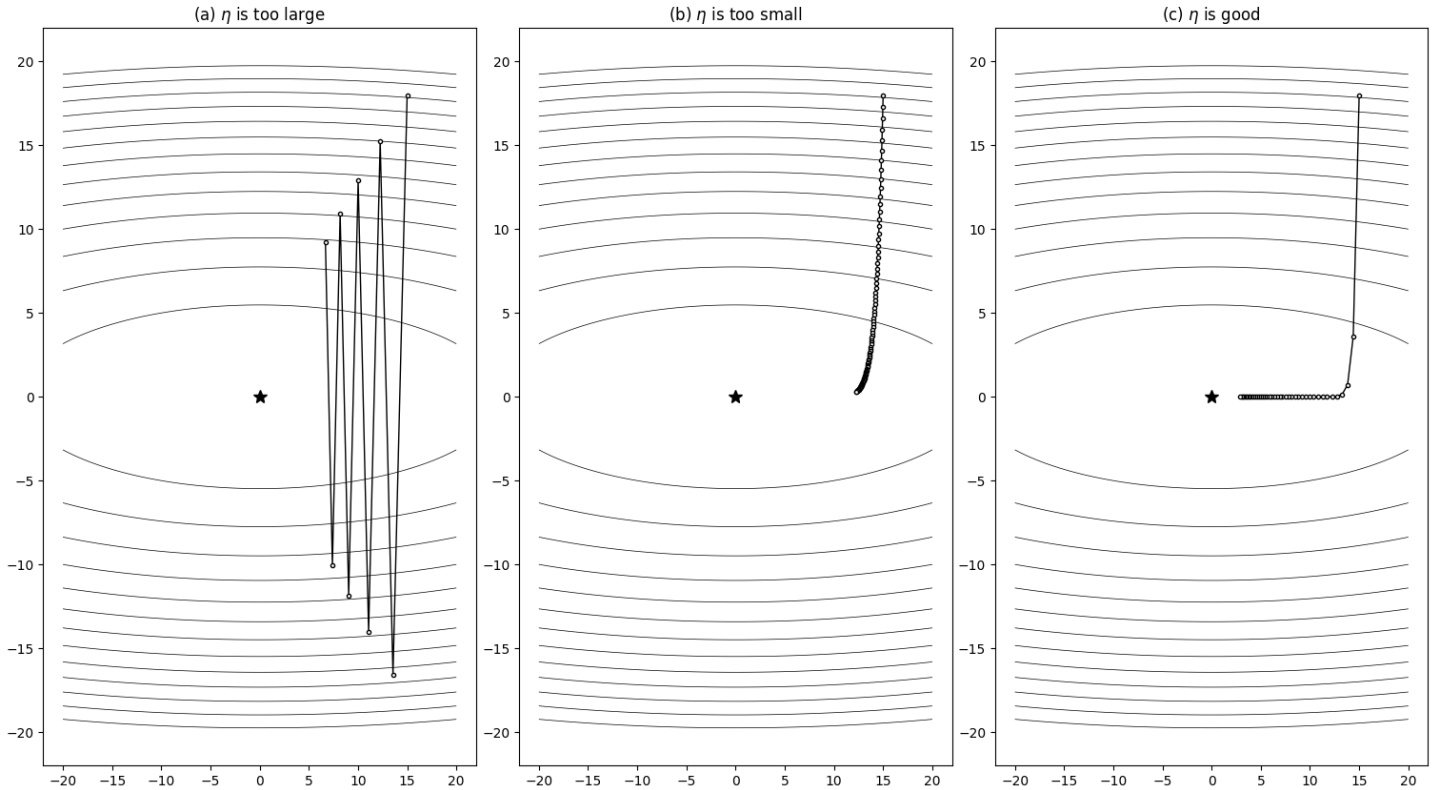


Figure 5: 3 possible convergences.

Theorem 4 (Fixed-η Descent Final Iterate Convergence Bound)**.** *If the objective f is convex, with Lipschitz gradient at $L > 0$, then Gradient Descent with fixed $\eta = \frac{1}{L}$ satisfies the convergence bound:*

$$f(x_t) - f(x^*) \leq \frac{||x_1 - x_*||_2^2}{2\eta T}$$

Proof. Since ∇f is Lipschitz continuous

$$\forall x_t, x_{t+1} \in \mathbf{dom}(f), f(x_{t+1}) \leq f(x_t) + \langle \nabla f(x_t)_t, x_{t+1} - x_t \rangle + \frac{L}{2}||x_{t+1} - x_t||_2^2$$

Knowing that $x_{t+1} = x_t - \eta(\nabla f(x_t))$ from the Gradient Descent update rule, we can simplify the above inequality as such:

$$\begin{aligned} f(x_{t+1}) &\leq f(x_t) + \langle \nabla f(x_t), -\eta \nabla f(x_t) \rangle + \frac{L}{2}\eta^2 ||\nabla f(x_t)||_2^2 \\ &= f(x_t) - \eta ||\nabla f(x_t)||_2^2 + \frac{L\eta^2}{2}||\nabla f(x_t)||_2^2 \\ &= f(x_t) - (1 - \frac{L\eta}{2})\eta ||\nabla f(x_t)||_2^2 \end{aligned}$$

By inspection, $(1 - \frac{L\eta}{2})\eta \geq \frac{\eta}{2}$ for $\eta \in (0, \frac{1}{L}]$. Thus,

$$\begin{aligned} f(x_{t+1}) &\leq f(x_t) - \frac{\eta}{2}||\nabla f(x_t)||_2^2 \\ &= f(x_t) - \frac{\eta}{2}||\frac{x_t - x_{t+1}}{\eta}||_2^2 \text{ (by rearranging the update rule)} \\ &= f(x_t) - \frac{1}{2\eta}||x_t - x_{t+1}||_2^2 \end{aligned}$$

Since f is convex, the linear approximation of the optimal point $f(x_*)$[11] is a lower bound for the actual point itself:

$$\begin{aligned} f(x_*) &\geq f(x_t) + \langle \nabla f(x_t), x_* - x_t \rangle \\ &\iff f(x_t) \leq f(x_*) + \langle \nabla f(x_t), x_* - x_t \rangle \end{aligned}$$

Thus we have,

$$\begin{aligned} f(x_{t+1}) &\leq f(x_t) - \frac{1}{2\eta}||x_t - x_{t+1}||_2^2 \ (\textit{from earlier}) \\ &\leq f(x_*) + \langle \nabla f(x_t), x_t - x^x \rangle - \frac{1}{2\eta}||x_t - x_{t+1}||_2^2 \\ &= f(x_*) + \langle \frac{x_t - x_{t+1}}{\eta}, x_t - x_* \rangle - \frac{1}{2\eta}||x_t - x_{t+1}||_2^2 \\ &= f(x_*) + \frac{1}{\eta}\langle x_t - x_{t+1}, x_t - x_* \rangle - \frac{1}{2\eta}||x_t - x_{t+1}||_2^2 \\ &= f(x_*) + \frac{1}{2\eta}(2\langle x_t - x_{t+1}, x_t - x_* \rangle - ||x_t - x_{t+1}||_2^2) \\ &= f(x_*) + \frac{1}{2\eta}(2\langle x_t - x_{t+1}, x_t - x_* \rangle - ||x_t - x_{t+1}||_2^2 + ||x_t - x_*||_2^2 - ||x_t - x_*||_2^2) \\ &= f(x_*) + \frac{1}{2\eta}(-(||x_t - x_*||_2^2 - 2\langle x_t - x_{t+1}, x_t - x_* \rangle + ||x_t - x_{t+1}||_2^2) + ||x_t - x_*||_2^2) \\ &= f(x_*) + \frac{1}{2\eta}(-||x^{t+1} - x^*||_2^2 + ||x_t - x_*||_2^2) \ \ \textit{by Law of Cosines} \end{aligned}$$

Rearranging,

$$f(x_{t+1}) - f(x_*) \leq \frac{1}{2\eta}(-||x^{t+1} - x^*||_2^2 + ||x_t - x_*||_2^2)$$

Taking the total difference over T steps,

$$\begin{aligned} \sum_{i=1}^{t} f(x^i) - f(x_*) &= \frac{1}{2\eta}\sum_{i=1}^{t}(||x^{i-1} - x^*||_2^2 + ||x^i - x_*||_2^2) \\ &= \frac{1}{2\eta}(||x_1 - x_*||_2^2 - ||x_t - x_*||_2^2) \ (\textit{by telescoping}) \\ &\leq \frac{1}{2\eta}||x_1 - x_*||_2^2 \ (\textit{since } ||x_t - x_*||_2^2 \geq 0) \end{aligned}$$

Since $f(x_t) \leq f(x^{T-1}) \leq ... \leq f(x_1)$ by convexity,

$$T(f(x_t) - f(x_*)) \leq \sum_{i=1}^{t} f(x^i) - f(x_*) \leq \frac{1}{2\eta}||x_1 - x_*||_2^2$$

Dividing both sides by T, we get

$$f(x_t) - f(x_*) \leq \frac{||x_1 - x_*||_2^2}{2T\eta}$$

Which is the statement of the theorem. □

Corollary 2. *(Fixed-η Descent Convergence Rate) Picking $\eta = \frac{1}{L}, r = 3, \beta_t = \frac{t-1}{t+2}$ gives us the bound*

$$f(x_t) - f(x_*) \leq \frac{L||x^0 - x_*||_2^2}{2T}$$

Which corresponds to a $\frac{1}{T}$ convergence rate.

To summarize *Theorem 2*, in order to guarantee a certain convergence (indeed, for the theorem it is $\frac{||x_1 - x_*||_2^2}{2\eta T}$), we can attempt a fixed step-size $\eta = \frac{1}{L}$, whereby the definition of L depends on the objective f being L-smooth. Thus, if the previous

[11] In general, any point $x \in \mathbf{dom}(f)$.

sections introduced the constraints of L-smoothness and convexity, this section introduces the constraint of a fixed η.

A natural question at this juncture would be: does η need to be necessarily fixed? After all, Gradient Descent is an iterative, feedback-based algorithm that requires some (perhaps, small) reorientation at every step to ensure we are indeed converging toward the global minima x_*. Could this reorientation also work best with an *adaptive* η?

3.4 Adaptive Convergence?

In the previous section, we've shown that there is at least some mechanism by which we can guarantee some convergence for Gradient Descent (that is, we gave it a specific bound). Can we now improve our guarantee by making η adaptive rather than fixed? Again, we borrow and expand on some working from Tibshirani, 2015.

Algorithm 2 Gradient Descent with Backtracking Line Search

Input: $f, x_1, \eta, T, \alpha, \beta$
Output: x_*

```
last_point := x_1
for t := 1 to T do
    while f(x_t − η∇f(x_t)) ≥ f(x_t) − αη||∇f(x_t)||_2^2 do η := βη
    end while
    x_{t+1} := x_t − η(∇f(x_t))
    last_point = x_{t+1}
end for

x_T = last_point
return x_T
```

Algorithm 2 represents a simple modification of the basic Unconstrained Gradient Descent in *Algorithm 1*. For the sake of a complete summary, we now have:

1. f: now a convex objective.
2. x_1: still an arbitrary starting point.
3. η: now an adjustable step-size to moderate our rate of descent.
4. T: some number of attempts after which we terminate the search for x_*.
5. α, β: some constants that parameterize the adjustment of η.

The main idea behind *Algorithm 2*, is that we establish some threshold α which we use to ensure we are not just minimizing the objective f, but minimizing it proportional to the landscape at the time. So, instead of attempting Steepest Descent for all T steps (unconstrained, and may lead to divergence), or fixing an η for all T steps (constrained, but may be too slow), we adapt η at every t such that η is approximately as much as it needs to be to minimize f.

Hence the η update condition[12]:

$$f(x_t - \eta\nabla f(x_t)) < f(x_t) - \alpha\eta||\nabla f(x_t)||_2^2 \tag{6}$$

In *Eqn. 5*, the $x_t - \eta\nabla f(x_t)$ in the R.H.S., as we might be recall, can be found in the term $x_{t+1} = x_t - \eta\nabla f(x_t)$ in the Gradient Method update rule. So, $f(x_t - \eta\nabla f(x_t)$ just refers to the objective value at the newly selected point x_{t+1}.

On the other side, $f(x_t) - \eta||\nabla f(x_t)||_2^2$ represents the approximate (or loosely, "expected") new $f(x_{t+1})_{\text{approx}}$ because it subtracts the scaled magnitude of the gradient $\eta||\nabla f(x_t)||_2^2$ from the current value of the objective, $f(x)$. So, with the inclusion of α[13] in $f(x_t) - \alpha\eta||\nabla f(x_t)||_2^2$, we can add another "dial" by which we can adjust our expectation for the new objective value $f(x_{t+1})_{\text{approx}}$. If we select a large α, then we expect the actual new objective $f(x_t - \eta\nabla f(x_t))$ to be much lower than the linear approximation $f(x_t) - \alpha\eta||\nabla f(x_t)||_2^2$. If we select a small α, the opposite.

[12] Also known as the **Armijo Condition**
[13] α in this context is referred to as the Armijo Stopping-Condition

Hence, if we select $\alpha = 0.5$, $\beta = 0.9$, and initial $\eta = 1$, then as long as our actual new objective value $f(x_t - \eta\nabla f(x_t))$ is not as good as $f(x_t) - 0.5\eta||\nabla f(x_t)||_2^2$, we will shrink our η by 10%. Until $f(x_t - 0.9\eta\nabla f(x_t)) < f(x_t) - (0.5)(0.9)||\nabla f(x_t)||_2^2$, we will keep shrinking η by 10% some k times, before we finally accept the $x_t - 0.9^k\eta\nabla f(x_t) = x_{t+1}$ for which the exit condition is fulfilled.

Intuitively, this will lead to a descent rate that is fast at first (η will remain high since f is steeper when it is further away from x_*), and eventually slows down as $f(x_t) - \alpha\eta||\nabla f(x_t)||_2^2$ becomes a harder condition to fulfill (due to the gradient flattening). Our choice for β will adjust how exhaustive our search for η is, since intuitively a small β will lead to small increments in η while a large β means η will plunge faster[14].

In general, the domain constraints on α, β are: $0 < \alpha \leq 0.5$ and $0 < \beta < 1$.

3.4.1 Theoretical Analysis

Theorem 5 (BT Line-Search Final Iterate Convergence Bound)**.** *If the objective f is convex, with Lipschitz gradient at $L > 0$, then Gradient Descent with Backtracking Line Search satisfies the convergence bound:*

$$f(x_t) - f(x^*) \leq \frac{||x^0 - x_*||_2^2}{2\eta_{min}T}$$

where $\eta_{min} = min\{1, \frac{\beta}{L}\}$

Proof. It is obvious that the rate of convergence for *Algorithm 2* is dependent on the evolution of η. Fortunately, all cases need not be accounted for.

<u>Case 1, $\eta_{min} = \frac{\beta}{L}$:</u>
With backtracking, we search through a series: $\eta, \frac{\beta}{\eta}, \frac{\beta^2}{\eta}, \frac{\beta^3}{\eta}, ...$ Recall from *Theorem 2* that as long as $0 < \eta \leq \frac{1}{L}$, we are able to guarantee a convergence bound. Since we know that eventually the series $\eta, \frac{\beta}{\eta}, \frac{\beta^2}{\eta}, \frac{\beta^3}{\eta}, ...$ will result in some $\eta_* \leq \frac{1}{L}$, this means that the smallest possible step η can take before entering the range $\eta \leq \frac{1}{L}$ is at least $\frac{\beta}{L}$. Even assuming the worst case, that η is updated only at this smallest pace $\frac{\beta}{L}$, we can still be sure that since there is a way for which η depreciates to $\leq \frac{1}{L}$, the proof of *Theorem 2* can be replicated here. The final bound would then be:

$$\frac{L||x_1 - x_*||_2^2}{2\beta T}$$

compared to the fixed $\eta = \frac{1}{L}$ bound in *Theorem 2*:

$$\frac{L||x_1 - x_*||_2^2}{2T}$$

That is, there is only a constant $\frac{1}{\beta}$ difference in convergence guarantee. In plain terms, if $\beta = 0.5$ (for example), then we are doubling the margin for error of Gradient Descent in finding the true optimum in the same T steps.

However, there is a remedy: by also scaling T up to $\frac{T}{\beta}$, we can tighten the bound. Intuitively, if by the end of T steps, we are some steps away from the optimum where each step reduces the error by β where $\beta < 1$, then by simply performing an increased $\frac{T}{\beta}$ steps in total, we can ensure there is no last-hop gap. We would be trading off efficiency in time for an adaptive, safer, convergence. Although, the convergence bound would still be broadly the same as in *Theorem 2*.

<u>Case 2, $\eta_{min} = 1$:</u>
This accounts for the case that there is no need for any backtracking, i.e. $\eta = 1$ already produces the optimal objective value to satisfy the exit condition $f(x_t - \eta\nabla f(x_t)) < f(x_t) - \alpha\eta||\nabla f(x_t)||_2^2$ (perhaps, if the landscape is relatively very steep all the way). In this case, the smallest step $\eta = 1$ would already be the "worst"-case rate of descent, leading to the bound:

$$\frac{||x_1 - x_*||_2^2}{2T}$$

That is, there is only a constant L difference in convergence guarantee from the bound in *Theorem 2*. To elaborate, the new bound has a margin of error that is L-times smaller than the original convergence guarantee. However, asymptotically-speaking this may not be very impactful. Moreover, the case of $\eta = 1$ worst-case descent rate is unlikely to be relevant for many non-trivial objective landscapes. □

[14] β, in this context, is thus known as the **Armijo Shrinkage-Factor**

In summary, it is plain to see that the convergence bounds of Gradient Descent with fixed η and adaptive η using Backtracking Line Search, are broadly the same with only constant differences. While a trade-off is made in descent time by a worst-case $\eta = \frac{\beta}{L}$ descent rate in favor of eliminating constant differences with the original bound, this may not be an important consideration for objectives with complex landscapes.

3.4.2 Experimental Analysis

Let us test the convergence of Gradient Descent with both fixed and adaptive η on a simple convex objective with known L.

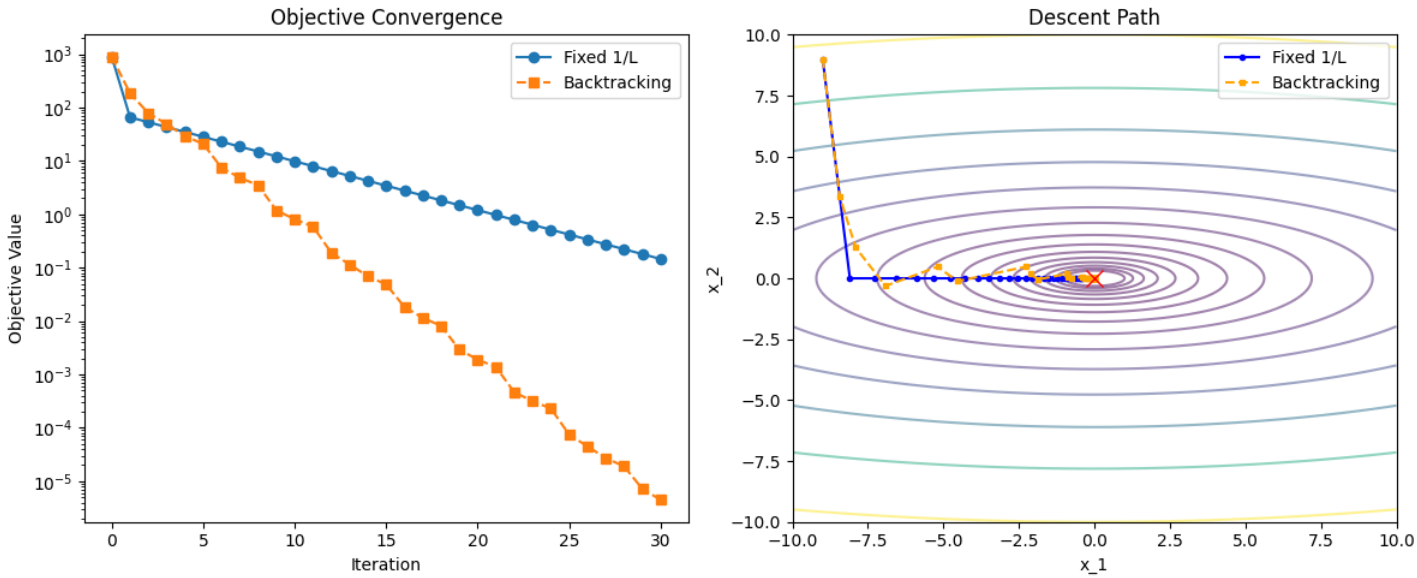


Figure 6: Convergence rates and descent paths for $f = x_1^2 + 10x_2^2$.

Clearly, Gradient Descent with adaptive η converges much closer towards the objective minimum in the same $T = 30$ steps as Gradient Descent with fixed η. Does the theory check out?

The convergence analysis in Section 2.3. tells us that Gradient Descent with fixed η has bound:

$$f(x_t) - f(x^*) \leq \frac{||x_1 - x_*||_2^2}{2\eta T}$$

In this case, $f(x_t) \approx 0.146, f(x_*) = 0$ which gives us an L.H.S. divergence of 0.146. On the other side, $x_1 = [-9; 9], x_* = [0; 0], \eta = \frac{1}{20}, T = 30$ which gives us an R.H.S. of 54. So, fixed-η convergence performs well within the stated bound.

Meanwhile, the convergence analysis in Section 2.4.1. tells us that Gradient Descent with adaptive η has bound:

$$f(x_t) - f(x^*) \leq \frac{L||x_1 - x_*||_2^2}{2\beta T}$$

In this case, $f(x_t) \approx 0.00000495, f(x_*) = 0$ which gives us an L.H.S. divergence of 0.00000495. On the other side, $x_1 = [-9; 9], x_* = [0; 0], L = 20, \beta = 0.5, T = 30$ which gives us an R.H.S. of 108. Indeed, as expected, Gradient Descent with Backtracking Line Search loosens the bound to $\frac{1}{\beta} = 2$ times of the previous bound ($54 \rightarrow 108$). But, the actual final divergence is 5 orders of magnitude closer than the descent with fixed η.

In order to know whether there is a true correlation between GD with adaptive η and quicker convergence, we can run more trials for the same $f = x_1^2 + 10x_2^2$ while varying hyperparameters x_1, T, α, β. Consider *Table 1* in Appendix A1.

As it happens, a grid search of x_1, T, α, β over 48 trials yielded a strong result: that the final convergence value $f(x_t) - f(x_1)$ of GD with adaptive η is almost always smaller than that of GD with fixed $\eta = \frac{1}{L}$. The same idea is represented in *Fig. 7* below.

But why is this outcome so replicable? Notice in *Fig. 6* the actual descent paths of both algorithms for 1 trial. GD with fixed η is uniform in convergence but stops at $x_t \approx [-0.382; 0]$ which is a distance of 0.382 from x_*. Meanwhile, GD with

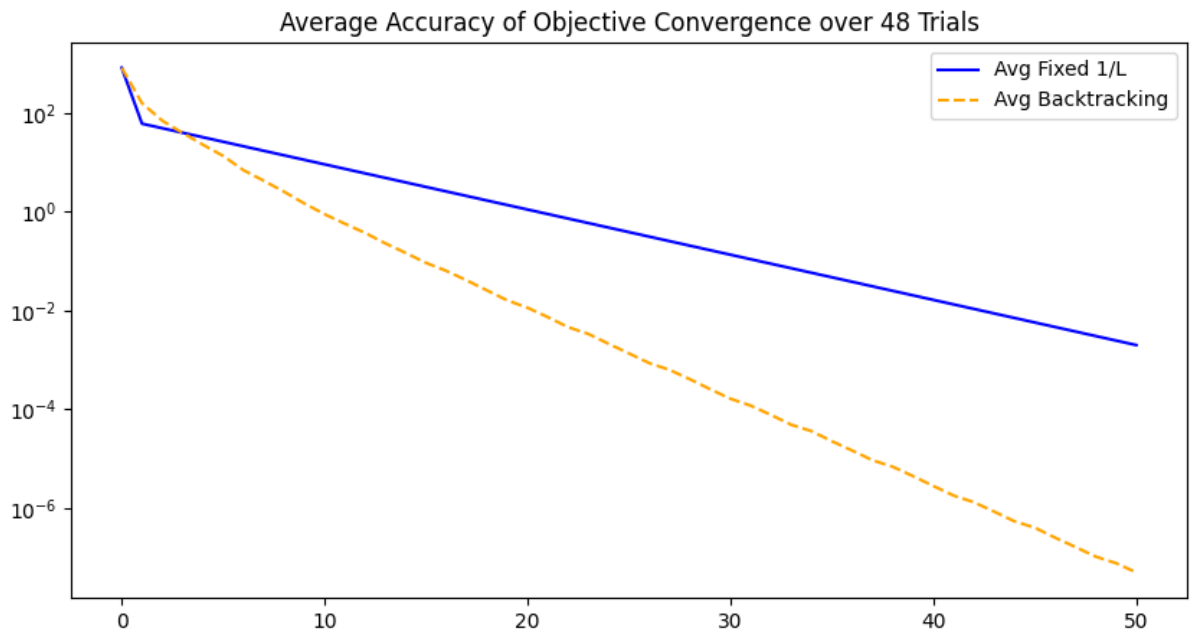


Figure 7: Average convergence rate over 50 iterations of minimizing $f = x_1^2 + 10x_2^2$ over 48 trials.

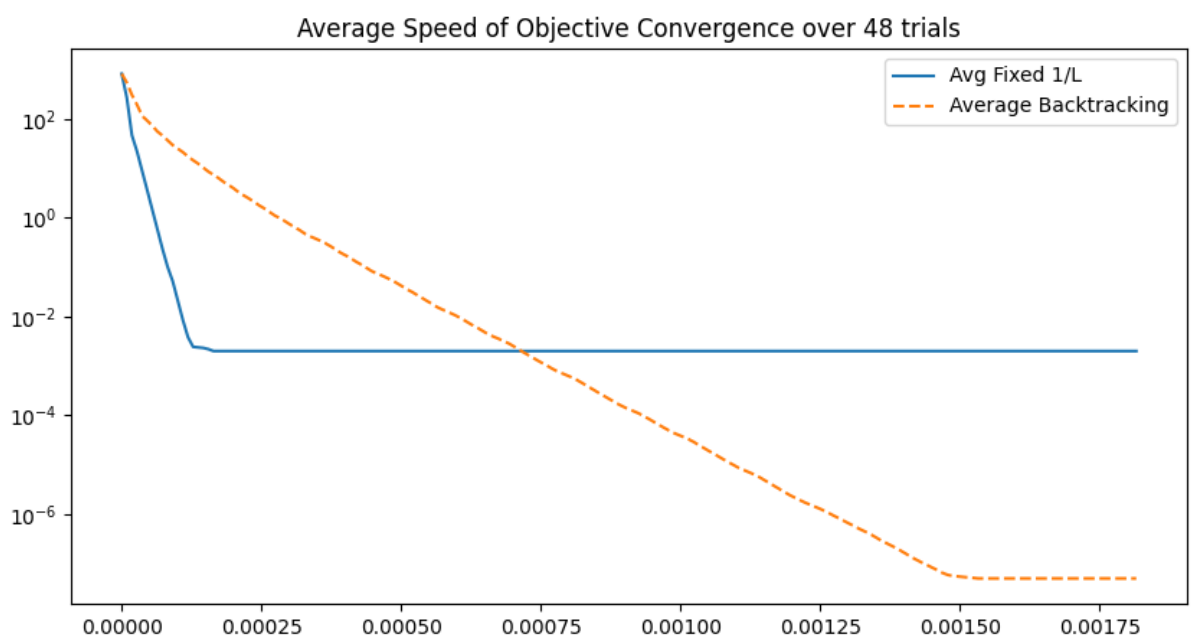


Figure 8: Average convergence rate against wall-clock time to minimize $f = x_1^2 + 10x_2^2$ over 48 trials.

adaptive η is less uniform during convergence but reaches $x_t = [-0.00204, 0.000186]$ which is a distance of 0.00205 from x_* (186 times closer). The slow convergence of fixed-η GD is likely because the Lipschitz Gradient of $L = 20$ leads to a small $\eta = 0.05$. This η constrains convergence in the x_1 direction even if f only requires small steps in the x_2 direction. Therein we empirically discover the limitation of non-adaptive GD.

What we observe is that a poor L can really slow down convergence for GD. But let us shed some more light on how to select L. After all, until now L has just been a given constant (even in the definition of L-smoothness). Formally,

Definition 3 (Lipschitz Constant)**.** *The Lipschitz Constant L of a function is defined as the supremum of the spectral norm of the Hessian across the entire domain:*

$$L = sup_x ||\nabla^2 f(x)||_2^2 = sup_x \lambda_{max}(\nabla^2 f(x))$$

This definition uses the concept of a **spectral norm**[15] which can be thought of informally, as measuring the maximum scale by which a matrix "stretches" a vector. Intuitively then, letting the spectral norm act on the 2nd Derivative (Hessian) matrix $\nabla^2 f(x)$, alerts us as to the steepest change we can expect in a point $f(x)$ as a result of the curvature of f. This is exactly what we need in *Theorem 2* to admit a quadratic upper bound when approximating $f(x')$.

However, we have the issue that it is not always tractable to compute L. Computing L exactly, requires computing the eigendecomposition $P^{-1}\nabla^2 f(x)P$ of $\nabla^2 f(x)$ and doing this can be prohibitively expensive especially for the 2nd-Derivative (Hessian) matrix of f (which scales up to n^2 values for an f of dimension n). The task is further complicated by the fact that we are not just calculating the spectral norm of the Hessian at 1 point, but taking the **supremum** of the spectral norms of the Hessian over every point in the domain of f, which is a task more Herculean.

Thus, often L is approximated using methods such as Power Iteration. Accordingly, the problems of approximation are not new: as seen with the Taylor Series, approximation introduces error, and as seen in the convergence rate above, the L-value can negatively affect convergence even if it is exact let alone approximated. This provides a further motivation to choose adaptive methods like GD with Backtracking Line Search, as these are more agnostic to the L value by introducing an intelligent scaling factor β.

[15] The actual definition of the spectral norm of a matrix A is $\sqrt{\lambda_{max}||A_* A||}$ where A_* is the conjugate transpose to A. However, since in this case the matrix in question, $\nabla^2 f(x)$, is real and symmetric, its conjugate transpose is just itself. This makes the spectral norm basically equivalent to $\lambda_{max}\nabla^2 f(x)$.

4 Stochasticity

4.1 The Deep Learning Demand

We've taken several steps so far towards formulating a constrained and computable algorithm for Gradient Descent that promises correctness in finding the objective minimizer, and within a certain bound.

Indeed, for a simple function like $f = x_1^2 + 10x_2^2$ as has been used in the section for Experimental Analysis, *Algorithm 1* or *Algorithm 2* works efficiently in finding either the global minimizer x_* or some x_t that is extremely close to it.

However, what if our objective f looked something like this:

$$f = \mathcal{L} = \frac{1}{m}\sum_{i=1}^{m} \ell(g(\boldsymbol{x_i}), y_i) \tag{7}$$

where $\ell : \mathbb{R}^m \times \mathbb{R} \to \mathbb{R}$ is some function that measures the discrepancy between some predicted output $g_\theta(\boldsymbol{x})$ and an expected output y.

This is a typical objective seen in many **Machine Learning** problems. The scenario is as follows:

- A **model** is given m independent samples of a vector $x \in \mathbb{R}^n$, $x_1, x_2, ..., x_m$[16]. Correspondingly, the model is also given m labels $y \in \mathbb{R}, y_1, y_2, ..., y_m$ for each of the m samples.
- The task of the model is then to use some function g to operate on each of the m samples $x_1, x_2, ..., x_m$ and generate m hypotheses $g(x_1), g(x_2), ..., g(x_m)$.
- The natural objective of the model is to ensure the hypotheses generated are as close as possible to the true labels $y_1, y_2, , , .y_m$, wherein "closeness" is determined by some predetermined metric ℓ.
- Ultimately, the model is supposed to minimize the **average disparity** over all m predictions and labels, leading to the objective $\frac{1}{m}\sum_{i=1}^{m} \ell(g_\theta(x_i), y_i)$. The disparity term is known as a "loss" (hence ℓ), and thus the overall objective of the model is to minimize average loss. Henceforth, instead of the general term "objective" and symbol f, we will use the term "loss" and symbol $\mathcal{L}$.

ML problems could be broadly classified under classification or regression. For example,

- In a typical binary classification problem such as sentiment analysis, the model might be given m samples of sentiments (through text, speech, images, etc) and be asked to label each one as positive or negative. The ℓ in this case would then typically be the **Binary Cross-Entropy Loss** function[17].
- In a typical regression problem such as housing price prediction, the model might be given m samples of house profiles and be asked to predict house price for each profile. The ℓ in this case would then typically be the squared distance $(\hat{y} - y)^2$.

But, how does the model learn to generate better hypotheses over m samples? In other words, what exactly is the model optimizing over, if not $\boldsymbol{x}$ (which is already given)? To answer this, we slightly modify the objective:

$$\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m} \ell(g_{\boldsymbol{\theta}}(x_i), y_i) \tag{8}$$

The addition here is θ. Intuitively, if each sample x is composed of several distinct features which inform the prediction $g(x)$, then it is in the model's interest to know how to weigh each feature. $\boldsymbol{\theta} \in \mathbb{R}^d$ is thus a **weights** tensor that is used to transform a raw feature vector x into a weighted tensor $\boldsymbol{U}$[18].

Finally, for an ML problem to become a **Deep Learning** problem, the model may additionally need to operate over several different layers - each with their own weights tensors (for richer representations), and activation functions (like g, to introduce non-linearity to the model to boost its ability to deal with complex features).

[16] As before, each $\boldsymbol{x}$ is really an m-dimensional point $(x_1, x_2, ..., x_m)$ in $\mathbb{R}$. In most ML contexts, these coordinates are called **features** .

[17] The *Binary Cross-Entropy Loss* function is a function that measures the ability of a model to predict the "class" of an object correctly, where the object's class is drawn from an unknown distribution of probable classes (of which there are exactly 2). Intuitively, the "loss" here arises from incorrect predictions/classifications made.

[18] Notice that the weights tensor is of dimension d, which does not necessarily equal the feature dimension n. This is because we may want to represent features in more sophisticated ways, and to do this we use linear projection to promote an n-dimensional feature vector to a d-dimensional tensor.

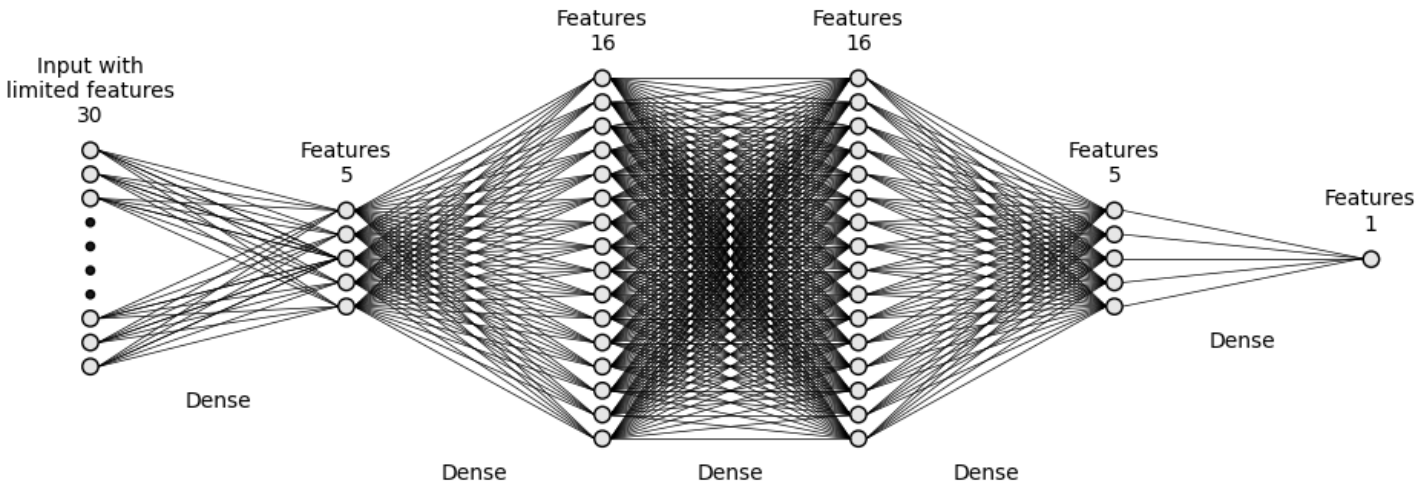


Figure 9: A multi-layer, or "deep" learning model

To return to the original dilemma, once we have an objective like L that involves optimizing over several ℓ, each of which may have challenging landscapes, it becomes inefficient to use a procedure like *Algorithm 1* which works by considering a single $\boldsymbol{x}$ at a time instead of some number of samples $x_1, x_2, ...x_m$ where m is often in the hundreds of thousands if not millions. In essence, the objective $\mathcal{L}$ is no longer a simple shape like $x_1^2 + 10x_2^2$ which predefines all $x \in \mathbf{dom}(\mathcal{L})$ but a linear combination of some set of x wherein every sample is involved.

The demand at this stage is clear: we need an algorithm that can effectively optimize over millions of samples while still implementing the core idea of Gradient Descent.

4.2 Stochastic Gradient Descent

Algorithm 3 Stochastic Gradient Descent

Input: $\mathcal{L}, \boldsymbol{X}, \boldsymbol{\theta}_1, \eta, T, b$
Output: $\boldsymbol{\theta}_*$

```
sample_points = {x_1, x_2, ..., x_b}
final_θ = θ_1
for t := 1 to T do
    for t := 1 to b do ∇L += ∇L(θ_t)
    end for
    ∇̂L := ∇̂L / b
    θ_{t+1} := θ_t − η(∇̂L)
    final_θ := θ_{t+1}
end for

θ_t = final_θ
return θ_t
```

Codified above is the general **Stochastic Gradient Descent (SGD)** algorithm. The main differences with *Algorithm 1* are listed below:

- Instead of a single initial starting point x_1, we are given m samples $\boldsymbol{X}$.
- There is a new $\boldsymbol{\theta}_*$ we are optimizing for, for which a starting point $\boldsymbol{\theta}_1$ is provided.
- Instead of updating θ based on $\nabla\mathcal{L}$ over all $\boldsymbol{X}$, b[19] samples $x_1, x_2, ..., x_b$ are drawn uniformly at random from $\boldsymbol{X}$ without replacement. Using these samples, b gradients are calculated (for steepest descent) which are then averaged to form

[19] A new hyperparameter: this version of *SGD* is called ***Mini-Batch SGD***.

$\hat{\nabla}\mathcal{L}$. This is then used to obtain $\boldsymbol{\theta}_{t+1}$ from $\boldsymbol{\theta}_t$ through the established Gradient Descent update rule.

The key thing to note is the use of $\hat{\nabla}\mathcal{L}$. This acts as a cheap and unbiased estimator for $\nabla\mathcal{L}$, which is the more expensive-to-compute gradient of $\mathcal{L}$ over all m samples. This estimator satisfies the property:

$$\mathbb{E}_t[\hat{\nabla} f(\boldsymbol{\theta}_t)] = \nabla\mathcal{L}(\boldsymbol{\theta}_t)$$

That is, the expectation of $\nabla\mathcal{L}(\boldsymbol{\theta}_t)$ conditioned on all past random choices $1, 2, ..., t-1$ is equal to $\mathcal{L}(\boldsymbol{\theta}_t)$. This nice property is supported by the *Linearity of Differentiation*, which allows us to claim that $\nabla(\frac{1}{m}\sum_{i=1}^{m}\mathcal{L}) = \frac{1}{m}\sum_{i=1}^{m}\nabla\mathcal{L}$[20].

Remark 1. Before proceeding further, it should be noted that *SGD*, or more specifically *Stochasticity*, is not a method that is aimed at necessarily minimizing the theoretical convergence rate or final iterate bound of the Gradient Descent method through some algorithmic enhancement (unlike methods later introduced such as *Adam*). It is simply a practical variation often deployed to tackle problems over large domains with complex landscapes.

In-fact, with *Stochasticity* the Gradient Descent method may actually degrade in accuracy due to a variance factor that is introduced and which grows larger over time. Our goal in this chapter is thus primarily to ensure that the convergence rate and accuracy do not degrade too heinously under *Stochasticity*.

4.2.1 Convergence

Here, we borrow and expand on working from Farina, 2024's lecture on *SGD*.

. . . . . × . . . . . .

Derivation 3. *Recall from Theorem 2 that a differentiable, L-smooth function $f : X \to \mathbb{R}$ over a convex domain X, admits the quadratic upper bound:*

$$f(x') \leq f(x) + \langle \nabla f(x), x' - x\rangle + \frac{L}{2}||x' - x||_2^2 \quad \forall x', x \in X$$

Substituting the terms with the Deep Learning formulation where $f = \mathcal{L}, x = \boldsymbol{\theta}_t, x' = \boldsymbol{\theta}_{t+1}$, we get:

$$\mathcal{L}(\boldsymbol{\theta}_{t+1}) \leq \mathcal{L}(\boldsymbol{\theta_t}) - \eta\langle\nabla\mathcal{L}(\boldsymbol{\theta}_t), \hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_{t+1}\rangle + \frac{L}{2}||\hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_t)||_2^2$$

Applying the conditional expectation to both sides, we get

$$\mathbb{E}_t[\mathcal{L}(\boldsymbol{\theta}_{t+1})] \leq \mathbb{E}_t[\mathcal{L}(\boldsymbol{\theta_t})] - \eta\mathbb{E}_t[\langle\nabla\mathcal{L}(\boldsymbol{\theta}_t), \hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_{t+1}\rangle] + \mathbb{E}_t[\frac{L}{2}\eta^2||\hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_t)||_2^2],$$

which simplifies to

$$\mathbb{E}_t[\mathcal{L}(\boldsymbol{\theta}_{t+1})] \leq \mathcal{L}(\boldsymbol{\theta_t}) - \eta||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2^2 + \frac{L}{2}\eta^2\mathbb{E}_t[||\hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_t)||_2^2],$$

. . . . . × . . . . . .

because

- We know that $\mathbb{E}_t[\mathcal{L}(\boldsymbol{\theta_t})] = \mathcal{L}(\boldsymbol{\theta_t})$ (*Expectation of a constant term*).
- We can use **Event Independence** to simplify $\eta\mathbb{E}_t[\langle\nabla\mathcal{L}(\boldsymbol{\theta}_t), \hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_{t+1}\rangle]$ to $\eta\langle\mathbb{E}_t[\nabla\mathcal{L}(\boldsymbol{\theta}_t)], \mathbb{E}_t[\hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_{t+1}]\rangle$. In turn, we know that $\mathbb{E}_t[\hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_t)] = \nabla\mathcal{L}(\boldsymbol{\theta}_t)$ (*From earlier*).
- We can use the **Linearity of Expectation** to simplify $\mathbb{E}_t[\frac{L}{2}\eta^2||\hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_t)||_2^2]$ to $\frac{L}{2}\eta^2\mathbb{E}_t[||\hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_t)||_2^2]$.

Therefore, we have:

[20] The formal statement of the *Linearity of Differentiation* is $\frac{d}{dx}(\alpha f(x) + \beta g(x)) = \alpha f'(x) + \beta g'(x)$.

Lemma 1. *(SGD Consecutive Iterate Proximity Bound) A differentiable, L-smooth loss $\mathcal{L} : \Omega \to \mathbb{R}$ over a convex domain Ω, admits the quadratic upper bound:*

$$\mathbb{E}_t[\mathcal{L}(\boldsymbol{\theta}_{t+1})] \leq \mathcal{L}(\boldsymbol{\theta_t}) - \eta||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2^2 + \frac{L}{2}\eta^2\mathbb{E}_t[||\hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_t)||_2^2],$$

$\forall \boldsymbol{\theta}_{t+1}, \boldsymbol{\theta}_t \in \Omega$, and $\hat{\nabla}\mathcal{L}$ is an unbiased conditional estimator for $\nabla\mathcal{L}$

Now that we are able to guarantee descent, i.e. that loss truly decreases between 2 successive iterates $\mathcal{L}(\boldsymbol{\theta}_t), \mathcal{L}(\boldsymbol{\theta}_{t+1})$ of the model,

Theorem 6. *(SGD Gradient Norm Bound) If the loss $\mathcal{L}$ is L-Lipschitz continuous and $\mathbb{E}_t[||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2^2] \leq G$ for any $t = 1, ..., T$, then Stochastic Gradient Descent satisfies the convergence bound:*

$$\frac{1}{T}\sum_{t=1}^{T}\mathbb{E}[||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2^2] \leq \frac{\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*)}{\eta T} + \frac{L}{2}\eta G$$

Proof. Given that $\mathbb{E}_t[||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2^2] \leq G$ at any t, then the statement of *Lemma 1* can be rewritten like so:

$$\begin{aligned}
\mathbb{E}_t[\mathcal{L}(\boldsymbol{\theta}_{t+1})] &\leq \mathcal{L}(\boldsymbol{\theta_t}) - \eta||\nabla f(\boldsymbol{\theta}_t)||_2^2 + \frac{L}{2}\eta^2\mathbb{E}_t[||\hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_t)||_2^2] \\
&\implies \mathbb{E}_t[\mathcal{L}(\boldsymbol{\theta}_{t+1})] - \frac{L}{2}\eta^2\mathbb{E}_t[||\hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_t)||_2^2] \leq \mathcal{L}(\boldsymbol{\theta_t}) - \eta||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2^2 \\
&\implies \eta||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2^2 \leq \mathcal{L}(\boldsymbol{\theta_t}) - \mathbb{E}_t[\mathcal{L}(\boldsymbol{\theta}_{t+1})] + \frac{L}{2}\eta^2\mathbb{E}_t[||\hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_t)||_2^2] \\
&\implies ||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2^2 \leq \frac{1}{\eta}(\mathcal{L}(\boldsymbol{\theta_t})) - \frac{1}{\eta}\mathbb{E}_t[\mathcal{L}(\boldsymbol{\theta}_{t+1})] + \frac{L}{2}\eta\mathbb{E}_t[||\hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_t)||_2^2] \\
&\implies ||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2^2 \leq \frac{1}{\eta}\mathbb{E}_t[\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta})^{\boldsymbol{t+1}}] + \frac{L}{2}\eta G
\end{aligned}$$

Summing over all times $t = 1, 2, ..., T$, we obtain

$$\sum_{i=1}^{t}||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2^2 \leq \frac{1}{\eta}(\sum_{i=1}^{t}\mathbb{E}_t[\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta})^{\boldsymbol{t+1}}]) + \frac{L}{2}\eta GT$$

Taking the expectation of both sides we get

$$\begin{aligned}
\mathbb{E}[\sum_{i=1}^{t}||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2^2] &\leq \mathbb{E}[\frac{1}{\eta}(\sum_{i=1}^{t}[\mathbb{E}_t[\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_{t+1})])] + \frac{L}{2}\eta GT] \\
&\implies \sum_{i=1}^{t}\mathbb{E}[||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2^2] \leq \frac{1}{\eta}(\sum_{i=1}^{t}\mathbb{E}[\mathbb{E}_t[\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_{t+1})]]) + \frac{L}{2}\eta GT] \\
&\text{(by the } \textit{Linearity of Expectation}\text{)} \\
&\implies \sum_{i=1}^{t}\mathbb{E}[||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2^2] \leq \frac{1}{\eta}(\sum_{i=1}^{t}\mathbb{E}[\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_{t+1})]) + \frac{L}{2}\eta GT] \\
&\text{(by the } \textit{Law of Total Expectation}\text{)} \\
&\implies \sum_{i=1}^{t}\mathbb{E}[||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2^2] \leq \frac{1}{\eta}(\mathcal{L}(\boldsymbol{\theta}^0) - \mathbb{E}[\mathcal{L}(\boldsymbol{\theta}^T)]) + \frac{L}{2}\eta GT] \\
&\text{(by } \textit{Telescoping}\text{)}
\end{aligned}$$

Assuming $\mathcal{L}$ is lower bounded by some $f(\boldsymbol{\theta})_*$, we have

$$\frac{1}{T}\sum_{i=1}^{t}\mathbb{E}[||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2^2] \leq \frac{\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*)}{\eta T} + \frac{L}{2}\eta G,$$

which is the statement of the theorem. □

Corollary 3. *(SGD Gradient Norm Convergence Rate) Picking $\eta = \frac{1}{\sqrt{T}}$ ensures that within T iterations, at least one*

$$\mathbb{E}[||\mathcal{L}(\boldsymbol{\theta}_t)||] \approx \frac{1}{\sqrt{T}}$$

Remark 2. Why, for *Stochastic Gradient Descent*, do we first bound the average gradient norm of the objective $\mathcal{L}$? A few considerations for why this is a useful exercise:

- Why not bound the objective value $\mathcal{L}$ directly instead of the gradient norm? Stochasticity does not allow us to guarantee anything meaningful for just the final term $f(\boldsymbol{\theta}_t)$. This is in-fact precisely why we average the gradient computed for the samples we draw randomly.
- Why bound the gradient norm specifically? The gradient norm can still indicate as to how well we've minimized the objective since a lower norm indicates a region close to the minima (which we're assured for a convex function is global).
- Why is bounding the average gradient norm useful? **Jensen's Inequality** reveals that if the average value of a convex function can be bounded by some value λ, then the average point within the domain of that function is likely to be lower than λ[a].

[a] The original probabilistic form of **Jensen's Inequality** is $\phi(\mathbb{E}[X]) \leq \mathbb{E}[\phi(X)]$ where ϕ is a convex function and X is an integrable real-valued random variable. In this case, $\phi = ||\cdot||$ i.e. the gradient norm, and $X = \nabla f(\boldsymbol{\theta}_t)$.

Lemma 2. *(SGD Convex Consecutive Iterate Error Bound) If the loss $\mathcal{L}$ is convex, then for any $\eta > 0$, two consecutive iterates $\boldsymbol{\theta}_t, \boldsymbol{\theta}_{t+1}$ satisfy*

$$\mathcal{L}(\boldsymbol{\theta}_t) \leq \mathcal{L}(\boldsymbol{\theta}_*) + \frac{1}{2\eta}\mathbb{E}_t[||\boldsymbol{\theta}_t - \boldsymbol{\theta}_*]_2^2 - ||\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_*]_2^2 + ||\boldsymbol{\theta}_t - \boldsymbol{\theta}_{t+1}]_2^2]$$

where $\boldsymbol{\theta}_*$ is as usual the true minimizer of $\mathcal{L}$.

Proof. We start with the known principle of the linear approximation to a convex function

$$\mathcal{L}(\boldsymbol{\theta}_*) \geq \mathcal{L}(\boldsymbol{\theta}_t) + \langle \nabla\mathcal{L}(\boldsymbol{\theta}_t), \boldsymbol{\theta}_* - \boldsymbol{\theta}_t\rangle$$

Which is equivalent to

$$\begin{aligned}
\mathcal{L}(\boldsymbol{\theta}_t) &\leq \mathcal{L}(\boldsymbol{\theta}_*) + \langle \nabla\mathcal{L}(\boldsymbol{\theta}_t), \boldsymbol{\theta}_* - \boldsymbol{\theta}_t\rangle \\
&= \mathcal{L}(\boldsymbol{\theta}_*) - \langle \mathbb{E}_t[\hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_t)], (\boldsymbol{\theta}_* - \boldsymbol{\theta}_t)\rangle \ (\textit{Definition of } \hat{\nabla}\mathcal{L}) \\
&= \mathcal{L}(\boldsymbol{\theta}_*) - \mathbb{E}_t[\frac{1}{\eta}\langle(\boldsymbol{\theta}_t - \boldsymbol{\theta}_{t+1}), (\boldsymbol{\theta}_* - \boldsymbol{\theta}_t)\rangle] \ (\textit{SGD update step})
\end{aligned} \tag{9}$$

Now, if we consider the distance between the next iterate, $\boldsymbol{\theta}_{t+1}$ to the objective $\boldsymbol{\theta}_*$ i.e. $||\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_*||_2^2$, we can expand it as such:

$$\begin{aligned}
||\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_*||_2^2 &= ||(\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t) + (\boldsymbol{\theta}_t - \boldsymbol{\theta}_*)||_2^2 \\
&= ||(\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t)||_2^2 + ||(\boldsymbol{\theta}_t - \boldsymbol{\theta}_*)||_2^2 + 2\langle(\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t), (\boldsymbol{\theta}_t - \boldsymbol{\theta}_*)\rangle
\end{aligned}$$

Rearranging, we get

$$-\langle(\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t), (\boldsymbol{\theta}_t - \boldsymbol{\theta}_*)\rangle = \frac{1}{2}(||(\boldsymbol{\theta}_t - \boldsymbol{\theta}_{t+1})||_2^2 - ||\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_*||_2^2 + ||(\boldsymbol{\theta}_t - \boldsymbol{\theta}_*)||_2^2)$$

Which is equivalent to

$$-\frac{1}{\eta}\langle(\boldsymbol{\theta}_t - \boldsymbol{\theta}_{t+1}), (\boldsymbol{\theta}_* - \boldsymbol{\theta}_t)\rangle = \frac{1}{2\eta}(||(\boldsymbol{\theta}_t - \boldsymbol{\theta}_{t+1})||_2^2 - ||\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_*||_2^2 + ||(\boldsymbol{\theta}_t - \boldsymbol{\theta}_*)||_2^2)$$

Substituting into *Eqn. 9* from above, we get the statement of the Lemma:

$$\mathcal{L}(\boldsymbol{\theta}_t) \leq \mathcal{L}(\boldsymbol{\theta}_*) + \frac{1}{2\eta}\mathbb{E}_t[||(\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t)||_2^2 - ||\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_*||_2^2 + ||(\boldsymbol{\theta}_t - \boldsymbol{\theta}_*)||_2^2]$$

□

Given that we are able to bound the distance to the objective minimizer, $\boldsymbol{\theta}_*$ based on any two consecutive iterates, $\boldsymbol{\theta}_t, \boldsymbol{\theta}_{t+1}$ of *Stochastic Gradient Descent*, we are ready for an average convergence bound:

Theorem 7. *(SGD Average Final Iterate Proximity Bound) If the loss $\mathcal{L}$ is convex as well as L-Lipschitz continuous and $\mathbb{E}_t[||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2^2] \leq G$ for any t, then Stochastic Gradient Descent satisfies the convergence bound:*

$$\mathbb{E}[\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*)] \leq \frac{1}{2\eta T}||\boldsymbol{\theta}^1 - \boldsymbol{\theta}_*||_2^2 + \frac{\eta}{2}G$$

for at least one iterate $\boldsymbol{\theta}_t$ from $\boldsymbol{\theta}^1, \boldsymbol{\theta}^2, ..., \boldsymbol{\theta}_t$

Proof. Starting from *Lemma 2* we have,

$$\mathcal{L}(\boldsymbol{\theta}_t) \leq \mathcal{L}(\boldsymbol{\theta}_*) + \frac{1}{2\eta}\mathbb{E}_t[||(\boldsymbol{\theta}_t - \boldsymbol{\theta}_{t+1})||_2^2 - ||\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_*||_2^2 + ||(\boldsymbol{\theta}_t - \boldsymbol{\theta}_*)||_2^2]$$

By summing this bound for all $t = 1, 2, ..., T$:

$$\begin{aligned}
\sum_{t=1}^{t}\mathcal{L}(\boldsymbol{\theta}_t) &\leq T\mathcal{L}(\boldsymbol{\theta}_*) + \frac{1}{2\eta}\sum_{t=1}^{t}\mathbb{E}_t[||(\boldsymbol{\theta}_t - \boldsymbol{\theta}_{t+1})||_2^2 - ||\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_*||_2^2 + ||(\boldsymbol{\theta}_t - \boldsymbol{\theta}_*)||_2^2] \\
&= T\mathcal{L}(\boldsymbol{\theta}_*) + \frac{1}{2\eta}\sum_{t=1}^{t}\mathbb{E}_t[\eta^2||\hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_t)||_2^2 - ||\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_*||_2^2 + ||(\boldsymbol{\theta}_t - \boldsymbol{\theta}_*)||_2^2] \\
&\leq T\mathcal{L}(\boldsymbol{\theta}_*) + \frac{1}{2\eta}\sum_{t=1}^{t}\mathbb{E}_t[\eta^2 G - ||\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_*||_2^2 + ||(\boldsymbol{\theta}_t - \boldsymbol{\theta}_*)||_2^2] \\
&= T\mathcal{L}(\boldsymbol{\theta}_*) + \frac{1}{2\eta}\sum_{t=1}^{t}\mathbb{E}_t[\eta^2 G + ||(\boldsymbol{\theta}_t - \boldsymbol{\theta}_*)||_2^2 - ||\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_*||_2^2]
\end{aligned}$$

Similar to *Theorem 6*, taking the expectation on both sides leads to the conditional expectations $\mathbb{E}_t$ decaying into regular expectations, allowing us to exploit telescoping for $||(\boldsymbol{\theta}_t - \boldsymbol{\theta}_*)||_2^2 - ||\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_*||_2^2$

$$\begin{aligned}
\sum_{t=1}^{t}\mathbb{E}[\mathcal{L}(\boldsymbol{\theta}_t)] &\leq T\mathcal{L}(\boldsymbol{\theta}_*) + \frac{1}{2\eta}\mathbb{E}[\sum_{t=1}^{t}\mathbb{E}_t[\eta^2 G + ||(\boldsymbol{\theta}_t - \boldsymbol{\theta}_*)||_2^2 - ||\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_*||_2^2]] \\
&\leq T\mathcal{L}(\boldsymbol{\theta}_*) + \frac{1}{2\eta}||\boldsymbol{\theta}^1 - \boldsymbol{\theta}_*||_2^2 + \frac{\eta}{2}GT
\end{aligned}$$

Rearranging and dividing by T gives us the statement of the theorem wherein for at least one iterate,

$$\mathbb{E}[\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*)] \leq \frac{1}{2\eta T}||\boldsymbol{\theta}_1 - \boldsymbol{\theta}_*||_2^2 + \frac{\eta}{2}G$$

□

Corollary 4. *(SGD Convergence Rate) Picking $\eta = \frac{1}{\sqrt{GT}}$ gives us the bound*

$$\mathbb{E}[\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*)] \leq \frac{\sqrt{G}}{2}(1 + ||\boldsymbol{\theta}^1 - \boldsymbol{\theta}_*||_2^2)\frac{1}{\sqrt{T}}$$

for at least one iterate $t = 1, 2, ..., T$. This corresponds to a $\frac{1}{\sqrt{T}}$ convergence rate.

And again, as with the gradient norm, by **Jensen's Inequality** we can say that the same bound also holds for $\mathbb{E}[\mathcal{L}(\bar{\boldsymbol{\theta}}_t) - \mathcal{L}(\boldsymbol{\theta}_*)]$ where $\bar{\boldsymbol{\theta}}_t$ is the average of the iterates $\boldsymbol{\theta}^1, \boldsymbol{\theta}^2, ..., \boldsymbol{\theta}_t$.

Remark 3. The choice of $\frac{1}{\sqrt{GT}}$ for η is far from arbitrary. Let us generalize the statement of *Theorem 7* to be:

$$\mathbb{E}[\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*)] \leq \frac{D^2}{2\eta T} + \frac{\eta}{2}G$$

where D is some distance metric (instead of the Euclidean distance $||\cdot||_2$ specifically). Now, what choice of η minimizes the term $\frac{D^2}{2\eta T} + \frac{\eta}{2}G$?

$$f(\eta) = \frac{D^2}{2\eta T} + \frac{\eta}{2}G$$
$$f'(\eta) = \frac{D^2}{2\eta^2 T} + \frac{G}{2}$$

Setting $f' = 0$,

$$\frac{D^2}{2\eta^2 T} = -\frac{G}{2} \iff \eta = \frac{D}{\sqrt{GT}}$$

Where $\frac{D}{\sqrt{GT}}$ is simplified to $\frac{1}{\sqrt{GT}}$ since the true distance $D = ||\boldsymbol{\theta}^1 - \boldsymbol{\theta}_*||_2$ is often unknown.

4.3 Stochastic Variance Reduced Gradient

Terms like $\frac{L}{2}\eta^2\mathbb{E}_t[||\hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_t)||_2^2]$ in *Lemma 1* and $\frac{\eta}{2}G$ in *Theorem 7*, represent **variance** terms emerge due to stochasticity. To reduce such variance, we may consider mutations of *SGD* such as **Stochastic Variance Reduced Gradient**, codified in *Algorithm 4* below. This algorithm was taken from Johnson and Zhang, 2013, where several important theoretical convergence bounds analyzed for *SVRG* can also be found.

Algorithm 4 Stochastic Variance Reduced Gradient

Input: $g, \boldsymbol{X}, \boldsymbol{\theta}_1, \eta, S, T$
Output: $\boldsymbol{\theta}_*$

1: **for** $s := 2$ to S **do**
2: $\quad\tilde{\boldsymbol{\theta}} = \tilde{\boldsymbol{\theta}}_{s-1}$
3: $\quad\tilde{\nabla} = \frac{1}{n}\sum_{i=1}^{n}\nabla g_i(\tilde{\boldsymbol{\theta}})$
4: $\quad\boldsymbol{\theta}^1 = \tilde{\boldsymbol{\theta}}$
5: $\quad$**for** $t := 2$ to T **do**
6: $\quad\quad$Randomly pick $i \in \{1, \ldots, |\boldsymbol{X}|\}$
7: $\quad\quad\hat{\nabla}_t = \nabla g_i(\boldsymbol{\theta}_{t-1}) - \nabla g_i(\tilde{\boldsymbol{\theta}}) + \tilde{\nabla}$
8: $\quad\quad\boldsymbol{\theta}_t = \boldsymbol{\theta}_{t-1} - \eta\hat{\nabla}_t$
9: $\quad$**end for**
10: $\quad\tilde{\boldsymbol{\theta}}_s = \boldsymbol{\theta}_t$ for randomly chosen $t \in \{1, \ldots, T\}$
11: **end for**
12: **return** $\tilde{w}_s$

The key differences between *Algorithm 4* and *Algorithm 3*:

- We now have a number of epochs S. For each epoch, we identify a "starting point" $\boldsymbol{\theta}_1$ based on the average gradient of the loss $\mathcal{L}$ based on an estimator $\boldsymbol{\theta}_{s-1}$ which is generated from the previous epoch.
- Using this starting point, we perform the usual gradient descent update step over T iterations (*Line 7*). The key difference is that instead of using just the gradient of the samples at the current step t (which suffers from variance), a combination of the current gradient along with the gradient of the anchor weights $\nabla g_i(\tilde{\boldsymbol{\theta}})$ and the anchor gradient $\tilde{\nabla}$ is used. In essence, variance is reduced by correcting the gradient update using historical information about the gradient.
- The representative weights for an epoch $\tilde{\boldsymbol{\theta}}_s$ are chosen randomly from any of the calculated weights $\boldsymbol{\theta}_2, \boldsymbol{\theta}_3, ..., \boldsymbol{\theta}_t$.

5 Acceleration

5.1 Looking Back in Time

It should be no surprise that we are always interested in certain attributes of our optimization:

- The speed of convergence in iterates: how quickly we can minimize our loss, benchmarked by a bounded proximity-term.
- The speed of convergence in wall-clock time: the same as above, but measured in real time rather than via iterations.
- The proximity term itself: how far we end up from the true minimizer.

We've so-far looked at various methods of speeding up convergence and minimizing proximity (adding smoothness, convexity, and testing fixed & adaptive step-sizes), and we've done so while keeping the gradient descent update rule largely untouched. That is, we use only a single "current" term $\boldsymbol{\theta}_t$ and an associated gradient $\nabla f(\boldsymbol{\theta})$ that has been routinely updated to inform a single "future" term $\boldsymbol{\theta}_{t+1}$.

The obvious question now is, what if we could modify the gradient descent update rule in a way that used not just a current term $\boldsymbol{\theta}_t$ but the immediate previous term $\boldsymbol{\theta}_{t-1}$? That is, what if we got slightly more introspective with our gradient update?

5.2 Momentum

Recall the standard gradient descent update rule we've been deploying till now:

$$\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \nabla \mathcal{L}(\boldsymbol{\theta}_t)$$

Consider this alternate variation:

$$\begin{aligned} \boldsymbol{\theta}_{t+1} &= \boldsymbol{W}_t - \eta \nabla \mathcal{L}(\boldsymbol{W}_t), \\ \boldsymbol{W}_t &= \boldsymbol{\theta}_t + \beta_t(\boldsymbol{\theta}_t - \boldsymbol{\theta}_{t-1}), t \geq 0 \end{aligned} \tag{10}$$

The differences are:

- A new intermediary weights tensor, $\boldsymbol{W}$.
- New adaptive hyperparameter β_t.
- The addition of a "past" term $\boldsymbol{\theta}_{t-1}$ along with present term $\boldsymbol{\theta}_t$ and future term $\boldsymbol{\theta}_{t+1}$.

The new update rule in *Eqn. 10* is used in an enhanced algorithm called **Nesterov Accelerated Gradient** and will be the basis of analysis in this chapter.

Algorithm 5 Nesterov Accelerated Gradient

Input: $\mathcal{L}, \boldsymbol{\theta}_1, \boldsymbol{W}_1, \eta, \beta, T$
Output: $\boldsymbol{\theta}_*$

```
final_θ = θ_1
for t := 1 to T do
    W_t = θ_t + β_t(θ_t − θ_{t−1})
    θ_{t+1} = W_t − η∇L(W_t)
    final_θ = θ_{t+1}
end for

θ_t = final_θ
return θ_t
```

The required check when introducing any new parameter, is to clarify what its exact contribution is. In the case of NAG, the joint-purpose of $\beta_t, \boldsymbol{W}$, and $\boldsymbol{\theta}_{t-1}$ is to answer 1 question: "Can we use our current **momentum** to preempt where we might end up next, and course-correct from there?". The reason this is called an "accelerated" descent should now be obvious: it employs a strategy called **lookahead** to circumvent a future descent step that may only increase loss ($\mathcal{L}$) instead of minimizing it.

The word **momentum** is not chosen lightly. In physics, it is used to describe the product of an object's mass, and its velocity. Accordingly in *NAG*, the (past) velocity is $\boldsymbol{\theta}_t - \boldsymbol{\theta}_{t-1}$, the mass is the tunable β_t, and therefore the lookahead position considering momentum is encapsulated in $\boldsymbol{W}_t = \boldsymbol{\theta}_t + \beta_t(\boldsymbol{\theta}_t - \boldsymbol{\theta}_{t-1})$ for the tth iterate of *NAG*.

The preemption (or correction) against this potentially wrongful $\boldsymbol{W}_t$, is then done through the regular Gradient Descent update wherein the loss-value at the lookahead step is judged via $\nabla f(\boldsymbol{W}_t)$ and descent away from it is ensured through executing $\boldsymbol{W}_t - \eta \nabla f(\boldsymbol{W}_t)$.

These ideas are quite simple (illustrated in *Fig. 10*) in their claim to improvement, but we still need to assess them via some of the metrics identified earlier. Namely, the final proximity bound guaranteed by *NAG*, and how quickly it converges in terms of fixed iterates.

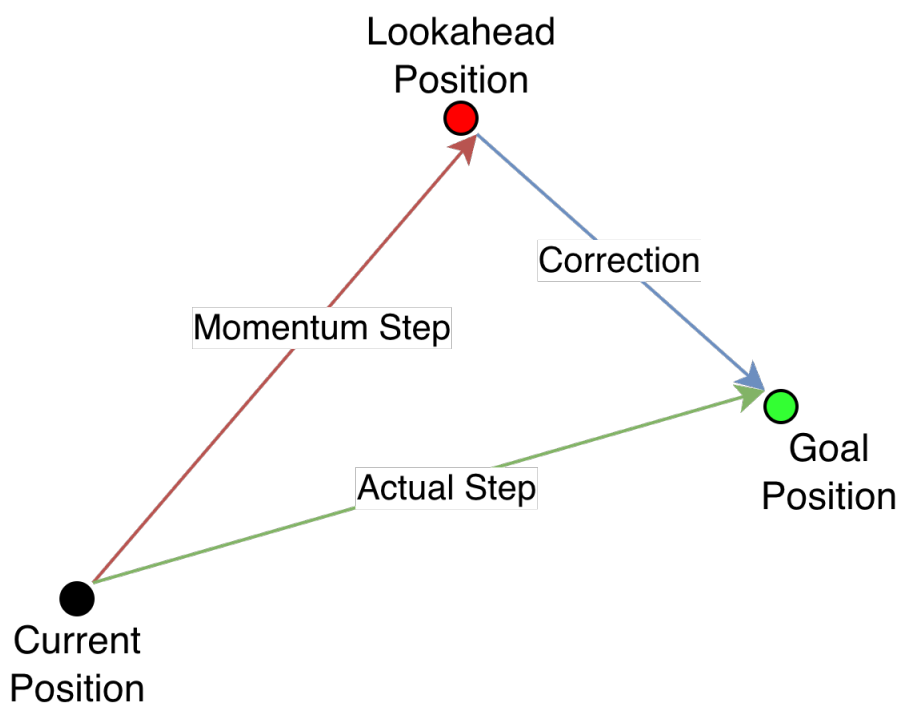


Figure 10: Principle behind Nesterov Momentum.

5.2.1 Convergence

Here, we borrow and expand on working from Liu, 2025, who presents a novel "Lyapunov Analysis" of *NAG*.

Theorem 8. *(NAG Final Iterate Proximity Bound) If $\mathcal{L}$ is $L-$smooth and convex, then Gradient Descent with the NAG update rule with $\eta \in (0, \frac{1}{L}], \beta_t = \frac{t-1}{t+r-1}$ where $r \geq 3$ admits the bound:*

$$\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*) \leq \frac{(r-1)^2 ||\boldsymbol{\theta}_1 - \boldsymbol{\theta}_*||_2^2}{2\alpha(t+r-2)^2}, t \geq 0$$

Proof. Define an acceleration function a where $a_0 = 0$ and $a_t = \frac{t+r-2}{r-1}$ for $t = 1, 2, , ...T$. This function scales linearly with t in that if we fix $r = 3$, then $a = \frac{t+1}{2}$. We can then rewrite the momentum constant in terms of this acceleration as,

$$\beta_t = \frac{\frac{t+r-2}{r-1} - 1}{\frac{t+r-1}{r-1}} = \frac{a_t - 1}{a_{t+1}}, t \geq 1$$

We can now define a potential point,

$$\begin{aligned} \boldsymbol{\phi}_t &:= (a_t - 1)(\boldsymbol{\theta}_t - \boldsymbol{\theta}_{t-1}) \\ &= a_{t+1}(\boldsymbol{W}_t - \boldsymbol{\theta}_t) \ (\textit{From Eqn. 10}) \end{aligned}$$

where $t \geq 1, \boldsymbol{\phi}_0 = 0$. Then, we have that

$$\begin{aligned}
\boldsymbol{\phi}_{t+1} + \boldsymbol{\theta}_{t+1} &= (a_{t+1} - 1)(\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t) + \boldsymbol{\theta}_{t+1} \ (\textit{Def. of } \phi) \\
&= a_{t+1}(\boldsymbol{\theta}_{t+1}) - a_{t+1}(\boldsymbol{\theta}_t) - \boldsymbol{\theta}_{t+1} + \boldsymbol{\theta}_t + \boldsymbol{\theta}_{t+1} \\
&= a_{t+1}(\boldsymbol{\theta}_{t+1}) - a_{t+1}(\boldsymbol{\theta}_t) + \boldsymbol{\theta}_t \\
&= a_{t+1}(\boldsymbol{W}_t - \eta \nabla f(\boldsymbol{W}_t)) - a_{t+1}(\boldsymbol{\theta}_t) + \boldsymbol{\theta}_t \ (\textit{From Eqn. 10}) \\
&= a_{t+1}(\boldsymbol{W}_t) - a_{t+1}\eta \nabla f(\boldsymbol{W}_t) - a_{t+1}(\boldsymbol{\theta}_t) + \boldsymbol{\theta}_t \\
&= a_{t+1}(\boldsymbol{W}_t - \boldsymbol{\theta}_t) + \boldsymbol{\theta}_t - a_{t+1}\eta \nabla f(\boldsymbol{W}_t) \\
&= \boldsymbol{\phi}_t + \boldsymbol{\theta}_t - a_{t+1}\eta \nabla f(\boldsymbol{W}_t) \ (\textit{Def. of } \phi)
\end{aligned}$$

And so we've given ourselves a way to project the effect ($= \phi^{t+1} + \boldsymbol{\theta}_{t+1}$) of *NAG* using information about acceleration (ϕ_t), location ($\boldsymbol{\theta}_t$), and gradient ($\eta \nabla f(\boldsymbol{W}_t)$) in the current time-step.

Naturally, we then want to measure how far-off our projection ($\phi^{t+1} + \boldsymbol{\theta}_{t+1}$) is from the minimizer $\boldsymbol{\theta}_*$.

$$\begin{aligned}
||\boldsymbol{\phi}_{t+1} + \boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_*||_2^2 &= ||\boldsymbol{\phi}_t + \boldsymbol{\theta}_t - a_{t+1}\eta \nabla f(\boldsymbol{W}_t) - \boldsymbol{\theta}_*||_2^2 \\
&= ||\boldsymbol{\phi}_t + \boldsymbol{\theta}_t - \boldsymbol{\theta}_*||_2^2 - \langle 2\eta a_{t+1}(\boldsymbol{\phi}_t + \boldsymbol{\theta}_t - \boldsymbol{\theta}_*), \nabla f(\boldsymbol{W}_t)\rangle \\
&+ (a_{t+1})^2\eta^2 ||\nabla f(\boldsymbol{W}_t)||_2^2 \\
&= ||\boldsymbol{\phi}_t + \boldsymbol{\theta}_t - \boldsymbol{\theta}_*||_2^2 \\
&- \langle 2\eta a_{t+1}(\boldsymbol{\phi}_t + (\boldsymbol{W}_t - \boldsymbol{\theta}_t) + (\boldsymbol{W}_t - \boldsymbol{\theta}_*)), \nabla f(\boldsymbol{W}_t)\rangle \\
&+ (a_{t+1})^2\eta^2 ||\nabla f(\boldsymbol{W}_t)||_2^2 \\
&= ||\boldsymbol{\phi}_t + \boldsymbol{\theta}_t - \boldsymbol{\theta}_*||_2^2 \\
&- \langle 2\eta a_{t+1}(a_{t+1}(\boldsymbol{W}_t - \boldsymbol{\theta}_t) + (\boldsymbol{W}_t - \boldsymbol{\theta}_t) + (\boldsymbol{W}_t - \boldsymbol{\theta}_*)), \nabla f(\boldsymbol{W}_t)\rangle \\
&+ (a_{t+1})^2\eta^2 ||\nabla f(\boldsymbol{W}_t)||_2^2 \\
&= ||\boldsymbol{\phi}_t + \boldsymbol{\theta}_t - \boldsymbol{\theta}_*||_2^2 \\
&- \langle 2\eta a_{t+1}((a_{t+1} - 1)(\boldsymbol{W}_t - \boldsymbol{\theta}_t) + (\boldsymbol{W}_t - \boldsymbol{\theta}_*)), \nabla f(\boldsymbol{W}_t)\rangle \\
&+ (a_{t+1})^2\eta^2 ||\nabla f(\boldsymbol{W}_t)||_2^2 \\
&= ||\boldsymbol{\phi}_t + \boldsymbol{\theta}_t - \boldsymbol{\theta}_*||_2^2 - \langle 2\eta a_{t+1}(a_{t+1} - 1)(\boldsymbol{W}_t - \boldsymbol{\theta}_t), \nabla f(\boldsymbol{W}_t)\rangle \\
&- \langle 2\eta a_{t+1}(\boldsymbol{W}_t - \boldsymbol{\theta}_*), \nabla f(\boldsymbol{W}_t)\rangle + (a_{t+1})^2\eta^2 ||\nabla f(\boldsymbol{W}_t)||_2^2
\end{aligned}$$

Using L-smoothness and convexity, we have that

$$\begin{aligned}
||\boldsymbol{\phi}_{t+1} + \boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_*||_2^2 &\leq ||\boldsymbol{\phi}_t + \boldsymbol{\theta}_t - \boldsymbol{\theta}_*||_2^2 - 2\eta a_{t+1}(a_{t+1} - 1)(\mathcal{L}(\boldsymbol{W}_t) - \mathcal{L}(\boldsymbol{\theta}_t)) \\
&- 2\eta a_{t+1}(\mathcal{L}(\boldsymbol{W}_t) - \mathcal{L}(\boldsymbol{\theta}_*)) + (a_{t+1})^2\eta^2 ||\nabla f(\boldsymbol{W}_t)||_2^2 \ (\textit{By convexity of } \mathcal{L}.) \\
&\leq ||\boldsymbol{\phi}_t + \boldsymbol{\theta}_t - \boldsymbol{\theta}_*||_2^2 - 2\eta a_{t+1}(a_{t+1})^2(\mathcal{L}(\boldsymbol{W}_t) - \mathcal{L}(\boldsymbol{\theta}_*)) \\
&+ 2\eta(a_t)^2(\mathcal{L}(\boldsymbol{\theta}_t)) - \mathcal{L}(\boldsymbol{\theta}_*) + (a_{t+1})^2\eta^2 ||\nabla f(\boldsymbol{W}_t)||_2^2 \\
&\leq ||\boldsymbol{\phi}_t + \boldsymbol{\theta}_t - \boldsymbol{\theta}_*||_2^2 - 2\eta a_{t+1}(a_{t+1})^2(\mathcal{L}(\boldsymbol{\theta}_{t+1}) - \mathcal{L}(\boldsymbol{\theta}_*)) \\
&+ 2\eta(a_t)^2(\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*)) \\
&+ (a_{t+1})^2\eta^2(-1 + L\eta)||\nabla f(\boldsymbol{W}_t)||_2^2 \ (\textit{By } L\textit{-smoothness of } \mathcal{L}.) \\
&\leq ||\boldsymbol{\phi}_t + \boldsymbol{\theta}_t - \boldsymbol{\theta}_*||_2^2 - 2\eta a_{t+1}(a_{t+1})^2(\mathcal{L}(\boldsymbol{\theta}_{t+1}) - \mathcal{L}(\boldsymbol{\theta}_*)) \\
&+ 2\eta(a_t)^2(\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*)) \ (\textit{By hypothesis that } \eta \leq \frac{1}{L}.)
\end{aligned}$$

Now, taking the final inequality,

$$\begin{aligned}
||\boldsymbol{\phi}_{t+1} + \boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_*||_2^2 &\leq ||\boldsymbol{\phi}_t + \boldsymbol{\theta}_t - \boldsymbol{\theta}_*||_2^2 - 2\eta a_{t+1}(a_{t+1})^2(\mathcal{L}(\boldsymbol{\theta}_{t+1}) - \mathcal{L}(\boldsymbol{\theta}_*)) \\
&+ 2\eta(a_t)^2(\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*))
\end{aligned}$$

We can rearrange this to express the recursive relationship:

$$\begin{aligned}
||\boldsymbol{\phi}_{t+1} + \boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_*||_2^2 &+ 2\eta a_{t+1}(a_{t+1})^2(\mathcal{L}(\boldsymbol{\theta}_{t+1}) - \mathcal{L}(\boldsymbol{\theta}_*)) \\
&\leq ||\boldsymbol{\phi}_t + \boldsymbol{\theta}_t - \boldsymbol{\theta}_*||_2^2 + 2\eta(a_t)^2(\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*))
\end{aligned}$$

Which helps assure us that,

$$2\eta(a_t)^2(\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*)) \leq ||\boldsymbol{\theta}_1 - \boldsymbol{\theta}_*||_2^2$$

Where $||\boldsymbol{\theta}_1 - \boldsymbol{\theta}_*||_2^2$ is obtained by considering $||\boldsymbol{\phi}_t + \boldsymbol{\theta}_t - \boldsymbol{\theta}_*||_2^2 + 2\eta(a_t)^2(\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*))$ at ϕ_0, a_0.

Manipulating the inequality $2\eta(a_t)^2(\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*)) \leq ||\boldsymbol{\theta}^0 - \boldsymbol{\theta}_*||_2^2$ gives us,

$$\begin{aligned}\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*) &\leq \frac{||\boldsymbol{\theta}_1 - \boldsymbol{\theta}_*||_2^2}{2\eta(a_t)^2} \\ &= \frac{(r-1)^2||\boldsymbol{\theta}_1 - \boldsymbol{\theta}_*||_2^2}{2\eta(t+r-2)^2} \text{ (By definition of } a.)\end{aligned}$$

Which is the statement of the theorem[21]. □

Corollary 5. *(NAG Convergence Rate) Picking* $\eta = \frac{1}{L}, r = 3, \beta_t = \frac{T-1}{T+2}$ *gives us the bound*

$$\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*) = \frac{2L||\boldsymbol{\theta}_1 - \boldsymbol{\theta}_*||_2^2}{(T+1)^2}, t \geq 0$$

after T *iterations. Which corresponds to a* $\frac{1}{T^2}$ *convergence rate for NAG.*

As demonstrated by *Corollary 4*, *NAG* promises an accelerated $\frac{1}{T^2}$ convergence rate compared to the $\frac{1}{T}$ rate for *Fixed-*η *Descent*, and the slower $\frac{1}{\sqrt{T}}$ rate for *SGD*.

[21] *Lyapunov Analysis* employs a non-decreasing scalar function, in this case $||\boldsymbol{\phi}_t + \boldsymbol{\theta}_t - \boldsymbol{\theta}_*||_2^2 + 2\eta(a_t)^2(\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*))$, to represent the energy of a nonlinear dynamical system to prove local or global asymptoptic stability for the system.

6 Adaptive Gradient

6.1 A History of Gradients

In the previous chapter, we varied the standard gradient descent update rule to incorporate the concepts of momentum and lookahead correction. Specifically, we looked at not just the current position $\boldsymbol{\theta}_t$ to inform the future position $\boldsymbol{\theta}_{t+1}$, but the previous position $\boldsymbol{\theta}_{t-1}$ too.

However, notice that we still only use the gradient at 1 point in time. In the standard gradient descent *Algorithm 1*, it is the gradient at the current position $\nabla\mathcal{L}(\boldsymbol{\theta}_t)$. In the stochastic version *Algorithm 3*, it is the average gradient $\hat{\nabla}\mathcal{L}$ of many samples $x_1, ..., x_b$ at the same (current) position t. In the accelerated version *Algorithm 5*, it is the gradient at the lookahead position $\mathcal{L}(\boldsymbol{W})$ relative to the current time t.

What if we adapted our gradient just as we adapted our η when exploring Quicker Convergence, or as we adapted our $\boldsymbol{\theta}$ update when exploring momentum? This chapter therefore introduces the concept of **Adaptive Gradient**, which takes into account gradient history when performing updates.

To help with the analysis for the 2 key Adaptive Gradient algorithms introduced here, we consulted Basu et al., 2018.

6.2 RMSProp

Root-Mean-Square Propagation, or *RMSProp*, is an adaptive-gradient method that conditions the update of $\boldsymbol{\theta}_{t+1}$ on all past gradients from steps $1...t$ using 2 key concepts: the **Diagonal Preconditioner**, and the **Exponential Moving Average**. This method is illustrated in *Algorithm 6*:

Algorithm 6 Root-Mean-Square Propagation

Input: $\mathcal{L}, \boldsymbol{\theta}_1, \eta, \beta, \xi, T$
Output: $\boldsymbol{\theta}_*$

$final_\theta = \theta_1$
$w_0 = 0$
for $t := 1$ to T **do**
  $\eth_t := \nabla\mathcal{L}(\boldsymbol{\theta}_t)$
  $w_t = (\beta)w_{t-1} + (1-\beta)(\eth_t^2 + \xi)$
  $\boldsymbol{W}_t = \text{diag}(w_t)$
  $\boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta \boldsymbol{W}_t^{-\frac{1}{2}} \eth_t$
  $final_{\boldsymbol{\theta}} = \boldsymbol{\theta}_{t+1}$
end for

$\boldsymbol{\theta}_t = final_{\boldsymbol{\theta}}$
return $\boldsymbol{\theta}_t$

How does this differ from *NAG*?

- In *NAG*, the new hyperparameter $\beta \in [0, 1)$ represented an adaptive (i.e. t-dependant) "mass" or velocity dial that decided how much the immediate previous velocity $\boldsymbol{\theta}_t - \boldsymbol{\theta}^{t-1}$ should matter when computing $\boldsymbol{\theta}_{t+1}$. Here, $\beta \in [0, 1)$ is a fixed smoothing term, meant to assign a relative importance between <u>all</u> past terms and the present term[22].
- In *NAG*, the intermediary weights $\boldsymbol{W}_t$ represented the "lookahead" point based on current momentum. Here, <u>$\boldsymbol{W}_t$ becomes a diagonal preconditioner when updating $\eth_t$</u>. This is exactly the "adaptive gradient" part of *RMSProp*: the vector w_t encapsulates gradient history, and this vector is converted into a matrix $\boldsymbol{W}_t$ which performs per-dimension scaling of the gradient $\eth_t$ during update through use of the **Penrose Inverse** $\boldsymbol{W}_t^{-\frac{1}{2}}$[23].
- A new slack vector ξ, that provides some buffer in case the gradient $\eth_t$ is 0.

[22] The idea is inspired by that of the Exponential Moving Average, commonly used in stock pricing and queueing theory. The EMA is a linear combination of a present point with all past points, but unlike a simple average it assigns exponentially smaller weight to points that have been passed further in the past. The Exponential Moving Average is a recursive formula, $EMA_t = \alpha EMA^{t-1} + (1-\alpha)x$. Here, $\alpha := \beta, EMA_t = w_t, x = \eth_t + \xi$.

[23] The Penrose Inverse is defined for $w \in \mathbb{R}^d$ and $\boldsymbol{W} = \text{diag}(w)$ as $\boldsymbol{W}^{-\frac{1}{2}} := \sum_{i \in \text{Support}(w)} \frac{1}{\sqrt{w_i}} e_i (e_i)_t$ where $\{e_i\}_{i=1,...,d}$ is the standard basis of $\mathbb{R}^d$. In words: the inverse square root of a diagonal matrix is the inverse square root of its non-zero diagonal entries

6.2.1 Convergence

Let's explore some convergence for *RMSProp*. However, unlike for *NAG* wherein we bounded the final proximity term for a deterministic version of *NAG*, here let's focus on the stochastic version of *RMSProp* and bound the gradient norm (similar to Theorem 6).

Theorem 9. *(Stochastic RMSProp Gradient Norm Bound) If $\mathcal{L}$ is $L-$smooth, then Algorithm 6 with $||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2^2 = \eth_t \leq G$ for any $t = 1, ..., T$, and $\eta = \frac{1}{\sqrt{T}}\sqrt{\frac{2\xi(1-\beta)(\mathcal{L}(\boldsymbol{\theta}_t)-\mathcal{L}(\boldsymbol{\theta}_*))}{G^2 L}}$ admits the bound:*

$$\min_{1\ldots t} ||\mathbb{E}[\nabla\mathcal{L}(\boldsymbol{\theta}_t)]||_2^2 \leq \frac{1}{\sqrt{T}}\sqrt{\frac{2(\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*))(G^2 L)(G^2+\xi)}{(1-\beta)\xi}}$$

Proof. Define $G_k := max_{k=1\ldots t}\eth_t = ||\hat{\nabla}\mathcal{L}(\boldsymbol{\theta}_k)||_2^2$[24].

Expand the recursive update for $w_t = (\beta)w^{t-1} + (1-\beta)(\eth_t^2 + \xi)$ over $k = 1, ..., t$ to obtain:

$$w_t = (1-\beta)\sum_{k=1}^{t}\beta^{t-k}(\eth_k^2 + \xi). \tag{11}$$

Now, we have the lower bound[25],

$$\begin{aligned}
\lambda_{\min}(\boldsymbol{W}_t^{-\frac{1}{2}}) &\geq \frac{1}{\sqrt{\max_{i=1,\ldots,d}(w_t)_i}} \quad (\textit{Eigenvalue of a Diagonal Matrix}) \\
&\geq \frac{1}{\sqrt{\max_{i=1,\ldots,d}((1-\beta)\sum_{k=1}^{t}\beta^{t-k}(\eth_k^2+\xi))_i}} \quad (\textit{Substituting Eqn. 11}) \\
&\geq \frac{1}{\sqrt{(1-\beta)\sum_{k=1}^{t}\beta^{t-k}(G_k^2+\xi)}} \quad (\textit{By the defined bound above}) \\
&= \frac{1}{\sqrt{(1-\beta)(G_k^2+\xi)\sum_{k=1}^{t}\beta^{t-k}}} \\
&= \frac{1}{\sqrt{(1-\beta)(G_k^2+\xi)\frac{1-\beta_t}{1-\beta_t}}} \quad (\textit{By GP summation}) \\
&= \frac{1}{\sqrt{(1-\beta_t)(G_k^2+\xi)}}
\end{aligned}$$

Then, we define $\varepsilon_t := min_{k=1,\ldots,t,i=1,\ldots,d}(\eth_t)_i^2$. Now, we get the upper bound using a very similar derivation[26],

$$\begin{aligned}
\lambda_{max}(\boldsymbol{W}_t^{-\frac{1}{2}}) &\leq \frac{1}{min_{i=1,\ldots,d}(\sqrt{(w_t)_i}} \\
&\leq \frac{1}{\sqrt{(1-\beta_t)}\sqrt{(\xi+\varepsilon_t)}}
\end{aligned}$$

Now, we introduce worst-case estimates, $\mu_{\min}$ and $\mu_{\max}$:

$$\begin{aligned}
\lambda_{\min}(\boldsymbol{W}_t^{-\frac{1}{2}}) &\geq \frac{1}{\sqrt{G^2+\xi}} \quad (\textit{The global bound } G^2 \geq G_k^2 \textit{ which is specific to step } k) \\
&:= \mu_{\min} \\
\lambda_{\max}(\boldsymbol{W}_t^{-\frac{1}{2}}) &\leq \frac{1}{\sqrt{(1-\beta)\sqrt{\xi}}} \\
&:= \mu_{\max}
\end{aligned}$$

[24]Recall that for stochastic gradient descent algorithms, we work with a noisy estimator $\hat{\nabla}$ of the gradient. This estimator is in conditional expectation $\mathbb{E}$, equal to the true gradient ∇.

[25]The value of the final bound $\lambda_{min}(\boldsymbol{W}_t^{-\frac{1}{2}}) \geq \frac{1}{\sqrt{(1-\beta_t)(G_k+\xi)}}$ is that we are able to establish a minimum factor by which we scale-down the gradient $\eth_t$. This, of-course, affects the magnitude of the update for $\boldsymbol{\theta}_{t+1}$.

[26]Similarly, the value of this bound is establishing a maximum factor by which we scale-up the gradient at the update-step.

Next, we're going to start from a very similar process as we had used when analysing the convergence of the gradient norm of *SGD*.

As before, we invoke the L-smoothness guarantee:

$$\mathcal{L}(\boldsymbol{\theta}_{t+1}) \leq \mathcal{L}(\boldsymbol{\theta}_t) + \langle \nabla \mathcal{L}(\boldsymbol{\theta}_t), \boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t \rangle + \frac{L}{2}||\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t||_2^2$$

Taking the expectation conditioned over steps $1, ..., t$ on both sides,

$$\begin{aligned}
\mathbb{E}_t[\mathcal{L}(\boldsymbol{\theta}_{t+1})] &\leq \mathcal{L}(\boldsymbol{\theta}_t) - \eta \mathbb{E}_t[\langle \nabla \mathcal{L}(\boldsymbol{\theta}_t), (\boldsymbol{W}_t)^{-\frac{1}{2}} \eth_t \rangle] + \frac{\eta^2 L}{2} \mathbb{E}_t[||(\boldsymbol{W}_t)^{-\frac{1}{2}} \eth_t||_2^2] \\
&\leq \mathcal{L}(\boldsymbol{\theta}_t) - \eta \mathbb{E}_t[\langle \nabla \mathcal{L}(\boldsymbol{\theta}_t), (\boldsymbol{W}_t)^{-\frac{1}{2}} \eth_t \rangle] + \mu_{\max}^2 \frac{\eta^2 L}{2} \mathbb{E}_t[||\eth_t||_2^2] \quad (\textit{By definition of } \mu_{\max}) \\
&\leq \mathcal{L}(\boldsymbol{\theta}_t) - \eta \mathbb{E}_t[\langle \nabla \mathcal{L}(\boldsymbol{\theta}_t), (\boldsymbol{W}_t)^{-\frac{1}{2}} \eth_t \rangle] + \mu_{\max}^2 \frac{\eta^2 G^2 L}{2} \quad (\textit{By definition of } G) \qquad (12)
\end{aligned}$$

Lemma 3. *We can be assured that the update made by RMSProp, $(\boldsymbol{W}_t)^{-\frac{1}{2}} \eth_t$ aligns with the true descent $\nabla \mathcal{L}(\boldsymbol{\theta}_t)$, by a predefined minimum factor:*

$$\eta \mathbb{E}_t[\langle \nabla \mathcal{L}(\boldsymbol{\theta}_t), (\boldsymbol{W}_t)^{-\frac{1}{2}} \eth_t \rangle] \geq \mu_{\min} ||\nabla \mathcal{L}(\boldsymbol{\theta}_t)||_2^2$$

Substituting the above[27] into *Eqn. 12* and rearranging, we have:

$$\begin{aligned}
\mathbb{E}_t[\mathcal{L}(\boldsymbol{\theta}_{t+1}) - \mathcal{L}(\boldsymbol{\theta}_t)] &\leq -\eta \mu_{\min} ||\nabla \mathcal{L}(\boldsymbol{\theta}_t)||_2^2 + \mu_{\max}^2 \frac{\eta^2 G^2 L}{2} \\
&\iff \mathbb{E}[||\mathcal{L}(\boldsymbol{\theta}_t)||_2^2] \leq \frac{1}{\eta \mu_{\min}} \mathbb{E}[\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_{t+1})] + \frac{\mu_{\max}^2}{\mu_{\min}} \frac{\eta G^2 L}{2} \quad (\textit{Law of Total Expectation})
\end{aligned}$$

Summing this inquality over T steps and taking the average, we get:

$$\frac{1}{T} \sum_{t=1}^{t} \mathbb{E}[||\mathcal{L}(\boldsymbol{\theta}_t)||_2^2] \leq \frac{1}{T \eta \mu_{\min}} \sum_{t=1}^{t} \mathbb{E}[\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_{t+1})] + \frac{\mu_{\max}^2}{\mu_{\min}} \frac{\eta G^2 L}{2}$$

Taking the lower bound of the L.H.S.,

$$\begin{aligned}
\min_{t=1,...,T} \mathbb{E}[||\mathcal{L}(\boldsymbol{\theta}_t)||_2^2] &\leq \frac{1}{T \eta \mu_{\min}} \sum_{t=1}^{t} \mathbb{E}[\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_{t+1})] + \frac{\mu_{\max}^2}{\mu_{\min}} \frac{\eta G^2 L}{2} \\
&\leq \frac{1}{T \eta \mu_{\min}} \mathbb{E}[\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_{t+1})] + \frac{\mu_{\max}^2}{\mu_{\min}} \frac{\eta G^2 L}{2} \\
&\leq \frac{1}{T \eta \mu_{\min}} (\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*)) + \frac{\mu_{\max}^2}{\mu_{\min}} \frac{\eta G^2 L}{2}
\end{aligned}$$

Given the optimal choice $\eta = \frac{1}{\sqrt{T}} \sqrt{\frac{2\xi(1-\beta)(\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*))}{G^2 L}}$ by hypothesis,

$$\begin{aligned}
\min_{t=1,...,T} \mathbb{E}[||\mathcal{L}(\boldsymbol{\theta}_t)||_2^2] &\leq 2 \sqrt{\frac{1}{T \mu_{\min}} (\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*)) \times \frac{\mu_{\max}^2}{\mu_{\min}} \frac{G^2 L}{2}} \quad (\textit{AM-GM Inequality}) \\
&= 2 \sqrt{\frac{(\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*))(G^2 L)(G^2 + \xi)}{(1-\beta)\xi}} \\
&= \frac{1}{\sqrt{T}} \sqrt{\frac{2(\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*))(G^2 L)(G^2 + \xi)}{(1-\beta)\xi}}
\end{aligned}$$

which completes the proof. □

[27] Proof of the lemma can be found in Basu et al., 2018.

Corollary 6. *(RMSProp Gradient Norm Convergence Rate) Picking* $\eta = \frac{1}{\sqrt{t}}\sqrt{\frac{2\xi(1-\beta)(\mathcal{L}(\boldsymbol{\theta}_t)-\mathcal{L}(\boldsymbol{\theta}_*))}{G^2 L}}$ *and* $T \leq \frac{1}{\epsilon^4}\left(\frac{2(\mathcal{L}(\boldsymbol{\theta}_t)-\mathcal{L}(\boldsymbol{\theta}_*))G^2 L(G^2+\xi)}{(1-\beta)\xi}\right)$ *ensures that within* T *iterations, at least one* $t = 1...T$ *is* ϵ*-critical such that:*

$$\mathbb{E}[||\mathcal{L}(\boldsymbol{\theta}_t)||_2^2] \leq \epsilon^2$$

which indicates that Stochastic RMSProp has a $\frac{1}{\sqrt[4]{T}}$ *convergence rate.*

Remark 4. In all other algorithms so far, setting η was fairly practical in that it depended on constants such as T, L and G that were perhaps a little random (for T) or tedious to calculate (for L, G) but not unknowable. However, in the case of *RMSProp*, η involves the term $\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_*)$. The difficulty with setting such an η is obvious: $\mathcal{L}(\boldsymbol{\theta}_*)$ is exactly the value we're optimizing for! So, how to use it in the process of optimizing it?

The thing to note here is that *Theorem 9* is a theoretical bound, meant to demonstrate that *RMSProp* can converge[a] under some reasonable conditions. In practice, $\mathcal{L}(\boldsymbol{\theta}_*)$ is either assumed to be 0 or a variety of η are tested using a Grid Search technique.

Thus, the more important aspect of defining $\eta = \frac{1}{\sqrt{T}}\sqrt{\frac{2\xi(1-\beta)(\mathcal{L}(\boldsymbol{\theta}_t)-\mathcal{L}(\boldsymbol{\theta}_*))}{G^2 L}}$ is the $\frac{1}{\sqrt{T}}$ factor. We can easily view η as $\frac{\Phi}{\sqrt{T}}$, where Φ may be any tunable parameter but $\sqrt{T}$ is still the convergence rate that remains the same.

[a] This was also the first mention of "ϵ-criticality" in relation to convergence. This term simply formalizes the concept, already seen before, that at a large enough T, the convergence term (which depends on T), will approach optimality with only some tiny ϵ-difference.

6.3 Adam

Adaptive Moment Estimation, *Adam*, is another adaptive-gradient method that combines both *Momentum* and *RMSProp*.

Algorithm 7 Adaptive Moment Estimation

Input: $\mathcal{L}, \boldsymbol{\theta}_1, \eta, \beta_1, \beta_2, \xi, T$
Output: $\boldsymbol{\theta}_*$

1: $final_\theta = \boldsymbol{\theta}_1$
2: $w_0 = 0$
3: $v_0 = 0$
4: **for** $t := 1$ to T **do**
5: $\quad \eth_t := \nabla\mathcal{L}(\boldsymbol{\theta}_t)$
6: $\quad v_t = (\beta_1)v_{t-1} + (1-\beta_1)\eth_t$
7: $\quad w_t = (\beta_2)w_{t-1} + (1-\beta_2)\eth_t^2$
8: $\quad \boldsymbol{W}_t = \text{diag}(w_t)$
9: $\quad \boldsymbol{\theta}_{t+1} = \boldsymbol{\theta}_t - \eta(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t$
10: $\quad final_{\boldsymbol{\theta}} = \boldsymbol{\theta}_{t+1}$
11: **end for**

12: $\boldsymbol{\theta}_t = final_{\boldsymbol{\theta}}$
13: return $\boldsymbol{\theta}_t$

Key points about the algorithm:

- The similarities with *Algorithm 6* are obvious:
  - We still use an intermediary weights w that represents an *Exponential Moving Average*, to construct a *Diagonal Preconditoner* $\boldsymbol{W}$ that is used for scaling during $\boldsymbol{\theta}$ update.
  - We still use a slack variable $\xi > 0$ that provides some buffer for a zeroed-out preconditioner.
- The differences are:
  - We no longer scale the gradient $\eth$ directly. Rather, we modify the gradient (*Line 6*) to produce an additional intermediary weights v which represents a separate *Exponential Moving Average*.

– There are now two separate smoothing parameters β_1, β_2 each for v and w.

The intuition behind *Adam* is simple: instead of using the gradient history, $\boldsymbol{W}$ to scale the gradient, $\eth$, directly (which assumes the gradient is pointing in the correct direction in all dimensions), we use it to scale the direction-informed gradient, v. The key difference is in the $\eth^2$ used to construct w versus the $\eth$ used for v (when squaring the gradient, direction is ignored since only scale matters). Some of this is helped, of-course, by good settings for the hyperparameters β_1, β_2; i.e. the model is allowed to test during Deep Learning which values of β_1, β_2 work best to balance scale with direction.

6.3.1 Convergence

Theorem 10. *(Adam Gradient Norm Bound) If $\mathcal{L}$ is $L-$smooth, then Algorithm 7 with $||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2^2 = \eth_t \leq G$ for any $t = 1, ..., T$, $\beta_1 = \frac{\epsilon}{\epsilon+G}$, $\xi = 2G$, and $\eta = \frac{||\eth_t||_2^2}{L(1-\beta_{1_t})^2}\frac{4\epsilon}{3(\epsilon+2G)^2}$ admits the bound:*

$$||\nabla\mathcal{L}(\boldsymbol{\theta}_t)||_2 \leq \epsilon$$

for some $t \leq T$.

Proof. Once again, we start from the L-smoothness upper-bound and substitute $\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t$ for $(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t$ as per the *Adam* update rule (Line 9 of *Algorithm 7*):

$$\begin{aligned}
\mathcal{L}(\boldsymbol{\theta}_{t+1}) &\leq \mathcal{L}(\boldsymbol{\theta}_t) + \langle\nabla\mathcal{L}(\boldsymbol{\theta}_t), \boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t\rangle + \frac{L}{2}||\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t||_2^2 \\
&\leq \mathcal{L}(\boldsymbol{\theta}_t) - \eta_t\langle\nabla\mathcal{L}(\boldsymbol{\theta}_t), (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t\rangle + \frac{L\eta_t^2}{2}||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t||_2^2 \\
&\iff \mathcal{L}(\boldsymbol{\theta}_{t+1}) - \mathcal{L}(\boldsymbol{\theta}_t) \leq \eta_t(-\langle\eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t\rangle + \frac{L\eta_t}{2}||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t||_2^2)
\end{aligned}$$

Notice that the R.H.S. in the final derivation above is a quadratic expression in η_t:

$$\frac{L\eta_t^2}{2}||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t||_2^2 - \eta_t(\langle\eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t\rangle)$$

By inspection, the roots of this quadratic are 0, $\frac{\langle\eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t\rangle}{\frac{L\eta_t^2}{2}||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t||_2^2}$. This gives us a minimizer at the midpoint:

$$\frac{1}{2}\frac{\langle\eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t\rangle}{\frac{L}{2}||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t||_2^2}$$

, with the value of the quadratic[28]:

$$-\frac{1}{4}\frac{(\langle\eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t\rangle)^2}{\frac{L}{2}||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t||_2^2}$$

This minimizer is now a good candidate for η_t. That is, with $\eta_t := \frac{1}{2}\frac{\langle\eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t\rangle}{\frac{L}{2}||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t||_2^2}$, we can guarantee:

$$\begin{aligned}
\mathcal{L}(\boldsymbol{\theta}_{t+1}) - \mathcal{L}(\boldsymbol{\theta}_t) &\leq -\frac{1}{4}\frac{(\langle\eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t\rangle)^2}{\frac{L}{2}||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t||_2^2} \\
&= -\frac{1}{2L}\frac{(\langle\eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t\rangle)^2}{||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t||_2^2}
\end{aligned}$$

The goal now is to maximize $\mathcal{L}(\boldsymbol{\theta}_{t+1}) - \mathcal{L}(\boldsymbol{\theta}_t)$, and to do so we want to maximize $-\frac{1}{2L}\frac{(\langle\eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t\rangle)^2}{||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t||_2^2}$. Let us now place an upperbound on $||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t||_2^2$ and a lowerbound on $(\langle\eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}v_t\rangle)^2$.

..... × .....

[28] Recall that for a quadratic $f = a(x_1)^2 + b(x_2)$, its value f at $x_1 = x_2 = -\frac{b}{2a}$ is $f = a(-\frac{b}{2a})^2 + b(-\frac{b}{2a}) = -\frac{b^2}{4a}$.

To upperbound $||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t||_2^2$, we start with:

$$\lambda_{\max}((\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}) \leq \frac{1}{\xi + \min_{i=1,\dots,d} \sqrt{(w_t)_i}}$$

As with the proof of *Theorem 10*, we can modify the above upperbound by expanding the recursive w_t as $w_t = (1 - \beta_2) \sum_{k=1}^{t} \beta_2^{t-k} \eth_k^2$ and defining $\varepsilon_t = \min_{k=1,\dots,t, i=1,\dots,d}(\eth_k^2)$:

$$\lambda_{\max}((\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}) \leq \frac{1}{\xi + \sqrt{1 - \beta_2^t)\varepsilon_t}} \tag{13}$$

There is also now the matter of v_t to factor in, when considering the full expression $||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t||_2^2$. As with w_t, we can expand the recursion $v_t = (1 - \beta_1) \sum_{k=1}^{t} \beta_1^{t-k} \eth_k$. And as with *Theorem 10*, we define an iterate-specific bound on the gradient norm $G_t = \max_{i=1,\dots t} ||\eth_t||_2^2$. Then,

$$\begin{aligned}
||v_t||_2 &\leq ||(1 - \beta_1) \sum_{k=1}^{t} \beta_1^{t-k} \eth_k||_2 \quad (\textit{Generalized Triangle Inequality}) \\
&\leq (1 - \beta_1) \sum_{k=1}^{t} \beta_1^{t-k} ||\eth_k||_2 \\
&\leq (1 - \beta_1) G_t \sum_{k=1}^{t} \beta_1^{t-k} \\
&\leq (1 - \beta_1^t) G_t \quad (\textit{GP Summation})
\end{aligned}$$

So, with the norm bound $||v_t||_2 \leq (1 - \beta_1^t) G_t$, we can modify the bound in *Eqn. 13* as:

$$\begin{aligned}
||((\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}) v_t|| &\leq \frac{(1 - \beta_1^t) G_t}{\xi + \sqrt{1 - \beta_2^t)\varepsilon_t}} \\
&\leq \frac{G_t (1 - \beta_1^t)}{\xi}
\end{aligned} \tag{14}$$

..... × .....

To lowerbound $(\langle \eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t \rangle)^2$, we start with defining a function that indexes over the v_t:

$$Q_i = \langle \eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_i \rangle$$

Then we have,

$$\begin{aligned}
Q_i - \beta_1 (Q_{i-1}) &= \langle \eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} (v_i - \beta_1 v_{i-1}) \rangle \quad (\textit{Linearity of Inner Product}) \\
&= (1 - \beta_1) \langle \eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} \eth_i \rangle \quad (\textit{Adam Line 6})
\end{aligned}$$

Since the inner product in the L.H.S, $\langle \eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} \eth_i \rangle$, can be expressed using the quadratic form $\eth_t^T (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} \eth_t$ and $\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}$ is symmetric, we have the lower-bound

$$Q_t - \beta_1 (Q_{t-1}) \geq (1 - \beta_1) ||\eth_t||_2^2 \lambda_{\min} (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}$$

for some arbitrary $i = t$.

If we define $G_{t-1} := \max_{i=1,\dots,t-1} ||\eth_t||$ which gives us for $i \in \{1, ..., t-1\}$:

$$Q_t - \beta_1 (Q_{t-1}) \geq -(1 - \beta_1) ||\eth_t|| G_{t-1} \lambda_{\max} (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}$$

Constructing the recurrence:

$$Q_t - \beta_1^t Q_0 = (Q_t - \beta_1 (Q_{t-1})) + \beta_1 (Q_{t-1} - \beta_1 (Q_{t-2})) + \beta_1^2 (Q_{t-2} - \beta_1 (Q_{t-3})) + ... + \beta_1^{t-1} (Q_1 - \beta_1 Q_0)$$

Using the bounds established before,

$$
\begin{aligned}
Q_t - \beta_1^t Q_0 &\geq (1-\beta_1)||\eth_t||_2^2 \lambda_{\min}((\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}) \\
&- (1-\beta_1)||\eth_t||_2 G_{t-1} \lambda_{\min}((\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}) \sum_{k=1}^{t-1} \beta_i^k \\
&\geq (1-\beta_1)||\eth_t||_2^2 \lambda_{\min}((\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}) \\
&- (\beta_1 - \beta_1^t)||\eth_t||_2 G_{t-1} \lambda_{\min}((\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1})
\end{aligned} \tag{15}
$$

As before, we have $\lambda_{\min}((\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}) \geq \frac{1}{\xi + \sqrt{\max_{i=1,\ldots,d}(w_t)_i}}$, $w_t = (1-\beta_2)\sum_{k=1}^t \beta_2^{t-k} \eth_k^2$, and $G_t := \max_{i=1,\ldots,t}||\eth_i||$. We can use these to establish:

$$
\lambda_{\min}((\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1}) \geq \frac{1}{\xi + \sqrt{(1-\beta_2^t)G_t^2}}
$$

We can then use this result in combination with *Eqn. 13* and *Eqn. 15* to establish:

$$
\begin{aligned}
Q_t &\geq (1-\beta_1)||\eth_t||_2^2 \frac{1}{\xi + \sqrt{(1-\beta_2^t)G_t^2}} - (\beta_1 - \beta_1^t)||\eth_t||_2 G_{t-1} \frac{1}{\xi + \sqrt{(1-\beta_2^t)\varepsilon_t}} \\
&\geq ||\eth_t||_2^2 (\frac{(1-\beta_1)}{\xi + G\sqrt{(1-\beta_2^t)}} - \frac{(\beta_1 - \beta_1^t)G}{\xi ||\eth_t||_2})
\end{aligned} \tag{16}
$$

where the looser bound emerges from setting $\varepsilon = 0$ and $G_{t-1} = G_t = G$

Further, we can rearrange $\frac{(1-\beta_1)}{\xi + G\sqrt{(1-\beta_2^t)}} - \frac{(\beta_1 - \beta_1^t)G}{\xi ||\eth_t||_2}$:

$$
\begin{aligned}
\frac{(1-\beta_1)}{\xi + G\sqrt{(1-\beta_2^t)}} - \frac{(\beta_1 - \beta_1^t)G}{\xi ||\eth_t||_2} &= \frac{\xi ||\eth_t||_2 (1-\beta_1) - (\xi + G\sqrt{(1-\beta_2^t)})(\beta_1 - \beta_1^t)G}{(\xi + G\sqrt{(1-\beta_2^t)})\xi ||\eth_t||_2} \\
&= (\beta_1 - \beta_1^t)G \frac{\xi(\frac{||\eth_t||_2(1-\beta_1)}{(\beta_1 - \beta_1^t)G} - 1) - G\sqrt{(1-\beta_2^t)})}{(\xi + G\sqrt{(1-\beta_2^t)})\xi ||\eth_t||_2} \\
&= G(\beta_1 - \beta_1^t)(\frac{||\eth_t||_2(1-\beta_1)}{G(\beta_1 - \beta_1^t)} - 1) \frac{\xi - (\frac{G\sqrt{(1-\beta_2^t)}}{\frac{||\eth_t||_2(1-\beta_1)}{(\beta_1 - \beta_1^t)G} - 1})}{(\xi + G\sqrt{(1-\beta_2^t)})\xi ||\eth_t||_2}
\end{aligned} \tag{17}
$$

We can simplify this expression by introducing some new constants. First, let us focus on the term $\frac{||\eth_t||_2(1-\beta_1)}{G(\beta_1 - \beta_1^t)} - 1$. Define $\phi_1 > 0$ such that:

$$
\begin{aligned}
\phi_1 &:= \frac{\epsilon(1-\beta_1)}{G\beta_1} - 1 \\
&\leq \frac{||\eth_t||_2(1-\beta_1)}{G(\beta_1 - \beta_1^t)} - 1
\end{aligned}
$$

Now, focusing on $\frac{G\sqrt{(1-\beta_2^t)}}{\frac{||\eth_t||_2(1-\beta_1)}{(\beta_1 - \beta_1^t)G} - 1}$, define $\phi_2 > 0$ such that:

$$
\begin{aligned}
\frac{G\sqrt{(1-\beta_2^t)}}{\frac{||\eth_t||_2(1-\beta_1)}{(\beta_1 - \beta_1^t)G} - 1} &< \frac{G}{\phi_1} \\
&= \xi - \phi_2
\end{aligned}
$$

Substituting ϕ_1, ϕ_2 back into (17) we thus get a simplified bound for *Eqn. 16*[29]:

$$
\begin{aligned}
Q_t &\geq ||\eth_t||_2^2 (\frac{G(\beta_1 - \beta_1^2)\phi_1\phi_2}{(\xi + G)\xi G}) \\
&= ||\eth_t||_2^2 (\frac{(\beta_1 - \beta_1^2)\phi_1\phi_2}{(\xi + G)\xi})
\end{aligned} \tag{18}
$$

[29]This bound was further loosened by substituting $(\beta_1 - \beta_1^t)$ in the fraction $(\frac{(\beta_1 - \beta_1^t)\phi_1\phi_2}{(\xi+G)\xi}$ with the smaller $(\beta_1 - \beta_1^2)$ term.

Lastly, for clarity, we expand the definition of Q_t:

$$Q_t = \langle \eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t \rangle \geq ||\eth_t||_2^2 (\frac{(\beta_1 - \beta_1^t)\phi_1\phi_2}{(\xi + G)\xi})$$

which is exactly a lowerbound for $\langle \eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t \rangle$, as was our goal!

..... × .....

Now we return to the expression we started with:

$$\mathcal{L}(\boldsymbol{\theta}_{t+1}) - \mathcal{L}(\boldsymbol{\theta}_t) \leq \frac{1}{2L} \frac{(\langle \eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t \rangle)^2}{||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t||_2^2}$$

And we substitute the upper and lower bounds we established in *Eqn. 14* and *Eqn. 18* to get:

$$\begin{aligned}
\mathcal{L}(\boldsymbol{\theta}_{t+1}) - \mathcal{L}(\boldsymbol{\theta}_t) &\leq -\frac{1}{2L} \frac{(\langle \eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t \rangle)^2}{||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t||_2^2} \\
&\leq -\frac{1}{2L} \frac{Q_t^2}{||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t||_2^2} \\
&\leq -\frac{1}{2L} \frac{||\eth_t||_2^4 (\frac{(\beta_1 - \beta_1^2)\phi_1\phi_2}{(\xi+G)\xi})^2}{(\frac{G(1-\beta_1^t)}{\xi})^2} \\
&\leq -\frac{||\eth_t||_2^4}{2L} \frac{(\beta_1 - \beta_1^2)^2 \phi_1^2 \phi_2^2}{(\xi + G)^2 G^2 (1 - \beta_1^t)^2}
\end{aligned}$$

From this, we get that:

$$\begin{aligned}
\mathcal{L}(\boldsymbol{\theta}_t) - \mathcal{L}(\boldsymbol{\theta}_{t+1}) &\geq ||\eth_t||_2^4 \frac{(\beta_1 - \beta_1^2)^2 \phi_1^2 \phi_2^2}{2L(\xi + G)^2 G^2 (1 - \beta_1^t)^2} \\
&\iff \mathcal{L}(\boldsymbol{\theta}_2) - \mathcal{L}(\boldsymbol{\theta}_{T+1}) \geq ||\eth_t||_2^4 \sum_{t=2}^{t} \frac{(\beta_1 - \beta_1^2)^2 \phi_1^2 \phi_2^2}{2L(\xi + G)^2 G^2} \\
&\iff \min_{t=2,...,T} ||\eth_t||_2^4 \leq \frac{2L(\xi + G)^2 G^2}{(\beta_1 - \beta_1^2)^2 \phi_1^2 \phi_2^2} (\mathcal{L}(\boldsymbol{\theta}_2) - \mathcal{L}(\boldsymbol{\theta}_*)) \\
&\equiv \min_{t=2,...,T} ||\nabla \mathcal{L}(\boldsymbol{\theta}_t)||_2^4 \leq \frac{2L(\xi + G)^2 G^2}{(\beta_1 - \beta_1^2)^2 \phi_1^2 \phi_2^2} (\mathcal{L}(\boldsymbol{\theta}_2) - \mathcal{L}(\boldsymbol{\theta}_*))
\end{aligned}$$

Notice that this immediately entails a minimum T wherein:

Corollary 7. *(Adam Gradient Norm Intermediate Convergence Rate) Picking $T \geq \frac{2L(\xi+G)^2 G^2}{\epsilon^4 (\beta_1 - \beta_1^t)^2 \phi_1^2 \phi_2^2}$ ensures that for at least one $||\nabla \mathcal{L}(\boldsymbol{\theta}_t)||_2$ from $t = 2, ..., T$:*

$$min_{t=2,...,T} ||\nabla \mathcal{L}(\boldsymbol{\theta}_t)||_2 \leq \epsilon$$

*which indicates that under certain assumptions, Adam has **at-least** a $\frac{1}{\sqrt[4]{T}}$ convergence rate.*

Now, as per the hypothesis we chose $\beta_1 = \frac{\epsilon}{\epsilon + 2G}$ (which actually results from setting $\phi_1 = 1$), and $\xi = 2G$ (which actually results from setting $\phi_2 = G$). These choices result in the revised T bound[30]:

$$\begin{aligned}
T &\geq \frac{18LG^4}{2\epsilon^4 (\frac{2G\epsilon}{\epsilon+2G})^2 G^2} (\mathcal{L}(\boldsymbol{\theta}_2) - \mathcal{L}(\boldsymbol{\theta}_*)) \\
&\geq \frac{18L}{8\epsilon^6 (\frac{1}{\epsilon+2G})^2} (\mathcal{L}(\boldsymbol{\theta}_2) - \mathcal{L}(\boldsymbol{\theta}_*)) \\
&\geq \frac{9L}{\epsilon^6} (\mathcal{L}(\boldsymbol{\theta}_2) - \mathcal{L}(\boldsymbol{\theta}_*))
\end{aligned}$$

where the final bound $\frac{9L}{\epsilon^6}(\mathcal{L}(\boldsymbol{\theta}_2) - \mathcal{L}(\boldsymbol{\theta}_*))$ is derived from $\frac{18L}{8\epsilon^6 (\frac{1}{\epsilon+2G})^2}(\mathcal{L}(\boldsymbol{\theta}_2) - \mathcal{L}(\boldsymbol{\theta}_*))$ by making the simplifying assumption that $\epsilon << 2G$ and thus treats $\frac{1}{\epsilon+2G} \approx \frac{1}{2G}$ (asymptotically speaking).

[30] The revision of T here is required to maintain the bound $\min_{t=1,...,T} ||\nabla \mathcal{L}(\boldsymbol{\theta}_t)||_2 \leq \epsilon$, i.e. ϵ-criticality.

Separately, we observe the effect of these choices ($\beta_1 = \frac{\epsilon}{\epsilon+2G}, \xi = 2G$) on the original $\eta_t := \frac{1}{2}\frac{\langle \eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t \rangle}{\frac{L}{2}||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t||_2^2}$:

$$\begin{aligned}
\eta_t &:= \frac{1}{2}\frac{\langle \eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t \rangle}{\frac{L}{2}||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t||_2^2} \\
&= \frac{1}{L}\frac{\langle \eth_t, (\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t \rangle}{||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t||_2^2} \\
&= \frac{1}{L}\frac{Q_t}{||(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t||_2^2} \\
&\geq \frac{1}{L}\frac{||\eth_t||_2^2 (\frac{G^2(\beta_1 - \beta_1^2)}{\xi G(\xi+G)})}{(\frac{(1-\beta_1^t)G}{\xi})^2} \\
&= \frac{||\eth_t||_2^2}{L(1-\beta_1^t)^2}\frac{4\epsilon}{3(\epsilon+2G)^2}
\end{aligned}$$

Take note that there is an inequality within the derivation above, $\eta_t \geq \frac{1}{L}\frac{||\eth_t||_2^2 (\frac{G^2(\beta_1 - \beta_1^2)}{\xi G(\xi+G)})}{(\frac{(1-\beta_1^t)G}{\xi})^2}$. This represents the "final" value of η_t (i.e. the smallest value it can decay to).

Together, the bounds $T \geq \frac{9L}{\epsilon^6}(\mathcal{L}(\boldsymbol{\theta}_2) - \mathcal{L}(\boldsymbol{\theta}_*))$ and $\eta_t \geq \frac{1}{L}\frac{||\eth_t||_2^2 (\frac{G^2(\beta_1 - \beta_1^2)}{\xi G(\xi+G)})}{(\frac{(1-\beta_1^t)G}{\xi})^2} = \frac{||\eth_t||_2^2}{L(1-\beta_1^t)^2}\frac{4\epsilon}{3(\epsilon+2G)^2}$ result in the convergence:

Corollary 8. *(Adam Gradient Norm Convergence Rate) Picking* $\eta \geq \frac{||\eth_t||_2^2}{L(1-\beta_1^t)^2}\frac{4\epsilon}{3(\epsilon+2G)^2}$ *and* $T \geq \frac{9LG^2}{\epsilon^6}[\mathcal{L}(\boldsymbol{\theta}_2) - \mathcal{L}(\boldsymbol{\theta}_*)]$ *ensures that within* T *iterations, at least one* $t = 1...T$ *is* ϵ*-critical such that:*

$$||\nabla \mathcal{L}(\boldsymbol{\theta}_t)||_2 \leq \epsilon$$

which indicates that under certain assumptions, Adam has ***at-least*** *a* $\frac{1}{\sqrt[6]{T}}$ *convergence rate.*

Remark 5. Observe that the professed rate, $\frac{1}{\sqrt[6]{T}}$, is much poorer compared to the $\frac{1}{T^2}$ rate of *NAG*, $\frac{1}{T}$ rate of fixed-η Gradient Descent, $\frac{1}{\sqrt{T}}$ rate of *SGD*, and the $\frac{1}{\sqrt[4]{T}}$ rate of *RMSProp*.

Indeed, similar to *RMSProp*, the convergence proved above is mainly a theoretical exercise in guaranteeing that *Adam* can find an approximately critical point in finite time at all. Empirically, as will be demonstrated in the next section, *Adam* may, under specific hyperparameters, actually converge just as quickly as the other variants of Gradient Descent.

As such, *Corollary 7* was rather meant to square away the heightened complexity of the *Adam* method (with the combined use of momentum parameter β_1 and the enhanced update rule involving $(\boldsymbol{W}_t^{\frac{1}{2}} + \text{diag}(\xi 1_d))^{-1} v_t$. Proving exact convergence even for deterministic *Adam* is, unsurprisingly, not a trivial exercise.

□

7 Empirical Analysis

So far, we've established important theoretical bounds for several variations of the *Gradient Descent* algorithm (called Optimizers). While in isolation each of these theoretical bounds are verifiable through proofs, they are difficult to compare directly due to differences in their objectives as well as any associated constraints. Let's collate these analytical differences for clarity (*Table 1*):

Table 1: Comparison of Analytical Frameworks for Optimizers

Feature	**Fixed-η GD***	**Line-Search GD**	**SGD**	**NAG**	**RMSProp**	**Adam**
Objective Considered	$f(x)$	$f(x)$	$\mathcal{L}(\boldsymbol{\theta})$	$\mathcal{L}(\boldsymbol{\theta})$	$\mathcal{L}(\boldsymbol{\theta})$	$\mathcal{L}(\boldsymbol{\theta})$
Stochasticity	☒	☒	✓	☒	✓	☒
Bound Given For**	Final Iterate	Final Iterate	Final Iterate	Final Iterate	Grad. Norm	Grad. Norm
Convergence Rate	$\frac{1}{T}$	$\frac{1}{T}$	$\frac{1}{\sqrt{T}}$	$\frac{1}{T^2}$	$\frac{1}{\sqrt[4]{T}}$	$\frac{1}{\sqrt[6]{T}}$
Assumed L-Smoothness	✓	✓	✓	✓	✓	✓
Assumed Max Grad. (G)	☒	☒	✓	☒	✓	✓
Assumed Convexity	✓	✓	✓	✓	☒	☒

**Recall from our analysis of Algorithm 1 that in fixed-descent, η is fixed at $\frac{1}{L}$. But keep in mind also from the section defining the Lipschitz-L that the best-we can often do is estimate L. Thus, the "fixed" in Fixed-η GD is a bit of a misnomer. For the actual estimation of L in computing η, we consulted Yedida and Saha, 2019.*
***Final Iterate refers to how well the algorithm approximates the actual minimizer at the end of T steps ($f(x_T) - f(x_*)$ or $\mathcal{L}(\boldsymbol{\theta}_T) - \mathcal{L}(\boldsymbol{\theta}_*)$), while Grad. Norm refers to how much the algorithm has minimized the gradient magnitude $||\nabla||$ itself (which, as we recall, is an indicator for how well the objective has been minimized).*

Indeed, aside from the differences already made explicit above, each optimizer is also dependent on different tunable hyperparameters (*Table 2*):

Table 2: Comparison of Key Hyperparmeters of Optimizers

Feature	**Fixed-η GD**	**Line-Search GD**	**SGD**	**NAG**	**RMSProp**	**Adam**
Step-size η	✓	✓	✓	✓	✓	✓
Armijio Stopping-Condition	☒	✓	☒	☒	☒	☒
Armijio Shrinkage-Factor	☒	✓	☒	☒	☒	☒
Momentum	☒	☒	☒	✓	✓	✓
Gradient Preconditioner	☒	☒	☒	☒	✓	✓

Further, it was also mentioned through remarks (especially for *Adam* and *RMSProp*) that some of the analyses presented were merely first-attempts at proving that these optimizers could converge at all, rather than definite bounds of any sort.

Finally, we had not even considered (for the sake of keeping our analyses tractable) an additional important hyperparameter for the Adaptive Gradient methods (*NAG*, *RMSProp* and *Adam*): **Weight Decay**, a regularization factor often applied on the weights $\boldsymbol{\theta}_t$ during the update step to prevent **overfitting**[31].

Considering these, we may find it prudent to do an empirical comparison of these optimizers[32].

7.1 Methodology

Our experiment involved, in essence, the testing of 5 optimizers by deploying them in optimizing an ML-model and evaluating the model performance against a predetermined objective. The 5 optimizers are:

- *SGD* with *Fixed-η GD*.
- *SGD* with *Backtracking Line Search*.

[31] For most practical purposes, we do not aspire to a total optimization of the objective considered. Instead, we want the optimizers we employ to learn the given loss landscape $\boldsymbol{X}$ approximately or imperfectly, so they may generalize well to an unseen loss landscape $\boldsymbol{X}'$. As such, a technique called *Weight Decay* is applied to dampen the complexity of a learned update $\boldsymbol{\theta}_{t+1} - \boldsymbol{\theta}_t$.
[32] Our code for this experiment is available here.

- *SGD* with *NAG*.
- *SGD* with *RMSProp*.
- *SGD* with *Adam*.

It is important to note that our experiment is not entirely representative of the nuances in *Tables 1, 2* above. We had to standardize, in some ways, how we test the optimizers:

- We uniformly represented the objective via the cumulative loss-function $\mathcal{L}$ instead of a simple polynomial f. Recall that we define $\mathcal{L}(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m} \ell(g_{\boldsymbol{\theta}}(\boldsymbol{x_i}), y_i)$. In this case we assign ℓ to be the **Cross-Entropy Loss**[33].
- All of the optimizers are tested under stochasticity (whereas stochasticity was separately analyzed as *SGD* in theory, and not uniformly assumed for all optimizers). This was a choice made for practical purposes, as mentioned in *Remark 1*.

7.1.1 Setup

As a disclaimer, we had borrowed some of the experiment setup from the original *Adam* paper, presented in Kingma and Ba, 2017. We had done so to pre-empt poor choices of possible objectives and hyperparameters, and skip to a reasonable baseline performance for all optimizers from which we continue optimizing via hyperparamter tuning. We indicate borrowed setup using †.

We deployed these optimizers via a typical `Pytorch`-supported *Neural Network/Multi-Layer Perceptron* model with the following specifications†:

- Domain of interest: MNIST.
- Objective being optimized for: Multinomial Image Classification (classifying 10 handwritten digits "0" through "9").
- 784 input features with 20% dropout probability.
- 1000 hidden features with 10% dropout probability.
- 10 output predictions.

7.1.2 Hyperparameter Tuning

Since our objective was to compare all optimizers, we want to ensure we are comparing them under their best conditions before we draw any conclusions about performance. As such, we conduct "tuning" of all optimizers; this involves finding the best specific hyperparameters under which each optimizer performs.

We identified 25 possible total configurations of 2 distinct sets of hyperparameters. Specifically,

- For the Adaptive Gradient methods *NAG, RMSProp* and *Adam*, we spun-up 25 possible combinations of the hyperparameters ① step-size η, and ② weight-decay.
- For *SGD* with *Backtracking Line-Search*, we spun-up 25 possible combinations of the ① Armijio stopping-condition α and ② the Armijio shrinkage-factor β.

Some intermediate notes:

- The rest of the hyperparameters for each optimizer (such as β_1, β_2 for *Adam* or the momentum-factor β for *NAG* and *RMSProp*) were fixed at recommended values readily available in popular literature.
- *SGD* with *Fixed-η*, in-particular, has no unique hyperparameters. As such, the only source of variance for this optimizer will be as described below with k-fold cross-validation.

[33] The *Cross-Entropy Loss* function is a function that measures the ability of a model to predict the "class" of an object correctly, where the object's class is drawn from an unknown distribution of probable classes (of which there may be 2 or more). Intuitively, the "loss" here arises from incorrect predictions/classifications made.

We tuned the optimizers over these 25 total configurations, using the following template:

- 5-fold cross-validation for each configuration where the folds are drawn with random seed 42[34].
- 500 epochs of tuning within each of the folds for each optimizer under each configuration.
- A mini-batch size of 128 image samples† for each of the epochs.

In the end, our tuning produced a repository like so:

```
Configs
├─ Config 1
│  ├─ Fold 1
│  │  ├─ Optimizer 1
│  │  ├─ Optimizer 2
│  │  ├─ ...
│  │  └─ Optimizer 5
│  ├─ Fold 2
│  ├─ ...
│  └─ Fold 5
├─ Config 2
├─ ...
└─ Config 25
```

where each `Config/Fold/Optimizer` folder contains the 500 final $\mathcal{L}$s (for 500 epochs) of each optimizer during tuning under that specific configuration, along with the performance validation per fold[35]. In total, we will have 125 histories of $\mathcal{L}$ for each optimizer, and thus a total of 625 performance histories over 5 optimizers to use for comparison.

We determine the best configuration (i.e. hyperparameter combination) for each optimizer by taking the best average performance over the optimizer's 5 folds over all 25 configurations. In particular, we identified the following best-configurations for all optimizers (*Table 3*):

Table 3: Optimizer Best Configurations (Averaged over 5 folds)

Optimizer	Best Config	Avg Accuracy (%)	Std Dev
SGD with Fixed-η	Config 20	98.758000	0.123167
SGD with BT Line-Search	Config 2	98.768000	0.118617
SGD with NAG	Config 20	98.756000	0.134090
SGD with RMSProp	Config 0	98.734000	0.083845
SGD with Adam	Config 0	98.750000	0.102713

For a peek at the averaged optimizer history over the 5 folds and through 500 epochs, check *Fig. 13* in Appendix A1.

7.1.3 Full Model Optimization

Having found the best conditions for each optimizer, we then do away with k-fold cross validation and optimize our model using the optimizers over the full dataset. In essence,

- We train the model over the 5 optimizers for 500 epochs each with the same mini-batch size of 128 samples, but over the full dataset instead of through 5 separate folds.
- We repeat this full-dataset training over 3 random seeds to account for variance in model-initialization.

[34] In general, k-fold cross-validation involves splitting a dataset into k equal-sized samples ("folds") and optimizing for an objective $k-1$ times: once each over $k-1$ folds, before using the kth fold to test the performance of the model being optimized. The purpose of such a technique is to account for the variance in data (quality, order, etc) when optimizing. Additionally, we also use this technique to account for variance in model initialization (for e.g. in the initialization of $\boldsymbol{\theta}$) by setting k different random model seeds for each of the folds. In this case, we do something slightly different by optimizing the model using each optimizer over 5 folds for 1 configuration, and testing the performance for all 5 folds using different subsets of the fold for optimization versus testing.

[35] Here, performance is evaluated using % accuracy. For example, if there are 100 testing samples and the Neural Network with a specific optimizer has predicted 8 of them, the performance of that optimizer is recorded as 80%.

This time, the repository produced looked like this:

```
Final Test
├── Optimizer 1
│   ├── Seed 1
│   ├── Seed 2
│   └── Seed 3
├── ...
└── Optimizer 5
```

wherein the model is optimized over and evaluated for 3 repetitions with each optimizer, and each `Optimizer/Seed` directory contains the full trajectory of optimization and the final performance score, for that run.

For a peek at the averaged optimizer history over the 3 seeds and through 500 epochs, check *Fig. 14* in Appendix A1.

7.1.4 Final Evaluation

Finally, having optimized and evaluated our *Neural Network* over the full dataset 3 times with each optimizer, we measure the average performance of each optimizer to glean their best average accuracies.

In particular, we computed the following optimizer performances (*Table 4*):

Table 4: Final Optimizer Performance (Averaged over 3 seeds)

Optimizer	Avg Accuracy (%)	Std Dev
SGD with Fixed-η	98.856667	0.055578
SGD with BT Line-Search	98.916667	0.038586
SGD with NAG	98.910000	0.021602
SGD with RMSProp	98.850000	0.028284
SGD with Adam	98.936667	0.037712

7.2 Results

7.2.1 Empirical vs Theoretical Convergence

To wraparound to the point of this exercise, let us distill the essential points of interest with regards to how the experiment aligned or differed from the theory in terms of algorithmic motivation, and convergence rates. Observe *Figs. 13 and 14*.

Differences:

- Unlike our highly conservative theoretical convergence rates for *Adam* and *RMSProp*, it appears that their convergence during both tuning and testing was not at all much slower compared to the other optimizers. In-fact, *Adam* seemed to converge the fastest over iterations compared to the other optimizers while *RMSProp* seemed to converge the fastest over wall-clock time (flatlining sooner than the rest).
- Similarly, it appears that *SGD BT Line-Search*, *SGD Fixed-η*, and especially *NAG* did not necessarily converge much faster overall than the above 2 despite being simpler algorithms with more promising convergence rates[36].
- *SGD BT Line-Search* in-particular seemed to have fared quite poorly in convergence despite promising theoretical analysis. This may, we suspect, be attributable to a poor choice of hyperparameters. That is, this optimizer scales η depending on some manually defined stopping-criteria wherein the stopping-condition may not be suitably defined compared to the actual loss landscape.

Alignment:

- Interestingly, it appears that *NAG*'s convergence rate once the optimizer started getting closer to the loss minimizer is accelerated compared to its convergence rate at the start. Observe how in both figures, *NAG* seems to converge poorly per-iteration compared to *Adam* before rapidly accelerating to match-up to *Adam* in the end. In a sense, as *Fig. 11* suggests, *NAG*'s lookahead principle may have become a competitive advantage only towards the end, near the loss minimizer.

[36] Of course, one might fairly question whether this might be just due to the added stochasticity that was not assumed during theoretical analysis for these 3 optimizers. Still, even with stochasticity the difference in bounds between all of these algorithms was likely to be in several orders of magnitude; $\frac{1}{\sqrt[4]{T}}/\frac{1}{\sqrt[6]{T}}$ for *RMSProp*/*Adam*, versus $\frac{1}{\sqrt{T}}$ for the other 3 optimizers.

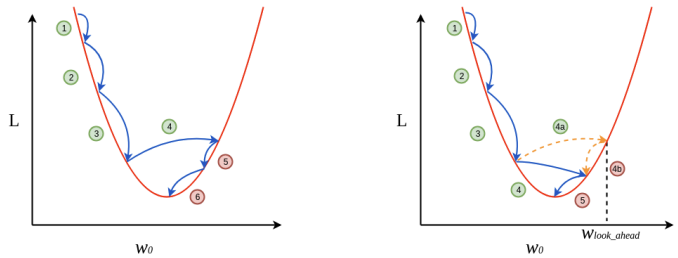


Figure 11: NAG's acceleration

- Similarly, it appears that while *SGD Fixed-η* pays a heavier price at the start, converging much slower per-iteration compared to both *NAG* and *Adam*, it also more-or-less matches-up at the ending. This suggests that while the estimation of L by the optimizer via the spectral norm was quite pessimistic at the start, it got more accurate approaching the minimizer.

7.2.2 Is Convergence Everything?

Notice that *Table 4* paints essentially the following picture:

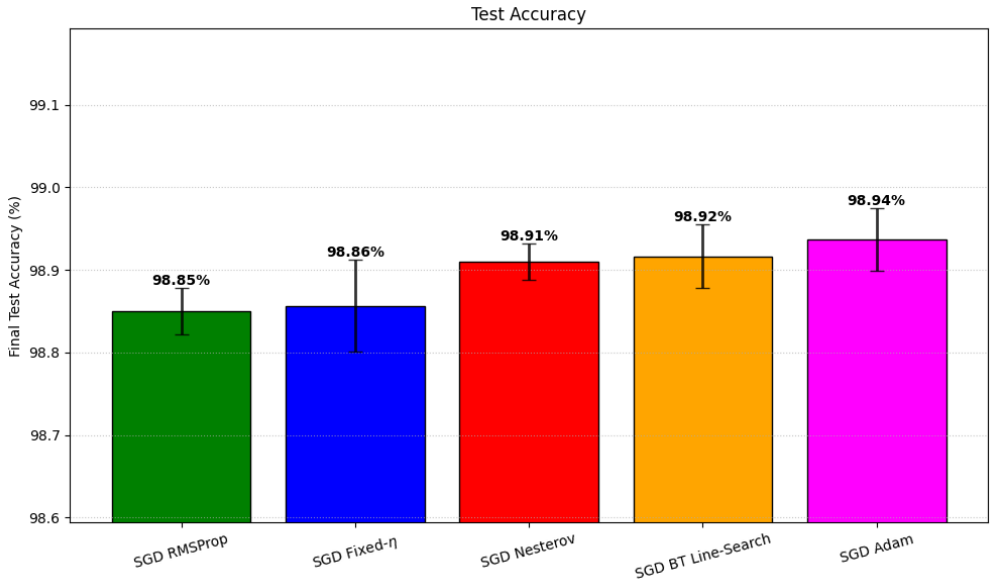


Figure 12: Final Optimizer Performance (Averaged over 3 seeds)

There are two immediately notable observations to be made here:

- For both hyperparameter tuning and full optimization, *SGD BT Line-Search* and *RMSProp* seemed to converge quite poorly relative to the other optimizers (*Figs. 13, 14* in Appendix A1). However after final testing, these optimizers are not only highly competitive (with only marginal differences in accuracy) with the rest, but *SGD BT Line-Search* in-particular (which converged the poorest) surpasses many other optimizers ranking only behind *Adam* (which converged the best).
- In a similar vein, *SGD Fixed-η* seems equally competitive without even having any tunable hyperparameters (other than η) at all.

Both these observations point to the same wisdom, already well-known in the ML-context: that better convergence does not necessarily generalize to better performance on unseen data. For instance, the more adaptive optimizers *Adam* and *NAG* converged better than the simpler *SGD Fixed-η* and *SGD BT Line-Search* (*Fig. 14*), suggesting that they adapted much better to the loss landscape given during initial optimization. However, the former did not necessarily perform proportionally better than the latter on an unseen landscape[37]. It seems then, that some imperfect convergence (or "learning", in the

[37]Strangely, *RMSProp* seemed to have performed poorly both in convergence and in testing despite being a more sophisticated Adaptive Gradient method. While this may be attributable to poor choice of hyperparameters, it is interesting to see that *RMSProp* still converged the fastest in wall-clock time among all optimizers; flatlining much earlier than the rest (*Fig. 14*). This may be a useful trade-off in that the optimizer converges much faster, and the eventual accuracy of a model using this optimizer is only 0.09% worse compared to the best optimizer

ML-context) may be a boon rather than bane after all.

7.2.3 Durability to Hyperparameter Variance

Aside from convergence and generalizability, we may also be interested in how sensitive the optimizers were to choices of hyperparameters, which is not really reflected in the averaged training history or final performance accuracy discussed so far. For this, we turn to *Fig. 15* in Appendix A1 which summarizes optimizer sensitivity to hyperparameter variation during the 25-configuration tuning, and which prompts the following takeaways

- As supported by Basu et al., 2018, the Adaptive Gradient methods, *Adam* and *RMSProp*, were the least sensitive to hyperparameters, with *NAG* not trailing too far behind.
- Meanwhile, *SGD BT Line-Search* seemed most sensitive, which is supported by the observation already-made in the section above that *SGD BT* is programmed by a choice of α, β that seems cruder compared to the how hyperparameters for other optimizers were chosen.
- And of-course, *SGD Fixed-η* was excluded here due to its plain lack of tunable hyperparameters with the only variance emerging from the estimation of L, the k-folds, and the random seeds[38].

[38] And this variance is still quantified in *Tables 3, 4* using the "Std Dev" column.

8 Future Work

There are still many avenues of development available for this project.

Theoretical Analysis:

- One could analyze theoretical bounds (whether consecutive or final iterate proximity bounds, or gradient magnitude) for the stochastic versions of *NAG* and *Adam*.
- One could analyze consecutive and final iterate proximity bounds for *RMSProp* and *Adam* to expand beyond the current gradient magnitude bounds.
- One could explore the final iterate proximity bound of *SGD* and *NAG* without convexity assumed.
- One could explore the bounds of all optimizers without L-smoothness assumed.
- One could explore the bounds of all optimizers with strong-convexity assumed.
- One could explore other popular optimizers such as **AdaGrad**, and **AdaDelta**.

Experimental Analysis:

- One could benchmark the 5 optimizers in the experiment above against other datasets than MNIST such as CIFAR-10 or against other objectives entirely such as sentiment classification on Stanford Large Movie Reviews or linear regression on California Housing Prices.
- A different, or more robust, set of hyperparameters could be tested to further refine the comparison between optimizers.
- As with the theoretical analysis, perhaps more variations of the same optimizers could be tested; primarily testing both stochastic and non-stochastic variants.

9 Appendix: A1

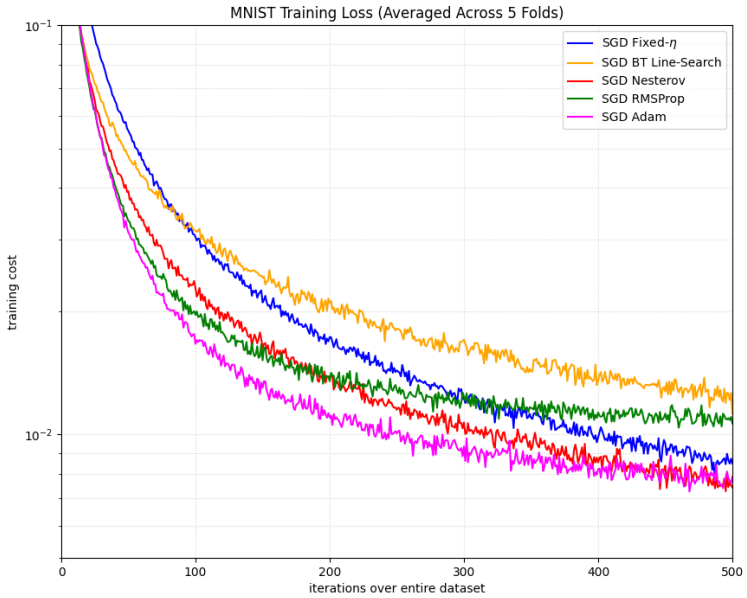


Figure 13: Averaged Optimizer Histories in Initial Tuning.

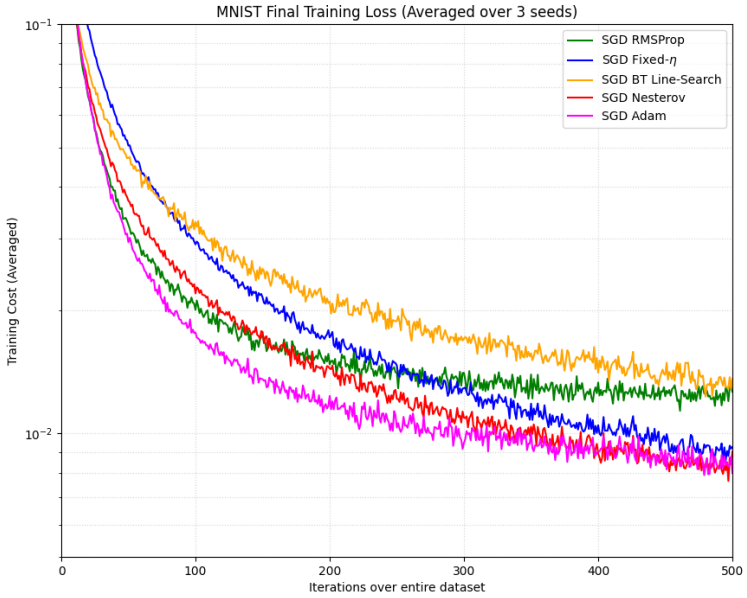
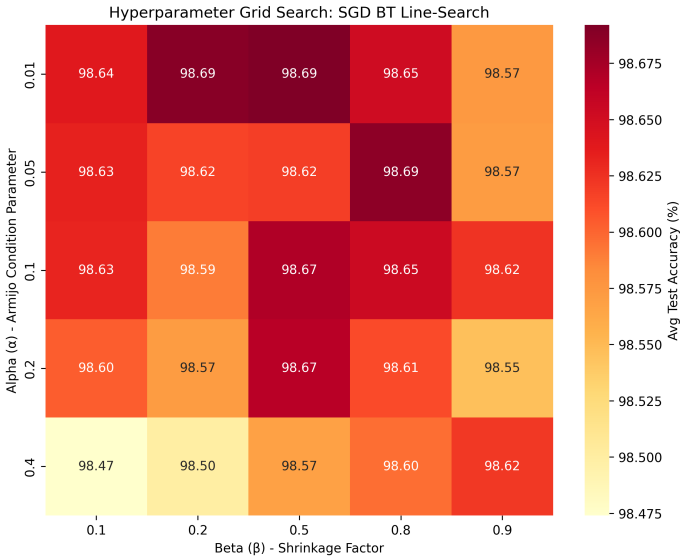
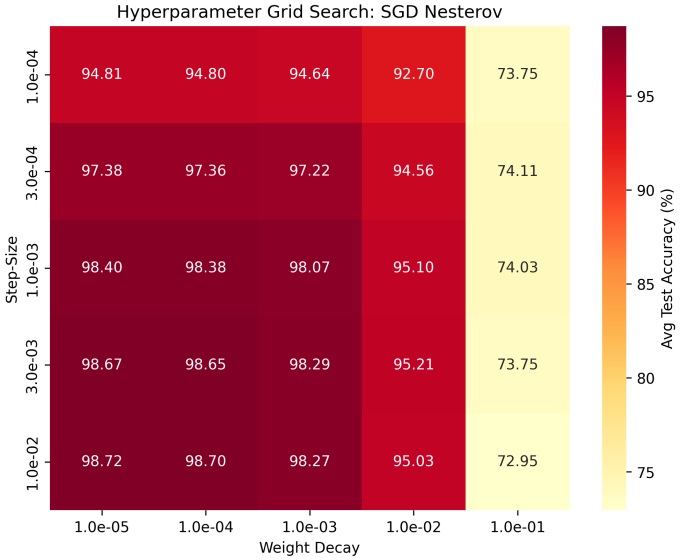


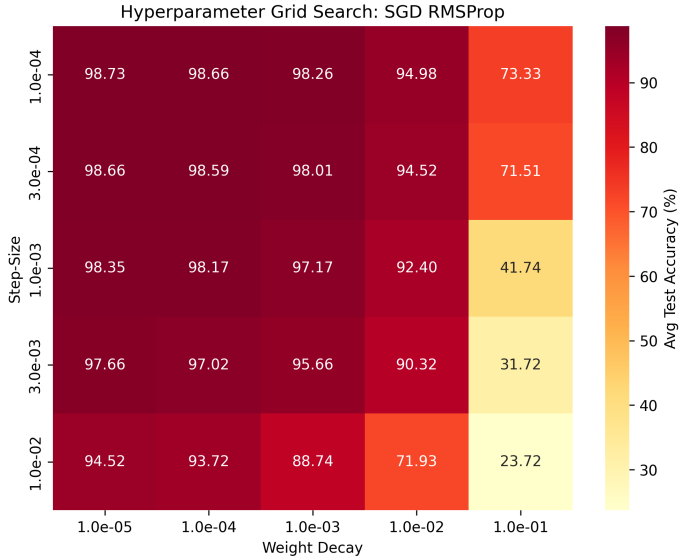
Figure 14: Averaged Optimizer Histories in Final Testing.



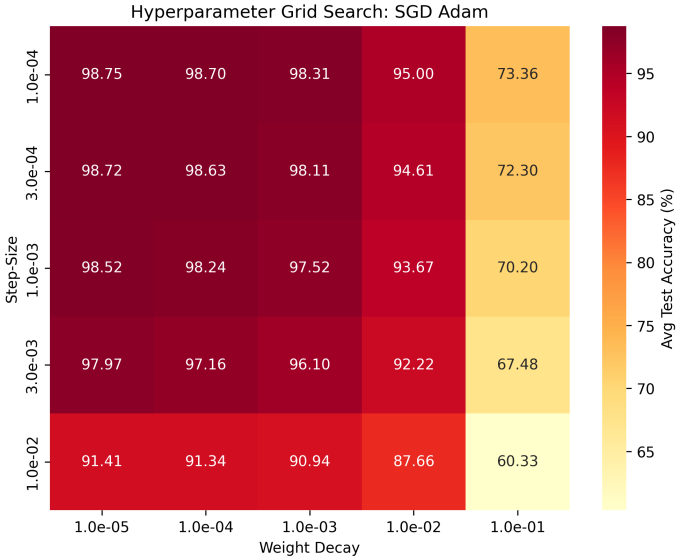
(a) SGD BT Line-Search Heatmap



(b) SGD with NAG Heatmap



(c) SGD with RMSProp Heatmap

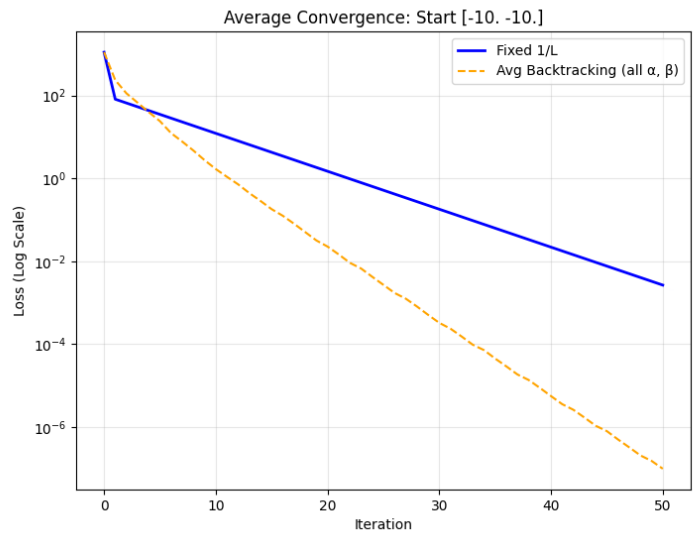
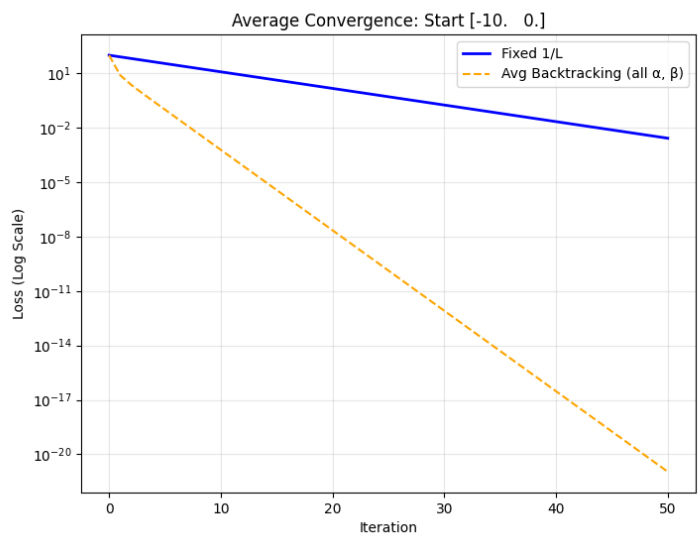


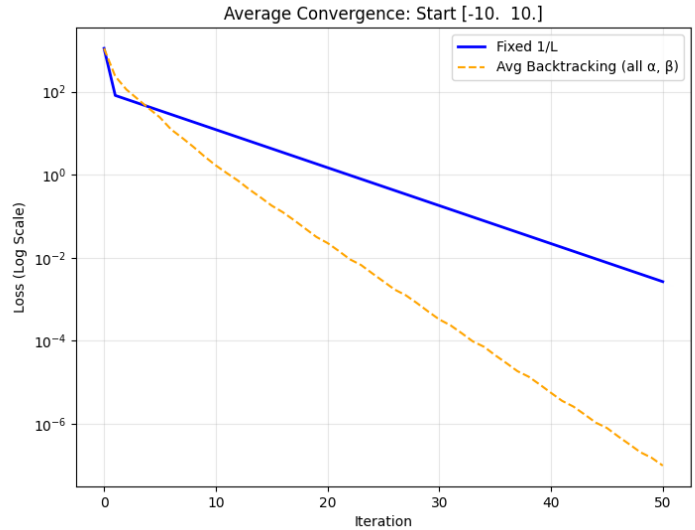
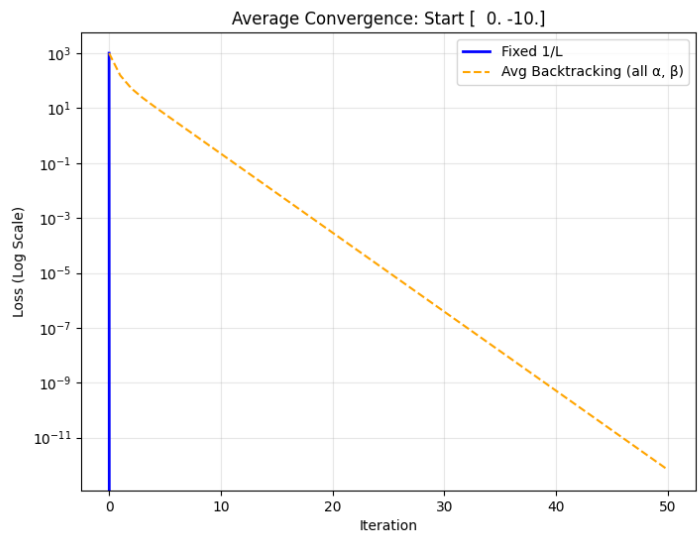
(d) SGD with Adam Heatmap

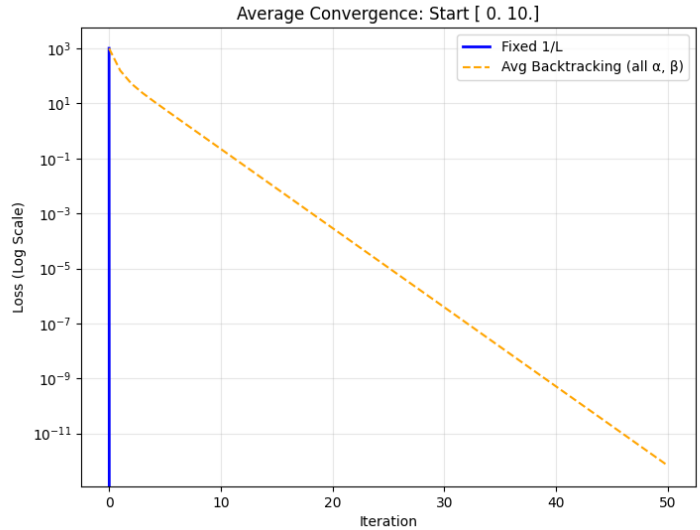
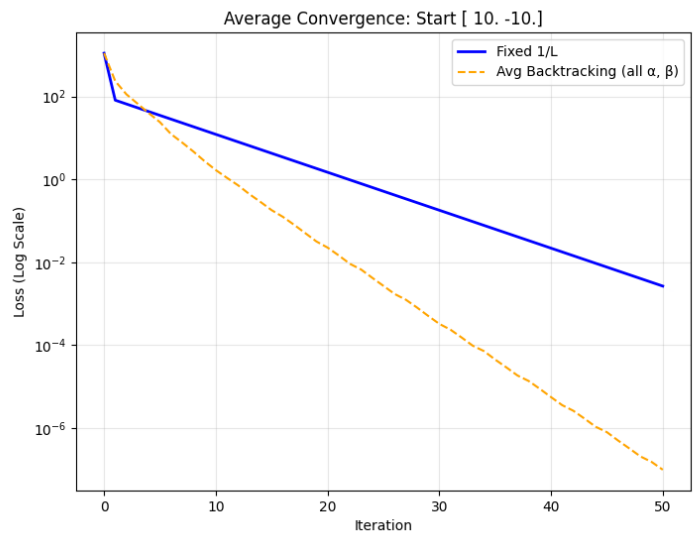
Figure 15: Heatmap Comparison of Optimizer Sensitivty to Hyperparameters during Tuning.

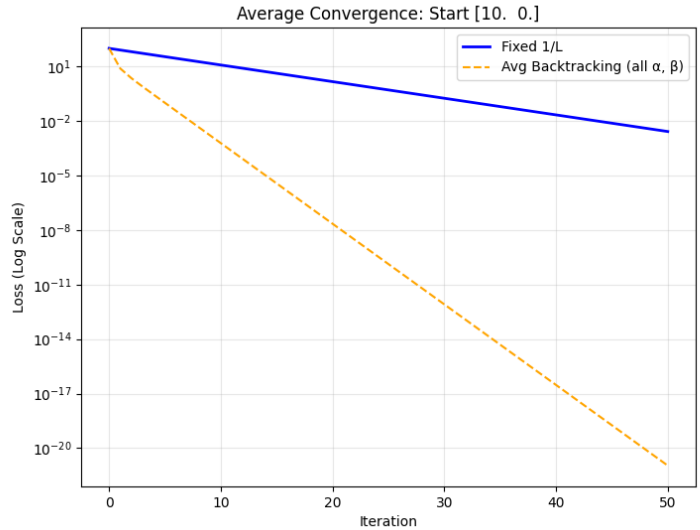
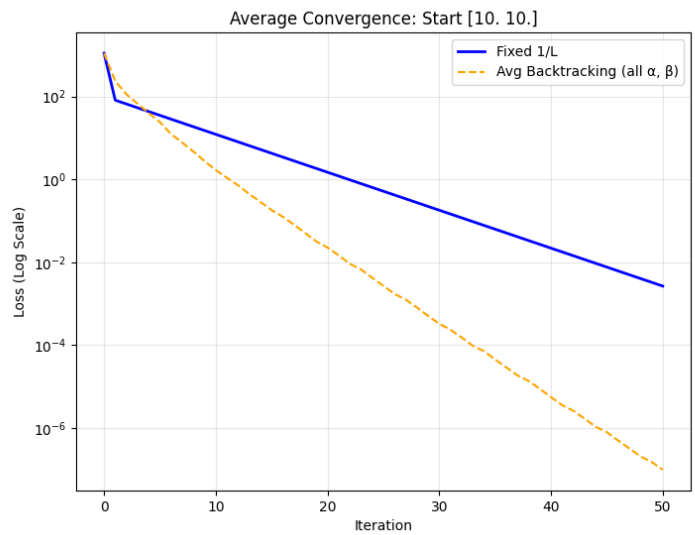
10 Appendix: A2

Table 5: Full Grid Search Results: Fixed-Step vs. Backtracking

Start Point (x, y)	α	β	**Final Loss (Convergence)** **Fixed**	**Backtrack**	**Average Convergence**
$[-10, -10]$	0.1	0.5	2.66e-03	7.17e-11	
$[-10, -10]$	0.1	0.8	2.66e-03	3.25e-07	
$[-10, -10]$	0.3	0.5	2.66e-03	6.64e-11	
$[-10, -10]$	0.3	0.8	2.66e-03	2.51e-07	
$[-10, -10]$	0.5	0.5	2.66e-03	4.38e-11	
$[-10, -10]$	0.5	0.8	2.66e-03	1.61e-08	
$[-10, 0]$	0.1	0.5	2.66e-03	0.00e+00	
$[-10, 0]$	0.1	0.8	2.66e-03	6.53e-21	
$[-10, 0]$	0.3	0.5	2.66e-03	0.00e+00	
$[-10, 0]$	0.3	0.8	2.66e-03	5.20e-54	
$[-10, 0]$	0.5	0.5	2.66e-03	0.00e+00	
$[-10, 0]$	0.5	0.8	2.66e-03	5.25e-73	

Start Point	α	β	Final Loss		Average Convergence
					
$[-10, 10]$	0.1	0.5	2.66e-03	7.17e-11	
$[-10, 10]$	0.1	0.8	2.66e-03	3.25e-07	
$[-10, 10]$	0.3	0.5	2.66e-03	6.64e-11	
$[-10, 10]$	0.3	0.8	2.66e-03	2.51e-07	
$[-10, 10]$	0.5	0.5	2.66e-03	4.38e-11	
$[-10, 10]$	0.5	0.8	2.66e-03	1.61e-08	
					
$[0, -10]$	0.1	0.5	0.00e+00	6.22e-58	
$[0, -10]$	0.1	0.8	0.00e+00	4.09e-12	
$[0, -10]$	0.3	0.5	0.00e+00	6.22e-58	
$[0, -10]$	0.3	0.8	0.00e+00	2.15e-40	
$[0, -10]$	0.5	0.5	0.00e+00	2.53e-40	
$[0, -10]$	0.5	0.8	0.00e+00	1.15e-89	

Start Point	α	β	Final Loss		Average Convergence
					
$[0, 10]$	0.1	0.5	0.00e+00	6.22e-58	
$[0, 10]$	0.1	0.8	0.00e+00	4.09e-12	
$[0, 10]$	0.3	0.5	0.00e+00	6.22e-58	
$[0, 10]$	0.3	0.8	0.00e+00	2.15e-40	
$[0, 10]$	0.5	0.5	0.00e+00	2.53e-40	
$[0, 10]$	0.5	0.8	0.00e+00	1.15e-89	
					
$[10, -10]$	0.1	0.5	2.66e-03	7.17e-11	
$[10, -10]$	0.1	0.8	2.66e-03	3.25e-07	
$[10, -10]$	0.3	0.5	2.66e-03	6.64e-11	
$[10, -10]$	0.3	0.8	2.66e-03	2.51e-07	
$[10, -10]$	0.5	0.5	2.66e-03	4.38e-11	
$[10, -10]$	0.5	0.8	2.66e-03	1.61e-08	

Start Point	α	β	Final Loss		Average Convergence
					
$[10, 0]$	0.1	0.5	2.66e-03	0.00e+00	
$[10, 0]$	0.1	0.8	2.66e-03	6.53e-21	
$[10, 0]$	0.3	0.5	2.66e-03	0.00e+00	
$[10, 0]$	0.3	0.8	2.66e-03	5.20e-54	
$[10, 0]$	0.5	0.5	2.66e-03	0.00e+00	
$[10, 0]$	0.5	0.8	2.66e-03	5.25e-73	
					
$[10, 10]$	0.1	0.5	2.66e-03	7.17e-11	
$[10, 10]$	0.1	0.8	2.66e-03	3.25e-07	
$[10, 10]$	0.3	0.5	2.66e-03	6.64e-11	
$[10, 10]$	0.3	0.8	2.66e-03	2.51e-07	
$[10, 10]$	0.5	0.5	2.66e-03	4.38e-11	
$[10, 10]$	0.5	0.8	2.66e-03	1.61e-08	

11 References

Amitabh Basu, Soham De, Anirbit Mukherjee, and Enayat Ullah. Convergence guarantees for rmsprop and ADAM in non-convex optimization and their comparison to nesterov acceleration on autoencoders. *CoRR*, abs/1807.06766, 2018. URL http://arxiv.org/abs/1807.06766.

Gabriele Farina. Lecture 10: Stochastic gradient descent. Lecture notes for MIT 6.7220/15.084 Nonlinear Optimization, April 2024. URL https://www.mit.edu/~gfarina/2025/67220s25_L15_sgd/L15.pdf. Revised April 2, 2024.

Gabriele Farina. Lecture 12: Gradient descent and descent lemmas. Lecture notes for MIT 6.7220/15.084 Nonlinear Optimization, April 2025. URL https://www.mit.edu/~gfarina/2025/67220s25_L12_gradient_descent/L12.pdf. Revised April 1, 2025.

Rie Johnson and Tong Zhang. Accelerating stochastic gradient descent using predictive variance reduction. In C.J. Burges, L. Bottou, M. Welling, Z. Ghahramani, and K.Q. Weinberger, editors, *Advances in Neural Information Processing Systems*, volume 26. Curran Associates, Inc., 2013. URL https://proceedings.neurips.cc/paper_files/paper/2013/file/ac1dd209cbcc5e5d1c6e28598e8cbbe8-Paper.pdf.

Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017. URL https://arxiv.org/abs/1412.6980.

Claude Lemaréchal. Cauchy and the gradient method. *Documenta Mathematica*, Extra Volume ISMP:251–254, 2012. URL https://www.yumpu.com/en/document/read/17213274/cauchy-and-the-gradient-method/1.

Jun Liu. A concise lyapunov analysis of nesterov's accelerated gradient method, 2025. URL https://arxiv.org/abs/2502.17373.

Ryan Tibshirani. Lecture 5: Gradient descent. Lecture notes for 10-725/36-725: Convex Optimization, Spring 2015. URL https://www.stat.cmu.edu/%7Eryantibs/convexopt-S15/scribes/05-grad-descent-scribed.pdf. Scribes: Loc Do.

Rahul Yedida and Snehanshu Saha. A novel adaptive learning rate scheduler for deep neural networks. *CoRR*, abs/1902.07399, 2019. URL http://arxiv.org/abs/1902.07399.